**Universidad de San Carlos de Guatemala**
**Centro Universitario de Occidente Sistemas de Bases de datos 1**
**Sección A**
**Ing. Pedro Domingo**

# Proyecto 2:
# Fase 2

**#1**
**Melanie del Rosario Tzul Baquiax 201930171**
**6 de Abril de 2026**

## Introducción

El presente proyecto tiene como finalidad diseñar un sistema de base de datos relacional que integre indicadores sociales y sanitarios de Guatemala, específicamente enfocados en dos ejes fundamentales: la violencia y la salud pública.

Guatemala presenta una realidad compleja en la que factores sociales, económicos y culturales inciden directamente tanto en la incidencia de violencia como en los problemas de salud. Por ello, la integración de estos datos en un sistema estructurado permite analizar relaciones significativas entre ambas dimensiones, facilitando la toma de decisiones basadas en evidencia.

El sistema propuesto permitirá almacenar, organizar y consultar información proveniente de diversas fuentes oficiales, garantizando integridad, consistencia y escalabilidad mediante un modelo normalizado en Tercera Forma Normal (3FN).

**Objetivo general:**

Diseñar e implementar una base de datos relacional completamente normalizada que integre indicadores de violencia y salud pública en Guatemala, permitiendo realizar análisis complejos y consultas cruzadas entre ambos dominios.

**Objetivos específicos:**

- Modelar entidades relacionadas con violencia, salud y factores socioeconómicos.
- Integrar múltiples fuentes de datos en un sistema unificado.
- Garantizar la normalización de la base de datos en 3FN.
- Implementar relaciones que permitan análisis multidimensional.
- Preparar el sistema para consultas analíticas complejas.
- Aplicar principios de álgebra relacional en consultas futuras.

# Contexto y Problemática

Guatemala enfrenta una realidad compleja en la que la violencia (delictiva, intrafamiliar, estructural, contra la mujer y la niñez) se entrelaza con indicadores de salud pública (desnutrición, enfermedades crónicas, enfermedades transmitidas por vectores y morbilidad materno-infantil).

El sistema a modelar debe permitir relacionar AMBOS ejes (violencia y salud) a nivel departamental y municipal, integrando además factores transversales como escolaridad, idioma, etnia, condición laboral y composición del hogar.

## Identificación de entidades principales

Se identificaron tres dominios principales:

1. **Geográfico**
    - País
    - Departamento
    - Municipio
2. **Social**
    - Persona
    - Nivel educativo
    - Empleo
    - Etnia, idioma, sexo
3. **Violencia**
    - Denuncia
    - Hecho delictivo
    - Violencia específica (mujer, infantil, intrafamiliar, estructural)
    - Sentencias
4. **Salud**
    - Indicadores agregados
    - Grupo etario
    - Enfermedades

## Relaciones clave

- Persona ↔ Denuncia (con roles)
- Denuncia ↔ Violencia específica
- Municipio ↔ Todos los hechos
- Salud ↔ Municipio + Año + Grupo etario

## Decisiones de diseño

- Uso de tablas intermedias para evitar redundancia.
- Separación entre datos transaccionales y agregados.
- Uso de catálogos para mantener la integridad.
- Modelo orientado a análisis (tipo data warehouse parcial).

## Normalización

- 1FN: atributos atómicos
- 2FN: eliminación de dependencias parciales
- 3FN: eliminación de dependencias transitivas

OBJETIVO DEL MAPEO

Diseñar una base de datos relacional, normalizada hasta 3FN, que integre:

- Eje 1: Violencia (general, contra la mujer, infantil, intrafamiliar, estructural, faltas judiciales, sentencias, embarazos, trabajo infantil)
- Eje 2: Salud pública (desnutrición, enfermedades crónicas CIE-10, enfermedades transmitidas por vectores, morbilidad materno-infantil)
- Factores transversales (persona, hogar, educación, empleo, idioma, etnia)

# DESCRIPCIÓN DE TABLAS:

SUBSISTEMA GEOGRÁFICO

Guatemala se divide en 8 regiones, 22 departamentos y 340 municipios.

Los registros de violencia y salud se reportan a nivel municipal, por lo que este es el nivel mínimo de desagregación geográfica.

Entidades:

- Pais : catálogo país (Guatemala como referencia principal)
- Region : agrupación política de departamentos
- Departamento : 22 departamentos de Guatemala
- Municipio : 340 municipios con población estimada y área urbana/rural
- Area_Geografica : catálogo urbana/rural/periurbana

Decisión: NO se baja a nivel de aldea porque los datasets del INE no llegan a ese detalle; sólo algunos registros transaccionales (domicilios) usan aldeas.

SUBSISTEMA DE PERSONA (entidad central)

La persona es el núcleo del modelo. Una persona puede:

- Ser víctima de violencia (en distintos ejes)
- Ser agresor/victimario
- Ser denunciante, testigo o sentenciado
- Ser sujeto de indicadores de salud (aunque la mayoría de registros de salud son agregados a nivel municipal/grupo etario)
- Ser trabajador infantil, adolescente embarazada, víctima de discriminación
- Hablar uno o varios idiomas
- Pertenecer a una etnia, tener un estado civil, un nivel de escolaridad
- Tener uno o varios domicilios (histórico)
- Tener uno o varios empleos (histórico)
- Pertenecer a un hogar

Entidades:

- Persona : datos básicos (nombre, apellido, fecha nac, sexo, etnia...)
- Sexo, Etnia, Idioma, Estado_Civil, Religion, Discapacidad : catálogos
- Persona_Idioma : relación N:M (una persona habla varios idiomas)
- Domicilio : históricos; un domicilio vigente (es_actual = 1)
- Contacto : teléfonos, correos
- Nivel_Educativo : catálogo (ninguno, preprimaria, primaria, básico, diversificado, universitario)
- Escolaridad_Persona: historial educativo
- Institucion_Educativa : catálogo
- Condicion_Laboral : ocupado, desocupado, inactivo, informal, formal
- Sector_Economico : CIIU (agricultura, industria, comercio, servicios, etc.)
- Ocupacion : CIUO
- Tipo_Empleo : asalariado, por cuenta propia, ayudante familiar, etc.
- Empleo_Persona : historial laboral
- Nivel_Socioeconomico : catálogo (estrato)
- Hogar : unidad habitacional, ingresos, número de personas
- Persona_Hogar : parentesco, jefatura
- Tipo_Vivienda : propia, alquilada, prestada, etc.

SUBSISTEMA DE DENUNCIAS (eje transversal a todos los tipos de violencia)

Toda forma de violencia registrada en el sistema parte de una denuncia o de un registro oficial. Una denuncia tiene un tipo (contra la mujer, VIF, discriminación,

falta judicial, etc.) y puede haber sido procesada o no.

Entidades:

- Eje_Violencia : catálogo (general, contra la mujer, infantil, intrafamiliar, estructural)
- Tipo_Denuncia : catálogo detallado por eje
- Institucion_Reportante : PNC, MP, PDH, Juzgados, etc.
- Estado_Denuncia : recibida, procesada, cerrada, archivada
- Denuncia : registro principal
- Persona_Denuncia : relación N:M con rol (denunciante, víctima, testigo, agresor)
- Seguimiento_Denuncia: cambios de estado con fechas

SUBSISTEMA DE HECHOS DELICTIVOS

Con base en los datasets del Observatorio MP y PNC:

Entidades:

- Clasificacion_Delito : grave, leve, gravísimo
- Tipo_Hecho_Delictivo : homicidio, asalto, robo, extorsión, secuestro, lesiones...
- Hecho_Delictivo : registro principal (fecha, municipio, lugar)
- Arma_Utilizada : catálogo (arma de fuego, blanca, contundente, ninguna)
- Hecho_Arma : N:M entre hechos y armas
- Movil_Delito : robo, venganza, sicariato, pasional, narcotráfico, etc.
- Hecho_Movil : N:M
- Victima : persona víctima + datos específicos (estado al momento)
- Agresor : persona agresora (puede ser desconocida)

- Necropsia : en caso de homicidio/muerte violenta
- Exhumacion : procedimiento forense posterior
- Forense : médico/perito que realizó el procedimiento
- Laboratorio_Forense : INACIF u otro

SUBSISTEMA DE SENTENCIAS

Datos del observatorio.mp.gob.gt/sentencias/. Se debe permitir que una sentencia general EXISTA SIN detalle de la persona sentenciada (opcional).

Entidades:

- Juzgado : catálogo
- Tipo_Delito_Juzgado : tipificación legal
- Sentencia : registro principal (fecha, tipo, años)
- Pena : catálogo (prisión, multa, conmutable, medidas sustitutivas)
- Persona_Sentenciada : detalle persona sentenciada (opcional 0..1)
- Detalle_Sentencia : información adicional detallada (opcional)

SUBSISTEMA DE VIOLENCIA CONTRA LA MUJER

Se diferencia por su especificidad (Ley contra el Femicidio).

Entidades:

- Tipo_Violencia_Mujer : física, psicológica, sexual, económica, patrimonial, femicidio
- Hecho_Violencia_Mujer: registro específico

- Tipo_Atencion : médica, psicológica, legal, albergue
- Centro_Atencion : catálogo
- Atencion_Mujer : atenciones brindadas a una víctima

SUBSISTEMA DE VIOLENCIA INTRAFAMILIAR

Entidades:

- Tipo_VIF : física, psicológica, sexual, patrimonial
- Relacion_Agresor_Victima : catálogo (cónyuge, padre, madre, hijo/a, etc.)
- Hecho_VIF : registro principal

SUBSISTEMA DE VIOLENCIA INFANTIL

Incluye trabajo infantil (guiado por estudio INE/IPEC-OIT) y embarazos en menores (fuente OSAR).

Entidades:

- Tipo_Violencia_Infantil : maltrato físico, psicológico, abandono, explotación
- Hecho_Violencia_Infantil
- Sector_Trabajo_Infantil : agricultura, industria, servicios, doméstico, informal
- Tipo_Trabajo_Infantil : asalariado, familiar no remunerado, cuenta propia
- Jornada_Trabajo_Infantil: tiempo completo, medio tiempo, ocasional
- Trabajo_Infantil : registro del menor trabajador
- Embarazo_Adolescente : registro por año, municipio, grupo etario
- Control_Prenatal : atenciones recibidas

SUBSISTEMA DE VIOLENCIA ESTRUCTURAL / DISCRIMINACIÓN

Entidades:

- Tipo_Discriminacion : étnica, de género, por edad, por discapacidad, religiosa
- Grupo_Poblacional : maya, garífuna, xinca, ladino, LGBTIQ+, migrantes, etc.
- Hecho_Discriminacion : registro (incluye si se procesó o no)

SUBSISTEMA DE FALTAS JUDICIALES

Entidades:

- Tipo_Falta_Judicial : catálogo (faltas leves)
- Falta_Judicial : registro
- Sancion_Falta : sanción impuesta

SUBSISTEMA DE SALUD PÚBLICA

La información de salud viene agregada por: departamento/municipio + grupo etario + sexo + año + indicador. Fuentes: MSPAS, SIGSA.

Entidades:

- Grupo_Etario : catálogo
- Capitulo_CIE10 : 22 capítulos
- CIE10 : códigos detallados
- Categoria_Indicador : desnutrición / crónicas / vectores / morbilidad MI

- Indicador_Salud : genérico
- Registro_Salud : tabla agregada principal
- Registro_Desnutricion : específico (aguda/retardo)
- Registro_Enfermedad_Cronica : con CIE-10
- Vector_Enfermedad : mosquito Aedes, Anopheles, vinchuca, etc.
- Registro_Enfermedad_Vector: dengue, malaria, chagas, zika, chikungunya
- Complicacion_Embarazo : preeclampsia, hemorragia, sepsis, etc.
- Registro_Morbilidad_Materna
- Centro_Salud, Tipo_Centro_Salud : catálogos

SUBSISTEMA DE ENCUESTAS

Para enriquecer análisis con datos socioeconómicos.

Entidades:

- Tipo_Encuesta, Encuesta, Respuesta_Encuesta

IDENTIFICACIÓN DE RELACIONES (CARDINALIDADES CLAVE)

- Pais (1) ----< Departamento (N) ----< Municipio (N)
- Municipio (1) ----< Domicilio (N) ----> Persona (1)
- Persona (1) ----< Empleo_Persona (N) >---- Tipo_Empleo (1)
- Persona (1) ----< Escolaridad_Persona (N) >---- Nivel_Educativo (1)
- Persona (N) >--< Idioma (N) [tabla Persona_Idioma]
- Persona (N) >--< Hogar (N) [tabla Persona_Hogar con parentesco]
- Denuncia (1) ----< Persona_Denuncia (N) >---- Persona (1) con ROL

- Tipo_Denuncia (1) ----< Denuncia (N)

- Eje_Violencia (1) ----< Tipo_Denuncia (N)

- Hecho_Delictivo (1) ----< Victima (N) >---- Persona (1)

- Hecho_Delictivo (1) ----< Agresor (N) >---- Persona (1)

- Hecho_Delictivo (1) ----< Necropsia (N) ----< Exhumacion (N)

- Sentencia (1) ----0..1 Persona_Sentenciada (opcional - puede no existir)

- Registro_Salud (N) >---- Indicador_Salud (1)

- Registro_Salud (N) >---- Municipio (1), Grupo_Etario (1), Sexo (1)

REGLAS DE NEGOCIO IMPORTANTES

R1. Una persona puede aparecer con MÚLTIPLES roles en distintas denuncias.

R2. Una sentencia puede existir SIN detalle de la persona sentenciada (opcional).

R3. Un hecho delictivo puede no tener víctima identificada (ej. robo sin testigos)
o puede tener varias víctimas.

R4. Un agresor puede ser "desconocido" (persona anónima registrada con rol).

R5. Los registros de salud son AGREGADOS (cantidad de casos), no individuales.

R6. Un embarazo adolescente se registra aunque la persona no esté identificada
(registro agregado por grupo etario y municipio).

R7. Una denuncia puede haber sido PROCESADA o NO; debe quedar claro.

R8. La edad al momento del hecho debe poderse calcular (fecha_nacimiento vs fecha_hecho).

R9. Un domicilio puede tener histórico: solo UNO debe ser el actual por persona.

R10. El trabajo infantil es una RELACIÓN entre persona menor y un tipo/sector laboral
con jornada y remuneración; una misma persona puede tener varios registros por año.

NORMALIZACIÓN (3FN)

1FN: Todos los atributos son atómicos; no hay grupos repetidos.

Ej: teléfonos se separan en tabla Contacto; idiomas en Persona_Idioma.

2FN: Todos los atributos no clave dependen completamente de la clave primaria.

Ej: En Persona_Denuncia (id_persona, id_denuncia, rol) el rol depende de la combinación completa, no de una parte.

3FN: No hay dependencias transitivas.

Ej: En Municipio no se guarda "nombre_departamento"; se guarda id_departamento y el nombre se obtiene de la tabla Departamento.

Ej: En Persona no se guarda "edad" (deriva de fecha_nacimiento).

CONSIDERACIONES TÉCNICAS DE FIREBIRD

- Se usarán SEQUENCES (generators) para PK autoincrementales.
- Se usarán TRIGGERS BEFORE INSERT para autocompletar PK con NEXT VALUE FOR.
- Tipos usados: INTEGER, VARCHAR, DATE, TIMESTAMP, DECIMAL, SMALLINT (booleano), BLOB SUB_TYPE TEXT (descripciones largas).
- No existe BOOLEAN en Firebird 2.x; se usa SMALLINT (0/1). En 3.x existe pero por compatibilidad se mantiene SMALLINT.
- Integridad referencial con ON UPDATE CASCADE y ON DELETE NO ACTION (para no perder históricos).
- Índices adicionales en columnas de búsqueda frecuente (fecha_hecho, id_municipio, anio, id_tipo_delito).

ESTRATEGIA DE DATOS

Fuentes reales utilizadas:

- Carpeta "Hechos-delictivos" (Drive): hechos delictivos y procesos asociados
- Carpeta "violencia-contra-la-mujer": denuncias y atenciones
- Carpeta "violencia-intrafamiliar": registros VIF
- Carpeta "faltas-judiciales": faltas leves
- observatorio.mp.gob.gt/sentencias/ : sentencias detalladas (web scraping con Selenium)
- osarguatemala.org/embarazo/datos/ : embarazos adolescentes (web scraping)
- Estudio INE/IPEC-OIT Trabajo Infantil (PDF adjunto, ENCOVI 2000) y esquema XML/JSON de México (INEGI RNM 919) para variables de trabajo infantil
- MSPAS/SIGSA: desnutrición, enfermedades crónicas, vectores, morbilidad MI

GEOGRAFÍA (5):

1. Pais
2. Region
3. Departamento
4. Municipio
5. Area_Geografica

CATÁLOGOS BASE (12):

6. Sexo
7. Etnia
8. Idioma
9. Estado_Civil

10. Religion
11. Discapacidad
12. Grupo_Etario
13. Nivel_Educativo
14. Institucion_Educativa
15. Tipo_Vivienda
16. Nivel_Socioeconomico
17. Sector_Economico

PERSONA Y HOGAR (10):

18. Persona
19. Persona_Idioma
20. Persona_Discapacidad
21. Domicilio
22. Contacto
23. Escolaridad_Persona
24. Ocupacion
25. Tipo_Empleo
26. Condicion_Laboral
27. Empleo_Persona
28. Hogar
29. Persona_Hogar

DENUNCIAS (7):

30. Eje_Violencia

31. Tipo_Denuncia
32. Institucion_Reportante
33. Estado_Denuncia
34. Denuncia
35. Persona_Denuncia
36. Seguimiento_Denuncia

HECHOS DELICTIVOS (10):

37. Clasificacion_Delito
38. Tipo_Hecho_Delictivo
39. Hecho_Delictivo
40. Arma_Utilizada
41. Hecho_Arma
42. Movil_Delito
43. Hecho_Movil
44. Lugar_Ocurrencia
45. Victima
46. Agresor

FORENSE (4):

47. Laboratorio_Forense
48. Forense
49. Necropsia
50. Exhumacion

SENTENCIAS (6):

51. Juzgado
52. Tipo_Delito_Juzgado
53. Pena
54. Sentencia
55. Persona_Sentenciada
56. Detalle_Sentencia

VIOLENCIA CONTRA LA MUJER (5):

57. Tipo_Violencia_Mujer
58. Centro_Atencion
59. Tipo_Atencion
60. Hecho_Violencia_Mujer
61. Atencion_Mujer

VIOLENCIA INTRAFAMILIAR (3):

62. Tipo_VIF
63. Relacion_Agresor_Victima
64. Hecho_VIF

VIOLENCIA INFANTIL (8):

65. Tipo_Violencia_Infantil
66. Hecho_Violencia_Infantil
67. Sector_Trabajo_Infantil
68. Tipo_Trabajo_Infantil

69. Jornada_Trabajo_Infantil

70. Trabajo_Infantil

71. Embarazo_Adolescente

72. Control_Prenatal

VIOLENCIA ESTRUCTURAL (3):

73. Tipo_Discriminacion

74. Grupo_Poblacional

75. Hecho_Discriminacion

FALTAS JUDICIALES (3):

76. Tipo_Falta_Judicial

77. Sancion_Falta

78. Falta_Judicial

SALUD PÚBLICA (13):

79. Capitulo_CIE10

80. CIE10

81. Categoria_Indicador

82. Indicador_Salud

83. Tipo_Centro_Salud

84. Centro_Salud

85. Registro_Salud

86. Registro_Desnutricion

87. Registro_Enfermedad_Cronica

88. Vector_Enfermedad

89. Registro_Enfermedad_Vector

90. Complicacion_Embarazo

91. Registro_Morbilidad_Materna

ENCUESTAS (3):

92. Tipo_Encuesta

93. Encuesta

94. Respuesta_Encuesta

TOTAL: 94 tablas (cumple el requisito de >= 90)

FLUJO DE IMPLEMENTACIÓN

==========

Paso 1: Crear generators (SEQUENCE).

Paso 2: Crear tablas en orden topológico (catálogos primero, luego transaccionales).

Paso 3: Crear triggers BEFORE INSERT para PK autoincrementales.

Paso 4: Crear índices secundarios.

Paso 5: Inserts de catálogos (datos estáticos: 22 deptos, 340 municipios, etc.).

Paso 6: Inserts de personas y hogares (Faker + datos reales donde existan).

Paso 7: Inserts de registros transaccionales (denuncias, hechos, sentencias).

Paso 8: Inserts de registros de salud (agregados reales MSPAS 2012-2024).

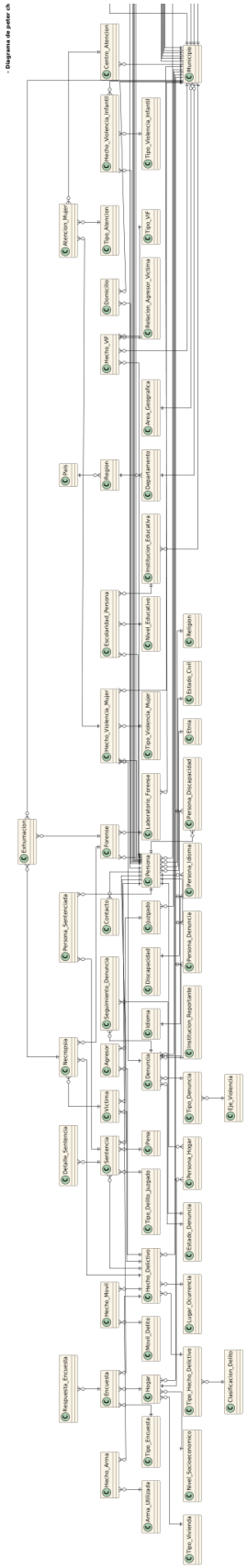
Paso 9: Ejecutar las 30 consultas y validar.

Paso 10: Documentar álgebra relacional por cada consulta.

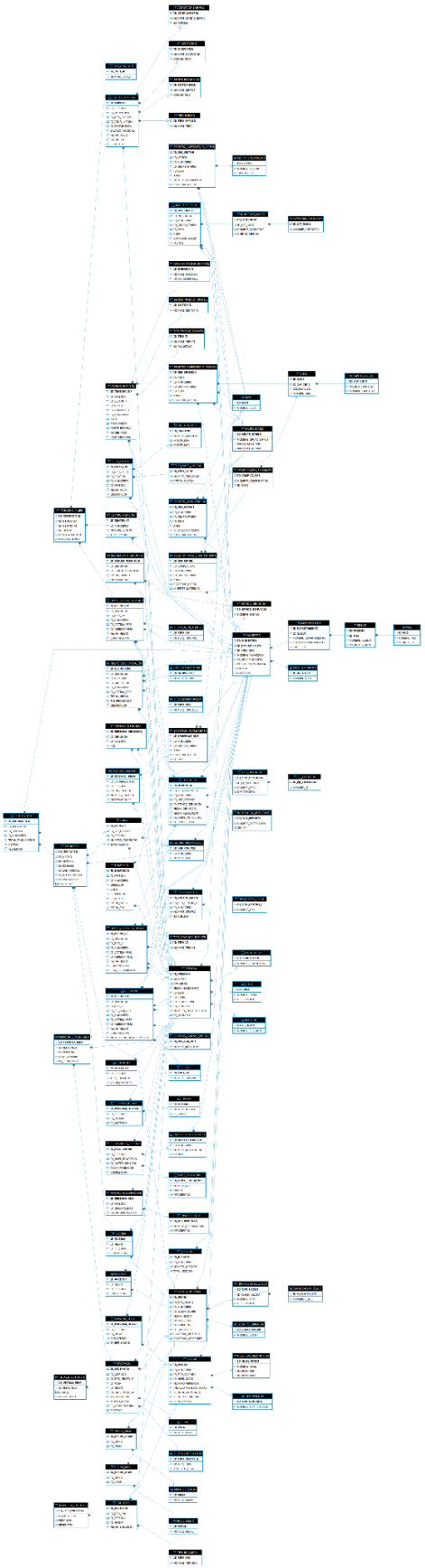
CRITERIOS DE VALIDACIÓN

- Toda tabla tiene PK.
- Toda FK apunta a una PK existente.
- No hay atributos redundantes (3FN).
- Todos los atributos multivaluados están en tablas separadas.
- Las consultas no requieren más de 5 JOINs innecesarios (modelo bien integrado).
- Los reportes cruzados violencia-salud son posibles vía Municipio y año.

# Diagrama entidad de relación:



# Diagrama de tablas:



# SQL FIREBIRD:

```
/*
==================================================================
===========

  PROYECTO 2 - MODELO FÍSICO (DDL)

  Indicadores Sociales y Sanitarios de Guatemala

  Total tablas: 94  |  Normalización: 3FN

==================================================================
=========== */

SET AUTODDL OFF;

COMMIT;

/*
==================================================================
===========

  1. SECUENCIAS (GENERATORS)

==================================================================
=========== */

CREATE SEQUENCE seq_pais;

CREATE SEQUENCE seq_region;

CREATE SEQUENCE seq_departamento;

CREATE SEQUENCE seq_municipio;

CREATE SEQUENCE seq_area_geo;

CREATE SEQUENCE seq_sexo;

CREATE SEQUENCE seq_etnia;
```

```
CREATE SEQUENCE seq_idioma;

CREATE SEQUENCE seq_estado_civil;

CREATE SEQUENCE seq_religion;

CREATE SEQUENCE seq_discapacidad;

CREATE SEQUENCE seq_grupo_etario;

CREATE SEQUENCE seq_nivel_educativo;

CREATE SEQUENCE seq_institucion_edu;

CREATE SEQUENCE seq_tipo_vivienda;

CREATE SEQUENCE seq_nivel_socio;

CREATE SEQUENCE seq_sector_econ;

CREATE SEQUENCE seq_persona;

CREATE SEQUENCE seq_persona_idioma;

CREATE SEQUENCE seq_persona_disc;

CREATE SEQUENCE seq_domicilio;

CREATE SEQUENCE seq_contacto;

CREATE SEQUENCE seq_escolaridad;

CREATE SEQUENCE seq_ocupacion;

CREATE SEQUENCE seq_tipo_empleo;

CREATE SEQUENCE seq_cond_laboral;

CREATE SEQUENCE seq_empleo;

CREATE SEQUENCE seq_hogar;

CREATE SEQUENCE seq_persona_hogar;

CREATE SEQUENCE seq_eje_violencia;

CREATE SEQUENCE seq_tipo_denuncia;

CREATE SEQUENCE seq_inst_reporte;
```

```
CREATE SEQUENCE seq_estado_denuncia;

CREATE SEQUENCE seq_denuncia;

CREATE SEQUENCE seq_persona_denuncia;

CREATE SEQUENCE seq_seguim_denuncia;

CREATE SEQUENCE seq_clasif_delito;

CREATE SEQUENCE seq_tipo_hecho;

CREATE SEQUENCE seq_hecho;

CREATE SEQUENCE seq_arma;

CREATE SEQUENCE seq_hecho_arma;

CREATE SEQUENCE seq_movil;

CREATE SEQUENCE seq_hecho_movil;

CREATE SEQUENCE seq_lugar_ocurr;

CREATE SEQUENCE seq_victima;

CREATE SEQUENCE seq_agresor;

CREATE SEQUENCE seq_lab_forense;

CREATE SEQUENCE seq_forense;

CREATE SEQUENCE seq_necropsia;

CREATE SEQUENCE seq_exhumacion;

CREATE SEQUENCE seq_juzgado;

CREATE SEQUENCE seq_tipo_delito_jz;

CREATE SEQUENCE seq_pena;

CREATE SEQUENCE seq_sentencia;

CREATE SEQUENCE seq_persona_sent;

CREATE SEQUENCE seq_detalle_sent;

CREATE SEQUENCE seq_tipo_vm;
```

```
CREATE SEQUENCE seq_centro_at;

CREATE SEQUENCE seq_tipo_at;

CREATE SEQUENCE seq_hecho_vm;

CREATE SEQUENCE seq_atencion_m;

CREATE SEQUENCE seq_tipo_vif;

CREATE SEQUENCE seq_rel_agr_vict;

CREATE SEQUENCE seq_hecho_vif;

CREATE SEQUENCE seq_tipo_vi;

CREATE SEQUENCE seq_hecho_vi;

CREATE SEQUENCE seq_sector_ti;

CREATE SEQUENCE seq_tipo_ti;

CREATE SEQUENCE seq_jornada_ti;

CREATE SEQUENCE seq_trabajo_inf;

CREATE SEQUENCE seq_embarazo_ado;

CREATE SEQUENCE seq_control_pren;

CREATE SEQUENCE seq_tipo_disc;

CREATE SEQUENCE seq_grupo_pob;

CREATE SEQUENCE seq_hecho_disc;

CREATE SEQUENCE seq_tipo_falta;

CREATE SEQUENCE seq_sancion;

CREATE SEQUENCE seq_falta_jud;

CREATE SEQUENCE seq_cap_cie10;

CREATE SEQUENCE seq_cie10;

CREATE SEQUENCE seq_cat_indic;

CREATE SEQUENCE seq_indicador;
```

```
CREATE SEQUENCE seq_tipo_centro_s;

CREATE SEQUENCE seq_centro_salud;

CREATE SEQUENCE seq_reg_salud;

CREATE SEQUENCE seq_reg_desnut;

CREATE SEQUENCE seq_reg_cronica;

CREATE SEQUENCE seq_vector;

CREATE SEQUENCE seq_reg_vector;

CREATE SEQUENCE seq_complic_emb;

CREATE SEQUENCE seq_reg_morbi;

CREATE SEQUENCE seq_tipo_enc;

CREATE SEQUENCE seq_encuesta;

CREATE SEQUENCE seq_resp_encuesta;


COMMIT;


/*
==========================================================================
===========

  2. GEOGRAFÍA

==========================================================================
=========== */


CREATE TABLE Pais (

   id_pais       INTEGER NOT NULL PRIMARY KEY,

   nombre_pais    VARCHAR(100) NOT NULL,

   codigo_iso     VARCHAR(10)
```

```
);

CREATE TABLE Region (
    id_region      INTEGER NOT NULL PRIMARY KEY,
    id_pais        INTEGER NOT NULL,
    nombre_region  VARCHAR(100) NOT NULL,
    codigo_region  VARCHAR(10),
    CONSTRAINT fk_region_pais FOREIGN KEY (id_pais) REFERENCES Pais(id_pais)
);

CREATE TABLE Departamento (
    id_departamento     INTEGER NOT NULL PRIMARY KEY,
    id_region           INTEGER NOT NULL,
    nombre_departamento VARCHAR(100) NOT NULL,
    codigo_departamento VARCHAR(10),
    cabecera            VARCHAR(100),
        CONSTRAINT fk_depto_region FOREIGN KEY (id_region) REFERENCES Region(id_region)
);

CREATE TABLE Area_Geografica (
    id_area_geo     INTEGER NOT NULL PRIMARY KEY,
    nombre_area     VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE Municipio (
    id_municipio        INTEGER NOT NULL PRIMARY KEY,
    id_departamento     INTEGER NOT NULL,
    id_area_geo         INTEGER,
    nombre_municipio    VARCHAR(100) NOT NULL,
    codigo_municipio    VARCHAR(10),
    poblacion_estimada  INTEGER,
    latitud             DECIMAL(10,6),
    longitud            DECIMAL(10,6),
        CONSTRAINT fk_muni_depto FOREIGN KEY (id_departamento) REFERENCES Departamento(id_departamento),
        CONSTRAINT fk_muni_area FOREIGN KEY (id_area_geo) REFERENCES Area_Geografica(id_area_geo)
);


/*
==================================================================================
   3. CATÁLOGOS BASE

==================================================================================
*/


CREATE TABLE Sexo (
    id_sexo     INTEGER NOT NULL PRIMARY KEY,
    nombre_sexo VARCHAR(20) NOT NULL
);
```

```
CREATE TABLE Etnia (
    id_etnia     INTEGER NOT NULL PRIMARY KEY,
    nombre_etnia VARCHAR(50) NOT NULL,
    descripcion  VARCHAR(200)
);

CREATE TABLE Idioma (
    id_idioma     INTEGER NOT NULL PRIMARY KEY,
    nombre_idioma VARCHAR(50) NOT NULL,
    es_oficial    SMALLINT DEFAULT 0
);

CREATE TABLE Estado_Civil (
    id_estado_civil     INTEGER NOT NULL PRIMARY KEY,
    nombre_estado_civil VARCHAR(50) NOT NULL
);

CREATE TABLE Religion (
    id_religion     INTEGER NOT NULL PRIMARY KEY,
    nombre_religion VARCHAR(80) NOT NULL
);

CREATE TABLE Discapacidad (
    id_discapacidad     INTEGER NOT NULL PRIMARY KEY,
    nombre_discapacidad VARCHAR(80) NOT NULL,
```

```
    descripcion          VARCHAR(200)
);

CREATE TABLE Grupo_Etario (
    id_grupo_etario     INTEGER NOT NULL PRIMARY KEY,
    nombre_grupo_etario VARCHAR(50) NOT NULL,
    rango_edad_min      INTEGER NOT NULL,
    rango_edad_max      INTEGER NOT NULL
);

CREATE TABLE Nivel_Educativo (
    id_nivel_educativo INTEGER NOT NULL PRIMARY KEY,
    nombre_nivel        VARCHAR(100) NOT NULL,
    orden               INTEGER,
    descripcion         VARCHAR(200)
);

CREATE TABLE Institucion_Educativa (
    id_institucion_edu  INTEGER NOT NULL PRIMARY KEY,
    id_municipio        INTEGER,
    nombre_institucion  VARCHAR(150) NOT NULL,
    sector              VARCHAR(30),
        CONSTRAINT fk_ie_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio)
);
```

```
CREATE TABLE Tipo_Vivienda (
    id_tipo_vivienda     INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo_vivienda VARCHAR(80) NOT NULL
);

CREATE TABLE Nivel_Socioeconomico (
    id_nivel_socio  INTEGER NOT NULL PRIMARY KEY,
    nombre_nivel    VARCHAR(50) NOT NULL,
    ingreso_min     DECIMAL(10,2),
    ingreso_max     DECIMAL(10,2)
);

CREATE TABLE Sector_Economico (
    id_sector_econ  INTEGER NOT NULL PRIMARY KEY,
    nombre_sector    VARCHAR(100) NOT NULL,
    codigo_ciiu      VARCHAR(10)
);

/*
==========================================================================
===========
   4. PERSONA Y HOGAR

==========================================================================
=========== */
```

```
CREATE TABLE Persona (
    id_persona          INTEGER NOT NULL PRIMARY KEY,
    nombres             VARCHAR(100),
    apellidos           VARCHAR(100),
    fecha_nacimiento    DATE,
    id_sexo             INTEGER,
    id_etnia            INTEGER,
    id_estado_civil     INTEGER,
    id_religion         INTEGER,
    numero_identificacion VARCHAR(50),
    es_anonimo          SMALLINT DEFAULT 0,
    CONSTRAINT fk_pers_sexo  FOREIGN KEY (id_sexo) REFERENCES Sexo(id_sexo),
    CONSTRAINT fk_pers_etnia FOREIGN KEY (id_etnia) REFERENCES Etnia(id_etnia),
    CONSTRAINT fk_pers_ec FOREIGN KEY (id_estado_civil) REFERENCES Estado_Civil(id_estado_civil),
    CONSTRAINT fk_pers_rel FOREIGN KEY (id_religion) REFERENCES Religion(id_religion)
);

CREATE TABLE Persona_Idioma (
    id_persona_idioma INTEGER NOT NULL PRIMARY KEY,
    id_persona        INTEGER NOT NULL,
    id_idioma         INTEGER NOT NULL,
    es_materno        SMALLINT DEFAULT 0,
    CONSTRAINT fk_pi_pers FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),
```

```
        CONSTRAINT fk_pi_idioma FOREIGN KEY (id_idioma) REFERENCES Idioma(id_idioma)

);


CREATE TABLE Persona_Discapacidad (

   id_persona_disc  INTEGER NOT NULL PRIMARY KEY,

   id_persona       INTEGER NOT NULL,

   id_discapacidad  INTEGER NOT NULL,

   fecha_diagnostico DATE,

        CONSTRAINT fk_pd_pers_disc FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),

      CONSTRAINT fk_pd_disc_base FOREIGN KEY (id_discapacidad) REFERENCES Discapacidad(id_discapacidad)

);


CREATE TABLE Domicilio (

   id_domicilio     INTEGER NOT NULL PRIMARY KEY,

   id_persona       INTEGER NOT NULL,

   id_municipio     INTEGER NOT NULL,

   direccion        VARCHAR(250),

   zona             VARCHAR(20),

   referencia       VARCHAR(250),

   es_actual        SMALLINT DEFAULT 1,

   fecha_inicio     DATE,

   fecha_fin        DATE,

        CONSTRAINT fk_dom_pers FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),
```

```
        CONSTRAINT fk_dom_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio)

);


CREATE TABLE Contacto (

   id_contacto    INTEGER NOT NULL PRIMARY KEY,

   id_persona     INTEGER NOT NULL,

   tipo_contacto  VARCHAR(30),

   valor_contacto VARCHAR(100) NOT NULL,

        CONSTRAINT fk_cont_pers FOREIGN KEY (id_persona) REFERENCES Persona(id_persona)

);


CREATE TABLE Escolaridad_Persona (

   id_escolaridad     INTEGER NOT NULL PRIMARY KEY,

   id_persona         INTEGER NOT NULL,

   id_nivel_educativo INTEGER NOT NULL,

   id_institucion_edu INTEGER,

   anio_aprobacion    INTEGER,

   completado         SMALLINT DEFAULT 0,

       CONSTRAINT fk_esc_pers_base   FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),

      CONSTRAINT fk_esc_nivel FOREIGN KEY (id_nivel_educativo) REFERENCES Nivel_Educativo(id_nivel_educativo),

      CONSTRAINT fk_esc_ie       FOREIGN KEY (id_institucion_edu) REFERENCES Institucion_Educativa(id_institucion_edu)

);
```

```
CREATE TABLE Ocupacion (
    id_ocupacion    INTEGER NOT NULL PRIMARY KEY,
    nombre_ocupacion VARCHAR(150) NOT NULL,
    codigo_ciuo      VARCHAR(10)
);

CREATE TABLE Tipo_Empleo (
    id_tipo_empleo   INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo      VARCHAR(100) NOT NULL
);

CREATE TABLE Condicion_Laboral (
    id_cond_laboral     INTEGER NOT NULL PRIMARY KEY,
    nombre_cond_laboral VARCHAR(60) NOT NULL,
    es_formal           SMALLINT DEFAULT 0
);

CREATE TABLE Empleo_Persona (
    id_empleo        INTEGER NOT NULL PRIMARY KEY,
    id_persona       INTEGER NOT NULL,
    id_ocupacion     INTEGER,
    id_tipo_empleo   INTEGER,
    id_cond_laboral  INTEGER,
    id_sector_econ   INTEGER,
```

```
    salario_mensual  DECIMAL(12,2),

    fecha_inicio     DATE,

    fecha_fin        DATE,

    es_actual        SMALLINT DEFAULT 1,

        CONSTRAINT  fk_emp_pers_base      FOREIGN  KEY  (id_persona)  REFERENCES
Persona(id_persona),

         CONSTRAINT  fk_emp_ocup      FOREIGN  KEY  (id_ocupacion)  REFERENCES
Ocupacion(id_ocupacion),

         CONSTRAINT  fk_emp_tipo      FOREIGN  KEY  (id_tipo_empleo)  REFERENCES
Tipo_Empleo(id_tipo_empleo),

        CONSTRAINT  fk_emp_cond      FOREIGN  KEY  (id_cond_laboral)  REFERENCES
Condicion_Laboral(id_cond_laboral),

         CONSTRAINT  fk_emp_sector  FOREIGN  KEY  (id_sector_econ)  REFERENCES
Sector_Economico(id_sector_econ)

);


CREATE TABLE Hogar (

    id_hogar              INTEGER NOT NULL PRIMARY KEY,

    id_municipio          INTEGER NOT NULL,

    id_tipo_vivienda      INTEGER,

    id_nivel_socio        INTEGER,

    numero_personas       INTEGER,

    ingreso_mensual_total DECIMAL(12,2),

    tiene_agua_potable    SMALLINT DEFAULT 0,

    tiene_electricidad    SMALLINT DEFAULT 0,

    tiene_drenaje         SMALLINT DEFAULT 0,

         CONSTRAINT  fk_hog_muni      FOREIGN  KEY  (id_municipio)  REFERENCES
Municipio(id_municipio),
```

```
    CONSTRAINT fk_hog_tv        FOREIGN KEY (id_tipo_vivienda) REFERENCES Tipo_Vivienda(id_tipo_vivienda),

    CONSTRAINT fk_hog_socio   FOREIGN KEY (id_nivel_socio) REFERENCES Nivel_Socioeconomico(id_nivel_socio)

);


CREATE TABLE Persona_Hogar (

  id_persona_hogar INTEGER NOT NULL PRIMARY KEY,

  id_persona       INTEGER NOT NULL,

  id_hogar         INTEGER NOT NULL,

  parentesco       VARCHAR(60),

  es_jefe_hogar    SMALLINT DEFAULT 0,

    CONSTRAINT fk_ph_pers_base FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),

  CONSTRAINT fk_ph_hog  FOREIGN KEY (id_hogar) REFERENCES Hogar(id_hogar)

);


/*
=================================================================================

  5. DENUNCIAS (transversal)

=================================================================================
=========== */


CREATE TABLE Eje_Violencia (

  id_eje_violencia INTEGER NOT NULL PRIMARY KEY,

  nombre_eje       VARCHAR(80) NOT NULL
```

```
);

CREATE TABLE Tipo_Denuncia (
    id_tipo_denuncia INTEGER NOT NULL PRIMARY KEY,
    id_eje_violencia INTEGER NOT NULL,
    nombre_tipo      VARCHAR(150) NOT NULL,
    descripcion      VARCHAR(300),
        CONSTRAINT fk_td_eje FOREIGN KEY (id_eje_violencia) REFERENCES Eje_Violencia(id_eje_violencia)
);

CREATE TABLE Institucion_Reportante (
    id_inst_reporte    INTEGER NOT NULL PRIMARY KEY,
    nombre_institucion VARCHAR(150) NOT NULL,
    siglas             VARCHAR(20)
);

CREATE TABLE Estado_Denuncia (
    id_estado_denuncia INTEGER NOT NULL PRIMARY KEY,
    nombre_estado      VARCHAR(50) NOT NULL
);

CREATE TABLE Denuncia (
    id_denuncia          INTEGER NOT NULL PRIMARY KEY,
    id_tipo_denuncia     INTEGER NOT NULL,
```

```
    id_municipio          INTEGER NOT NULL,

    id_inst_reporte       INTEGER,

    id_estado_denuncia    INTEGER,

    fecha_denuncia        DATE NOT NULL,

    fecha_resolucion      DATE,

    descripcion_hechos    BLOB SUB_TYPE TEXT,

    fue_procesada         SMALLINT DEFAULT 0,

      CONSTRAINT fk_den_tipo        FOREIGN KEY (id_tipo_denuncia) REFERENCES
Tipo_Denuncia(id_tipo_denuncia),

      CONSTRAINT fk_den_muni        FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio),

      CONSTRAINT fk_den_inst        FOREIGN KEY (id_inst_reporte) REFERENCES
Institucion_Reportante(id_inst_reporte),

     CONSTRAINT fk_den_estado    FOREIGN KEY (id_estado_denuncia) REFERENCES
Estado_Denuncia(id_estado_denuncia)

);


CREATE TABLE Persona_Denuncia (

    id_persona_denuncia INTEGER NOT NULL PRIMARY KEY,

    id_denuncia         INTEGER NOT NULL,

    id_persona          INTEGER NOT NULL,

    rol                 VARCHAR(30) NOT NULL,

         CONSTRAINT fk_pd_den_rel    FOREIGN KEY (id_denuncia) REFERENCES
Denuncia(id_denuncia),

         CONSTRAINT fk_pd_pers_rel FOREIGN KEY (id_persona) REFERENCES
Persona(id_persona)

);
```

```
CREATE TABLE Seguimiento_Denuncia (

   id_seguim_denuncia  INTEGER NOT NULL PRIMARY KEY,

   id_denuncia         INTEGER NOT NULL,

   id_estado_denuncia  INTEGER NOT NULL,

   fecha_cambio        DATE NOT NULL,

   observaciones       VARCHAR(500),

         CONSTRAINT fk_sd_den     FOREIGN KEY (id_denuncia) REFERENCES Denuncia(id_denuncia),

      CONSTRAINT fk_sd_estado FOREIGN KEY (id_estado_denuncia) REFERENCES Estado_Denuncia(id_estado_denuncia)

);


/*
==================================================================
===========

  6. HECHOS DELICTIVOS


==================================================================
=========== */


CREATE TABLE Clasificacion_Delito (

   id_clasif_delito INTEGER NOT NULL PRIMARY KEY,

   nombre_clasif    VARCHAR(50) NOT NULL

);


CREATE TABLE Tipo_Hecho_Delictivo (

   id_tipo_hecho    INTEGER NOT NULL PRIMARY KEY,

   id_clasif_delito INTEGER NOT NULL,
```

```
    nombre_tipo      VARCHAR(150) NOT NULL,

    descripcion      VARCHAR(300),

        CONSTRAINT fk_th_clasif FOREIGN KEY (id_clasif_delito) REFERENCES
Clasificacion_Delito(id_clasif_delito)

);


CREATE TABLE Lugar_Ocurrencia (

    id_lugar_ocurr INTEGER NOT NULL PRIMARY KEY,

    nombre_lugar   VARCHAR(100) NOT NULL

);


CREATE TABLE Hecho_Delictivo (

    id_hecho            INTEGER NOT NULL PRIMARY KEY,

    id_tipo_hecho       INTEGER NOT NULL,

    id_municipio        INTEGER NOT NULL,

    id_lugar_ocurr      INTEGER,

    fecha_hecho         DATE NOT NULL,

    hora_hecho          TIME,

    descripcion         BLOB SUB_TYPE TEXT,

    fue_resuelto        SMALLINT DEFAULT 0,

    cantidad_victimas   INTEGER DEFAULT 0,

    cantidad_agresores  INTEGER DEFAULT 0,

        CONSTRAINT fk_hd_tipo FOREIGN KEY (id_tipo_hecho) REFERENCES
Tipo_Hecho_Delictivo(id_tipo_hecho),

        CONSTRAINT fk_hd_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio),
```

```
        CONSTRAINT fk_hd_lugar FOREIGN KEY (id_lugar_ocurr) REFERENCES Lugar_Ocurrencia(id_lugar_ocurr)

);


CREATE TABLE Arma_Utilizada (

   id_arma      INTEGER NOT NULL PRIMARY KEY,

   nombre_arma  VARCHAR(100) NOT NULL

);


CREATE TABLE Hecho_Arma (

   id_hecho_arma INTEGER NOT NULL PRIMARY KEY,

   id_hecho      INTEGER NOT NULL,

   id_arma       INTEGER NOT NULL,

        CONSTRAINT fk_ha_hecho FOREIGN KEY (id_hecho) REFERENCES Hecho_Delictivo(id_hecho),

        CONSTRAINT fk_ha_arma FOREIGN KEY (id_arma) REFERENCES Arma_Utilizada(id_arma)

);


CREATE TABLE Movil_Delito (

   id_movil      INTEGER NOT NULL PRIMARY KEY,

   nombre_movil  VARCHAR(100) NOT NULL

);


CREATE TABLE Hecho_Movil (

   id_hecho_movil INTEGER NOT NULL PRIMARY KEY,
```

```
    id_hecho      INTEGER NOT NULL,

    id_movil      INTEGER NOT NULL,

            CONSTRAINT fk_hm_hecho FOREIGN KEY (id_hecho) REFERENCES
Hecho_Delictivo(id_hecho),

            CONSTRAINT fk_hm_movil FOREIGN KEY (id_movil) REFERENCES
Movil_Delito(id_movil)

);


CREATE TABLE Victima (

    id_victima   INTEGER NOT NULL PRIMARY KEY,

    id_hecho     INTEGER NOT NULL,

    id_persona   INTEGER NOT NULL,

    estado_final VARCHAR(50),

            CONSTRAINT fk_vic_hecho FOREIGN KEY (id_hecho) REFERENCES
Hecho_Delictivo(id_hecho),

         CONSTRAINT fk_vic_pers_base FOREIGN KEY (id_persona) REFERENCES
Persona(id_persona)

);


CREATE TABLE Agresor (

    id_agresor   INTEGER NOT NULL PRIMARY KEY,

    id_hecho     INTEGER NOT NULL,

    id_persona   INTEGER NOT NULL,

    capturado    SMALLINT DEFAULT 0,

            CONSTRAINT fk_agr_hecho FOREIGN KEY (id_hecho) REFERENCES
Hecho_Delictivo(id_hecho),

         CONSTRAINT fk_agr_pers_base FOREIGN KEY (id_persona) REFERENCES
Persona(id_persona)
```

```
);


/*
==========================================================================
===========

  7. FORENSE (Necropsias y Exhumaciones)

==========================================================================
=========== */


CREATE TABLE Laboratorio_Forense (

   id_lab_forense   INTEGER NOT NULL PRIMARY KEY,

   id_municipio     INTEGER,

   nombre_lab       VARCHAR(150) NOT NULL,

        CONSTRAINT fk_lab_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio)

);


CREATE TABLE Forense (

   id_forense   INTEGER NOT NULL PRIMARY KEY,

   id_lab_forense INTEGER,

   id_persona     INTEGER,

   numero_colegiado VARCHAR(30),

   especialidad   VARCHAR(100),

        CONSTRAINT fk_for_lab    FOREIGN KEY (id_lab_forense) REFERENCES
Laboratorio_Forense(id_lab_forense),

        CONSTRAINT fk_for_pers_base FOREIGN KEY (id_persona) REFERENCES
Persona(id_persona)
```

```
);

CREATE TABLE Necropsia (
    id_necropsia   INTEGER NOT NULL PRIMARY KEY,
    id_hecho       INTEGER NOT NULL,
    id_victima     INTEGER,
    id_forense     INTEGER,
    id_lab_forense INTEGER,
    fecha_necropsia DATE,
    causa_muerte   VARCHAR(250),
    hallazgos      BLOB SUB_TYPE TEXT,
        CONSTRAINT fk_nec_hecho FOREIGN KEY (id_hecho) REFERENCES Hecho_Delictivo(id_hecho),
        CONSTRAINT fk_nec_vic FOREIGN KEY (id_victima) REFERENCES Victima(id_victima),
        CONSTRAINT fk_nec_for FOREIGN KEY (id_forense) REFERENCES Forense(id_forense),
        CONSTRAINT fk_nec_lab FOREIGN KEY (id_lab_forense) REFERENCES Laboratorio_Forense(id_lab_forense)
);

CREATE TABLE Exhumacion (
    id_exhumacion   INTEGER NOT NULL PRIMARY KEY,
    id_necropsia    INTEGER,
    id_forense      INTEGER,
    id_municipio    INTEGER,
    fecha_exhumacion DATE,
```

```
    motivo          VARCHAR(300),

    resultado       BLOB SUB_TYPE TEXT,

        CONSTRAINT fk_exh_nec    FOREIGN KEY (id_necropsia) REFERENCES
Necropsia(id_necropsia),

        CONSTRAINT fk_exh_for    FOREIGN KEY (id_forense) REFERENCES
Forense(id_forense),

        CONSTRAINT fk_exh_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio)

);


/*
==================================================================
===========

  8. SENTENCIAS


==================================================================
=========== */


CREATE TABLE Juzgado (

    id_juzgado     INTEGER NOT NULL PRIMARY KEY,

    id_municipio   INTEGER,

    nombre_juzgado VARCHAR(150) NOT NULL,

    tipo_juzgado   VARCHAR(80),

        CONSTRAINT fk_jz_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio)

);


CREATE TABLE Tipo_Delito_Juzgado (
```

```
    id_tipo_delito_jz INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo       VARCHAR(200) NOT NULL,
    articulo_legal    VARCHAR(100)
);

CREATE TABLE Pena (
    id_pena     INTEGER NOT NULL PRIMARY KEY,
    nombre_pena VARCHAR(100) NOT NULL
);

CREATE TABLE Sentencia (
    id_sentencia       INTEGER NOT NULL PRIMARY KEY,
    id_juzgado         INTEGER NOT NULL,
    id_tipo_delito_jz  INTEGER NOT NULL,
    id_pena            INTEGER,
    id_hecho           INTEGER,
    fecha_sentencia    DATE NOT NULL,
    anios_prision      INTEGER,
    monto_multa        DECIMAL(12,2),
    es_condenatoria    SMALLINT DEFAULT 1,
    es_firme           SMALLINT DEFAULT 0,
        CONSTRAINT fk_sen_jz FOREIGN KEY (id_juzgado) REFERENCES Juzgado(id_juzgado),
        CONSTRAINT fk_sen_tipo FOREIGN KEY (id_tipo_delito_jz) REFERENCES Tipo_Delito_Juzgado(id_tipo_delito_jz),
    CONSTRAINT fk_sen_pena  FOREIGN KEY (id_pena) REFERENCES Pena(id_pena),
```

```
        CONSTRAINT fk_sen_hecho FOREIGN KEY (id_hecho) REFERENCES Hecho_Delictivo(id_hecho)

);


CREATE TABLE Persona_Sentenciada (

   id_persona_sent   INTEGER NOT NULL PRIMARY KEY,

   id_sentencia      INTEGER NOT NULL,

   id_persona        INTEGER NOT NULL,

   alias_nombre      VARCHAR(100),

   es_reincidente    SMALLINT DEFAULT 0,

        CONSTRAINT fk_ps_sen_rel    FOREIGN KEY (id_sentencia) REFERENCES Sentencia(id_sentencia),

        CONSTRAINT fk_ps_pers_rel FOREIGN KEY (id_persona) REFERENCES Persona(id_persona)

);


CREATE TABLE Detalle_Sentencia (

   id_detalle_sent   INTEGER NOT NULL PRIMARY KEY,

   id_sentencia      INTEGER NOT NULL,

   detalle           BLOB SUB_TYPE TEXT,

   fuente_info       VARCHAR(200),

        CONSTRAINT fk_ds_sen FOREIGN KEY (id_sentencia) REFERENCES Sentencia(id_sentencia)

);


/*
==========================================================================
===========
```

9. VIOLENCIA CONTRA LA MUJER

```
==========================================================================
=========== */

CREATE TABLE Tipo_Violencia_Mujer (
   id_tipo_vm      INTEGER NOT NULL PRIMARY KEY,
   nombre_tipo_vm  VARCHAR(100) NOT NULL
);

CREATE TABLE Centro_Atencion (
   id_centro_at     INTEGER NOT NULL PRIMARY KEY,
   id_municipio     INTEGER,
   nombre_centro    VARCHAR(150) NOT NULL,
   tipo_centro      VARCHAR(100),
         CONSTRAINT fk_ca_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio)
);

CREATE TABLE Tipo_Atencion (
   id_tipo_at     INTEGER NOT NULL PRIMARY KEY,
   nombre_tipo_at VARCHAR(80) NOT NULL
);

CREATE TABLE Hecho_Violencia_Mujer (
   id_hecho_vm     INTEGER NOT NULL PRIMARY KEY,
```

```
    id_denuncia     INTEGER,

    id_tipo_vm      INTEGER NOT NULL,

    id_municipio    INTEGER NOT NULL,

    id_victima_pers INTEGER NOT NULL,

    id_agresor_pers INTEGER,

    fecha_hecho     DATE NOT NULL,

    edad_victima    INTEGER,

        CONSTRAINT fk_hvm_den    FOREIGN KEY (id_denuncia) REFERENCES
Denuncia(id_denuncia),

        CONSTRAINT fk_hvm_tvm    FOREIGN KEY (id_tipo_vm) REFERENCES
Tipo_Violencia_Mujer(id_tipo_vm),

        CONSTRAINT fk_hvm_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio),

      CONSTRAINT fk_hvm_vic_p   FOREIGN KEY (id_victima_pers) REFERENCES
Persona(id_persona),

      CONSTRAINT fk_hvm_agr_p   FOREIGN KEY (id_agresor_pers) REFERENCES
Persona(id_persona)

);


CREATE TABLE Atencion_Mujer (

    id_atencion_m   INTEGER NOT NULL PRIMARY KEY,

    id_hecho_vm     INTEGER NOT NULL,

    id_centro_at    INTEGER NOT NULL,

    id_tipo_at      INTEGER NOT NULL,

    fecha_atencion  DATE NOT NULL,

    observaciones   VARCHAR(500),

        CONSTRAINT fk_am_hvm FOREIGN KEY (id_hecho_vm) REFERENCES
Hecho_Violencia_Mujer(id_hecho_vm),
```

```
        CONSTRAINT fk_am_ca    FOREIGN KEY (id_centro_at) REFERENCES Centro_Atencion(id_centro_at),

        CONSTRAINT fk_am_ta    FOREIGN KEY (id_tipo_at) REFERENCES Tipo_Atencion(id_tipo_at)

);


/*
==================================================================================
  10. VIOLENCIA INTRAFAMILIAR

==================================================================================
*/


CREATE TABLE Tipo_VIF (

   id_tipo_vif    INTEGER NOT NULL PRIMARY KEY,

   nombre_tipo_vif VARCHAR(80) NOT NULL

);


CREATE TABLE Relacion_Agresor_Victima (

   id_rel_agr_vict INTEGER NOT NULL PRIMARY KEY,

   nombre_relacion VARCHAR(80) NOT NULL

);


CREATE TABLE Hecho_VIF (

   id_hecho_vif     INTEGER NOT NULL PRIMARY KEY,

   id_denuncia      INTEGER,

   id_tipo_vif      INTEGER NOT NULL,
```

```
    id_rel_agr_vict  INTEGER NOT NULL,

    id_municipio     INTEGER NOT NULL,

    id_victima_pers  INTEGER NOT NULL,

    id_agresor_pers  INTEGER,

    fecha_hecho      DATE NOT NULL,

    edad_victima     INTEGER,

    numero_incidentes_previos INTEGER DEFAULT 0,

        CONSTRAINT fk_hvif_den      FOREIGN KEY (id_denuncia) REFERENCES
Denuncia(id_denuncia),

        CONSTRAINT fk_hvif_tipo     FOREIGN KEY (id_tipo_vif) REFERENCES
Tipo_VIF(id_tipo_vif),

        CONSTRAINT fk_hvif_rel      FOREIGN KEY (id_rel_agr_vict) REFERENCES
Relacion_Agresor_Victima(id_rel_agr_vict),

        CONSTRAINT fk_hvif_muni     FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio),

        CONSTRAINT fk_hvif_vic_p    FOREIGN KEY (id_victima_pers) REFERENCES
Persona(id_persona),

        CONSTRAINT fk_hvif_agr_p    FOREIGN KEY (id_agresor_pers) REFERENCES
Persona(id_persona)

);


/*
=================================================================
===========

  11. VIOLENCIA INFANTIL, TRABAJO INFANTIL, EMBARAZOS


=================================================================
=========== */
```

```
CREATE TABLE Tipo_Violencia_Infantil (
    id_tipo_vi    INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo_vi VARCHAR(100) NOT NULL
);


CREATE TABLE Hecho_Violencia_Infantil (
    id_hecho_vi      INTEGER NOT NULL PRIMARY KEY,
    id_denuncia      INTEGER,
    id_tipo_vi       INTEGER NOT NULL,
    id_municipio     INTEGER NOT NULL,
    id_victima_pers  INTEGER NOT NULL,
    id_agresor_pers  INTEGER,
    fecha_hecho      DATE NOT NULL,
    edad_victima     INTEGER,
    esta_escolarizado SMALLINT DEFAULT 0,
        CONSTRAINT fk_hvi_den FOREIGN KEY (id_denuncia) REFERENCES Denuncia(id_denuncia),
        CONSTRAINT fk_hvi_tipo FOREIGN KEY (id_tipo_vi) REFERENCES Tipo_Violencia_Infantil(id_tipo_vi),
        CONSTRAINT fk_hvi_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),
        CONSTRAINT fk_hvi_vic_p FOREIGN KEY (id_victima_pers) REFERENCES Persona(id_persona),
        CONSTRAINT fk_hvi_agr_p FOREIGN KEY (id_agresor_pers) REFERENCES Persona(id_persona)
);
```

```
CREATE TABLE Sector_Trabajo_Infantil (
    id_sector_ti     INTEGER NOT NULL PRIMARY KEY,
    nombre_sector_ti VARCHAR(100) NOT NULL
);

CREATE TABLE Tipo_Trabajo_Infantil (
    id_tipo_ti      INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo_ti  VARCHAR(100) NOT NULL,
    es_peligroso    SMALLINT DEFAULT 0
);

CREATE TABLE Jornada_Trabajo_Infantil (
    id_jornada_ti      INTEGER NOT NULL PRIMARY KEY,
    nombre_jornada      VARCHAR(50) NOT NULL,
    horas_semanales    INTEGER
);

CREATE TABLE Trabajo_Infantil (
    id_trabajo_inf   INTEGER NOT NULL PRIMARY KEY,
    id_persona       INTEGER NOT NULL,
    id_sector_ti     INTEGER NOT NULL,
    id_tipo_ti       INTEGER NOT NULL,
    id_jornada_ti    INTEGER NOT NULL,
    id_municipio     INTEGER NOT NULL,
    anio             INTEGER NOT NULL,
```

```
    edad_inicio      INTEGER,

    asiste_escuela   SMALLINT DEFAULT 0,

    recibe_pago      SMALLINT DEFAULT 0,

    pago_mensual     DECIMAL(10,2),

        CONSTRAINT fk_ti_pers_base      FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),

        CONSTRAINT fk_ti_sector     FOREIGN KEY (id_sector_ti) REFERENCES Sector_Trabajo_Infantil(id_sector_ti),

        CONSTRAINT fk_ti_tipo       FOREIGN KEY (id_tipo_ti) REFERENCES Tipo_Trabajo_Infantil(id_tipo_ti),

        CONSTRAINT fk_ti_jornada FOREIGN KEY (id_jornada_ti) REFERENCES Jornada_Trabajo_Infantil(id_jornada_ti),

        CONSTRAINT fk_ti_muni       FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio)

);


CREATE TABLE Embarazo_Adolescente (

    id_embarazo_ado   INTEGER NOT NULL PRIMARY KEY,

    id_municipio      INTEGER NOT NULL,

    id_grupo_etario   INTEGER NOT NULL,

    anio              INTEGER NOT NULL,

    cantidad_casos    INTEGER NOT NULL,

    fuente            VARCHAR(100) DEFAULT 'OSAR',

        CONSTRAINT fk_ea_muni     FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),

        CONSTRAINT fk_ea_ge       FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario)

);
```

```
CREATE TABLE Control_Prenatal (
    id_control_pren  INTEGER NOT NULL PRIMARY KEY,
    id_embarazo_ado  INTEGER,
    id_persona       INTEGER,
    fecha_control    DATE,
    numero_control   INTEGER,
    observaciones    VARCHAR(500),
        CONSTRAINT fk_cp_ea      FOREIGN KEY (id_embarazo_ado) REFERENCES Embarazo_Adolescente(id_embarazo_ado),
        CONSTRAINT fk_cp_pers_base FOREIGN KEY (id_persona) REFERENCES Persona(id_persona)
);


/*
==================================================================================
  12. VIOLENCIA ESTRUCTURAL / DISCRIMINACIÓN

==================================================================================
*/


CREATE TABLE Tipo_Discriminacion (
    id_tipo_disc    INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo_disc VARCHAR(100) NOT NULL
);


CREATE TABLE Grupo_Poblacional (
```

```
    id_grupo_pob    INTEGER NOT NULL PRIMARY KEY,
    nombre_grupo    VARCHAR(100) NOT NULL
);


CREATE TABLE Hecho_Discriminacion (
    id_hecho_disc    INTEGER NOT NULL PRIMARY KEY,
    id_denuncia      INTEGER,
    id_tipo_disc     INTEGER NOT NULL,
    id_grupo_pob     INTEGER NOT NULL,
    id_municipio     INTEGER NOT NULL,
    id_victima_pers  INTEGER,
    fecha_hecho      DATE NOT NULL,
    fue_procesado    SMALLINT DEFAULT 0,
    descripcion      BLOB SUB_TYPE TEXT,
        CONSTRAINT fk_hdi_den FOREIGN KEY (id_denuncia) REFERENCES Denuncia(id_denuncia),
        CONSTRAINT fk_hdi_tipo FOREIGN KEY (id_tipo_disc) REFERENCES Tipo_Discriminacion(id_tipo_disc),
        CONSTRAINT fk_hdi_grupo FOREIGN KEY (id_grupo_pob) REFERENCES Grupo_Poblacional(id_grupo_pob),
        CONSTRAINT fk_hdi_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),
        CONSTRAINT fk_hdi_vic_p FOREIGN KEY (id_victima_pers) REFERENCES Persona(id_persona)
);
```

```
/*
==========================================================================
===========

  13. FALTAS JUDICIALES (LEVES)

==========================================================================
=========== */


CREATE TABLE Tipo_Falta_Judicial (
   id_tipo_falta    INTEGER NOT NULL PRIMARY KEY,
   nombre_tipo_falta VARCHAR(150) NOT NULL,
   articulo_legal    VARCHAR(100)
);


CREATE TABLE Sancion_Falta (
   id_sancion    INTEGER NOT NULL PRIMARY KEY,
   nombre_sancion VARCHAR(100) NOT NULL,
   monto_min     DECIMAL(10,2),
   monto_max     DECIMAL(10,2)
);


CREATE TABLE Falta_Judicial (
   id_falta_jud    INTEGER NOT NULL PRIMARY KEY,
   id_tipo_falta   INTEGER NOT NULL,
   id_sancion      INTEGER,
   id_municipio    INTEGER NOT NULL,
   id_persona      INTEGER NOT NULL,
```

```
    fecha_falta     DATE NOT NULL,

    descripcion     VARCHAR(500),

            CONSTRAINT fk_fj_tipo FOREIGN KEY (id_tipo_falta) REFERENCES
Tipo_Falta_Judicial(id_tipo_falta),

            CONSTRAINT fk_fj_san     FOREIGN KEY (id_sancion) REFERENCES
Sancion_Falta(id_sancion),

           CONSTRAINT fk_fj_muni FOREIGN KEY (id_municipio) REFERENCES
Municipio(id_municipio),

          CONSTRAINT fk_fj_pers_base FOREIGN KEY (id_persona) REFERENCES
Persona(id_persona)

);



/*
=================================================================
===========

   14. SALUD PÚBLICA


=================================================================
=========== */


CREATE TABLE Capitulo_CIE10 (

    id_cap_cie10     INTEGER NOT NULL PRIMARY KEY,

    codigo_capitulo  VARCHAR(10) NOT NULL,

    nombre_capitulo  VARCHAR(200) NOT NULL

);


CREATE TABLE CIE10 (

    id_cie10       INTEGER NOT NULL PRIMARY KEY,
```

```
    id_cap_cie10   INTEGER NOT NULL,
    codigo_cie10   VARCHAR(10) NOT NULL,
    nombre_diag    VARCHAR(250) NOT NULL,
            CONSTRAINT fk_cie_cap FOREIGN KEY (id_cap_cie10) REFERENCES Capitulo_CIE10(id_cap_cie10)
);

CREATE TABLE Categoria_Indicador (
    id_cat_indic     INTEGER NOT NULL PRIMARY KEY,
    nombre_categoria VARCHAR(100) NOT NULL
);

CREATE TABLE Indicador_Salud (
    id_indicador      INTEGER NOT NULL PRIMARY KEY,
    id_cat_indic      INTEGER NOT NULL,
    nombre_indicador  VARCHAR(150) NOT NULL,
    unidad_medida     VARCHAR(50),
            CONSTRAINT fk_ind_cat FOREIGN KEY (id_cat_indic) REFERENCES Categoria_Indicador(id_cat_indic)
);

CREATE TABLE Tipo_Centro_Salud (
    id_tipo_centro_s  INTEGER NOT NULL PRIMARY KEY,
    nombre_tipo       VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE Centro_Salud (

   id_centro_salud   INTEGER NOT NULL PRIMARY KEY,

   id_tipo_centro_s  INTEGER NOT NULL,

   id_municipio      INTEGER NOT NULL,

   nombre_centro     VARCHAR(150) NOT NULL,

   es_publico        SMALLINT DEFAULT 1,

        CONSTRAINT fk_cs_tipo FOREIGN KEY (id_tipo_centro_s) REFERENCES Tipo_Centro_Salud(id_tipo_centro_s),

        CONSTRAINT fk_cs_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio)

);


CREATE TABLE Registro_Salud (

   id_reg_salud     INTEGER NOT NULL PRIMARY KEY,

   id_indicador     INTEGER NOT NULL,

   id_municipio     INTEGER NOT NULL,

   id_grupo_etario  INTEGER NOT NULL,

   id_sexo          INTEGER NOT NULL,

   anio             INTEGER NOT NULL,

   cantidad_casos   INTEGER NOT NULL,

   fuente           VARCHAR(100) DEFAULT 'MSPAS',

        CONSTRAINT fk_rs_ind FOREIGN KEY (id_indicador) REFERENCES Indicador_Salud(id_indicador),

        CONSTRAINT fk_rs_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),

        CONSTRAINT fk_rs_ge FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario),
```

```
    CONSTRAINT fk_rs_sexo   FOREIGN KEY (id_sexo) REFERENCES Sexo(id_sexo)
);

CREATE TABLE Registro_Desnutricion (
    id_reg_desnut    INTEGER NOT NULL PRIMARY KEY,
    id_municipio     INTEGER NOT NULL,
    id_grupo_etario  INTEGER NOT NULL,
    id_sexo          INTEGER NOT NULL,
    anio             INTEGER NOT NULL,
    tipo_desnutricion VARCHAR(30) NOT NULL,
    cantidad_casos    INTEGER NOT NULL,
    CONSTRAINT fk_rd_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),
    CONSTRAINT fk_rd_ge FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario),
    CONSTRAINT fk_rd_sexo FOREIGN KEY (id_sexo) REFERENCES Sexo(id_sexo)
);

CREATE TABLE Registro_Enfermedad_Cronica (
    id_reg_cronica   INTEGER NOT NULL PRIMARY KEY,
    id_cie10         INTEGER NOT NULL,
    id_municipio     INTEGER NOT NULL,
    id_grupo_etario  INTEGER NOT NULL,
    id_sexo          INTEGER NOT NULL,
    anio             INTEGER NOT NULL,
    cantidad_casos    INTEGER NOT NULL,
```

```
    CONSTRAINT fk_rec_cie  FOREIGN KEY (id_cie10) REFERENCES CIE10(id_cie10),

    CONSTRAINT fk_rec_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),

    CONSTRAINT fk_rec_ge FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario),

    CONSTRAINT fk_rec_sexo FOREIGN KEY (id_sexo) REFERENCES Sexo(id_sexo)

);


CREATE TABLE Vector_Enfermedad (

    id_vector     INTEGER NOT NULL PRIMARY KEY,

    nombre_vector VARCHAR(100) NOT NULL,

    tipo_vector    VARCHAR(50)

);


CREATE TABLE Registro_Enfermedad_Vector (

    id_reg_vector    INTEGER NOT NULL PRIMARY KEY,

    id_vector        INTEGER,

    id_municipio      INTEGER NOT NULL,

    id_grupo_etario  INTEGER NOT NULL,

    id_sexo          INTEGER NOT NULL,

    anio             INTEGER NOT NULL,

    nombre_enfermedad VARCHAR(50) NOT NULL,

    cantidad_casos    INTEGER NOT NULL,

    CONSTRAINT fk_rv_vec FOREIGN KEY (id_vector) REFERENCES Vector_Enfermedad(id_vector),

    CONSTRAINT fk_rv_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),
```

```
    CONSTRAINT fk_rv_ge FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario),
  CONSTRAINT fk_rv_sexo FOREIGN KEY (id_sexo) REFERENCES Sexo(id_sexo)
);

CREATE TABLE Complicacion_Embarazo (
  id_complic_emb INTEGER NOT NULL PRIMARY KEY,
  nombre_complicacion VARCHAR(150) NOT NULL,
  id_cie10 INTEGER,
  CONSTRAINT fk_ce_cie FOREIGN KEY (id_cie10) REFERENCES CIE10(id_cie10)
);

CREATE TABLE Registro_Morbilidad_Materna (
  id_reg_morbi INTEGER NOT NULL PRIMARY KEY,
  id_complic_emb INTEGER NOT NULL,
  id_municipio INTEGER NOT NULL,
  id_grupo_etario INTEGER NOT NULL,
  anio INTEGER NOT NULL,
  cantidad_casos INTEGER NOT NULL,
  muertes_maternas INTEGER DEFAULT 0,
    CONSTRAINT fk_rm_ce FOREIGN KEY (id_complic_emb) REFERENCES Complicacion_Embarazo(id_complic_emb),
    CONSTRAINT fk_rm_muni FOREIGN KEY (id_municipio) REFERENCES Municipio(id_municipio),
    CONSTRAINT fk_rm_ge FOREIGN KEY (id_grupo_etario) REFERENCES Grupo_Etario(id_grupo_etario)
);
```

/*
==========
==========

15. ENCUESTAS

==========
========== */

CREATE TABLE Tipo_Encuesta (

id_tipo_enc INTEGER NOT NULL PRIMARY KEY,

nombre_tipo_enc VARCHAR(100) NOT NULL

);

CREATE TABLE Encuesta (

id_encuesta INTEGER NOT NULL PRIMARY KEY,

id_tipo_enc INTEGER NOT NULL,

id_persona INTEGER,

id_hogar INTEGER,

fecha_encuesta DATE,

CONSTRAINT fk_enc_tipo FOREIGN KEY (id_tipo_enc) REFERENCES Tipo_Encuesta(id_tipo_enc),

CONSTRAINT fk_enc_pers_base FOREIGN KEY (id_persona) REFERENCES Persona(id_persona),

CONSTRAINT fk_enc_hog FOREIGN KEY (id_hogar) REFERENCES Hogar(id_hogar)

);

```
CREATE TABLE Respuesta_Encuesta (
   id_resp_encuesta INTEGER NOT NULL PRIMARY KEY,
   id_encuesta      INTEGER NOT NULL,
   pregunta         VARCHAR(250) NOT NULL,
   respuesta        VARCHAR(500),
            CONSTRAINT fk_re_enc FOREIGN KEY (id_encuesta) REFERENCES Encuesta(id_encuesta)
);

COMMIT;

/*
==================================================================
===========
  16. TRIGGERS BEFORE INSERT (autoincrement)

==================================================================
=========== */

SET TERM ^ ;

CREATE TRIGGER trg_pais_bi FOR Pais ACTIVE BEFORE INSERT POSITION 0 AS
BEGIN IF (NEW.id_pais IS NULL) THEN NEW.id_pais = NEXT VALUE FOR seq_pais; END^

CREATE TRIGGER trg_region_bi FOR Region ACTIVE BEFORE INSERT POSITION 0 AS
```

```
BEGIN IF (NEW.id_region IS NULL) THEN NEW.id_region = NEXT VALUE FOR seq_region; END^


CREATE TRIGGER trg_depto_bi FOR Departamento ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_departamento IS NULL) THEN NEW.id_departamento = NEXT VALUE FOR seq_departamento; END^


CREATE TRIGGER trg_muni_bi FOR Municipio ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_municipio IS NULL) THEN NEW.id_municipio = NEXT VALUE FOR seq_municipio; END^


CREATE TRIGGER trg_area_bi FOR Area_Geografica ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_area_geo IS NULL) THEN NEW.id_area_geo = NEXT VALUE FOR seq_area_geo; END^


CREATE TRIGGER trg_sexo_bi FOR Sexo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_sexo IS NULL) THEN NEW.id_sexo = NEXT VALUE FOR seq_sexo; END^


CREATE TRIGGER trg_etnia_bi FOR Etnia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_etnia IS NULL) THEN NEW.id_etnia = NEXT VALUE FOR seq_etnia; END^


CREATE TRIGGER trg_idioma_bi FOR Idioma ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_idioma IS NULL) THEN NEW.id_idioma = NEXT VALUE FOR seq_idioma; END^
```

```
CREATE TRIGGER trg_ec_bi FOR Estado_Civil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_estado_civil IS NULL) THEN NEW.id_estado_civil = NEXT VALUE FOR seq_estado_civil; END^


CREATE TRIGGER trg_rel_bi FOR Religion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_religion IS NULL) THEN NEW.id_religion = NEXT VALUE FOR seq_religion; END^


CREATE TRIGGER trg_disc_bi FOR Discapacidad ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_discapacidad IS NULL) THEN NEW.id_discapacidad = NEXT VALUE FOR seq_discapacidad; END^


CREATE TRIGGER trg_ge_bi FOR Grupo_Etario ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_grupo_etario IS NULL) THEN NEW.id_grupo_etario = NEXT VALUE FOR seq_grupo_etario; END^


CREATE TRIGGER trg_ne_bi FOR Nivel_Educativo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_nivel_educativo IS NULL) THEN NEW.id_nivel_educativo = NEXT VALUE FOR seq_nivel_educativo; END^


CREATE TRIGGER trg_ie_bi FOR Institucion_Educativa ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_institucion_edu IS NULL) THEN NEW.id_institucion_edu = NEXT VALUE FOR seq_institucion_edu; END^
```

```
CREATE TRIGGER trg_tv_bi FOR Tipo_Vivienda ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_vivienda IS NULL) THEN NEW.id_tipo_vivienda = NEXT VALUE FOR seq_tipo_vivienda; END^


CREATE TRIGGER trg_ns_bi FOR Nivel_Socioeconomico ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_nivel_socio IS NULL) THEN NEW.id_nivel_socio = NEXT VALUE FOR seq_nivel_socio; END^


CREATE TRIGGER trg_se_bi FOR Sector_Economico ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_sector_econ IS NULL) THEN NEW.id_sector_econ = NEXT VALUE FOR seq_sector_econ; END^


CREATE TRIGGER trg_pers_bi FOR Persona ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona IS NULL) THEN NEW.id_persona = NEXT VALUE FOR seq_persona; END^


CREATE TRIGGER trg_pi_bi FOR Persona_Idioma ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona_idioma IS NULL) THEN NEW.id_persona_idioma = NEXT VALUE FOR seq_persona_idioma; END^


CREATE TRIGGER trg_pd_bi FOR Persona_Discapacidad ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona_disc IS NULL) THEN NEW.id_persona_disc = NEXT VALUE FOR seq_persona_disc; END^
```

```
CREATE TRIGGER trg_dom_bi FOR Domicilio ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_domicilio IS NULL) THEN NEW.id_domicilio = NEXT VALUE FOR seq_domicilio; END^


CREATE TRIGGER trg_cont_bi FOR Contacto ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_contacto IS NULL) THEN NEW.id_contacto = NEXT VALUE FOR seq_contacto; END^


CREATE TRIGGER trg_esc_bi FOR Escolaridad_Persona ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_escolaridad IS NULL) THEN NEW.id_escolaridad = NEXT VALUE FOR seq_escolaridad; END^


CREATE TRIGGER trg_ocup_bi FOR Ocupacion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_ocupacion IS NULL) THEN NEW.id_ocupacion = NEXT VALUE FOR seq_ocupacion; END^


CREATE TRIGGER trg_te_bi FOR Tipo_Empleo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_empleo IS NULL) THEN NEW.id_tipo_empleo = NEXT VALUE FOR seq_tipo_empleo; END^


CREATE TRIGGER trg_cl_bi FOR Condicion_Laboral ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_cond_laboral IS NULL) THEN NEW.id_cond_laboral = NEXT VALUE FOR seq_cond_laboral; END^
```

```
CREATE TRIGGER trg_emp_bi FOR Empleo_Persona ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_empleo IS NULL) THEN NEW.id_empleo = NEXT VALUE FOR seq_empleo; END^


CREATE TRIGGER trg_hog_bi FOR Hogar ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hogar IS NULL) THEN NEW.id_hogar = NEXT VALUE FOR seq_hogar; END^


CREATE TRIGGER trg_ph_bi FOR Persona_Hogar ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona_hogar IS NULL) THEN NEW.id_persona_hogar = NEXT VALUE FOR seq_persona_hogar; END^


CREATE TRIGGER trg_ev_bi FOR Eje_Violencia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_eje_violencia IS NULL) THEN NEW.id_eje_violencia = NEXT VALUE FOR seq_eje_violencia; END^


CREATE TRIGGER trg_td_bi FOR Tipo_Denuncia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_denuncia IS NULL) THEN NEW.id_tipo_denuncia = NEXT VALUE FOR seq_tipo_denuncia; END^


CREATE TRIGGER trg_ir_bi FOR Institucion_Reportante ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_inst_reporte IS NULL) THEN NEW.id_inst_reporte = NEXT VALUE FOR seq_inst_reporte; END^
```

```
CREATE TRIGGER trg_ed_bi FOR Estado_Denuncia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_estado_denuncia IS NULL) THEN NEW.id_estado_denuncia = NEXT VALUE FOR seq_estado_denuncia; END^


CREATE TRIGGER trg_den_bi FOR Denuncia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_denuncia IS NULL) THEN NEW.id_denuncia = NEXT VALUE FOR seq_denuncia; END^


CREATE TRIGGER trg_pden_bi FOR Persona_Denuncia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona_denuncia IS NULL) THEN NEW.id_persona_denuncia = NEXT VALUE FOR seq_persona_denuncia; END^


CREATE TRIGGER trg_sd_bi FOR Seguimiento_Denuncia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_seguim_denuncia IS NULL) THEN NEW.id_seguim_denuncia = NEXT VALUE FOR seq_seguim_denuncia; END^


CREATE TRIGGER trg_cd_bi FOR Clasificacion_Delito ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_clasif_delito IS NULL) THEN NEW.id_clasif_delito = NEXT VALUE FOR seq_clasif_delito; END^


CREATE TRIGGER trg_th_bi FOR Tipo_Hecho_Delictivo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_hecho IS NULL) THEN NEW.id_tipo_hecho = NEXT VALUE FOR seq_tipo_hecho; END^
```

```
CREATE TRIGGER trg_lo_bi FOR Lugar_Ocurrencia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_lugar_ocurr IS NULL) THEN NEW.id_lugar_ocurr = NEXT VALUE FOR seq_lugar_ocurr; END^


CREATE TRIGGER trg_hecho_bi FOR Hecho_Delictivo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho IS NULL) THEN NEW.id_hecho = NEXT VALUE FOR seq_hecho; END^


CREATE TRIGGER trg_arma_bi FOR Arma_Utilizada ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_arma IS NULL) THEN NEW.id_arma = NEXT VALUE FOR seq_arma; END^


CREATE TRIGGER trg_ha_bi FOR Hecho_Arma ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_arma IS NULL) THEN NEW.id_hecho_arma = NEXT VALUE FOR seq_hecho_arma; END^


CREATE TRIGGER trg_mov_bi FOR Movil_Delito ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_movil IS NULL) THEN NEW.id_movil = NEXT VALUE FOR seq_movil; END^


CREATE TRIGGER trg_hm_bi FOR Hecho_Movil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_movil IS NULL) THEN NEW.id_hecho_movil = NEXT VALUE FOR seq_hecho_movil; END^
```

```
CREATE TRIGGER trg_vic_bi FOR Victima ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_victima IS NULL) THEN NEW.id_victima = NEXT VALUE FOR seq_victima; END^


CREATE TRIGGER trg_agr_bi FOR Agresor ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_agresor IS NULL) THEN NEW.id_agresor = NEXT VALUE FOR seq_agresor; END^


CREATE TRIGGER trg_lab_bi FOR Laboratorio_Forense ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_lab_forense IS NULL) THEN NEW.id_lab_forense = NEXT VALUE FOR seq_lab_forense; END^


CREATE TRIGGER trg_for_bi FOR Forense ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_forense IS NULL) THEN NEW.id_forense = NEXT VALUE FOR seq_forense; END^


CREATE TRIGGER trg_nec_bi FOR Necropsia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_necropsia IS NULL) THEN NEW.id_necropsia = NEXT VALUE FOR seq_necropsia; END^


CREATE TRIGGER trg_exh_bi FOR Exhumacion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_exhumacion IS NULL) THEN NEW.id_exhumacion = NEXT VALUE FOR seq_exhumacion; END^


CREATE TRIGGER trg_jz_bi FOR Juzgado ACTIVE BEFORE INSERT POSITION 0 AS
```

BEGIN IF (NEW.id_juzgado IS NULL) THEN NEW.id_juzgado = NEXT VALUE FOR seq_juzgado; END^

CREATE TRIGGER trg_tdj_bi FOR Tipo_Delito_Juzgado ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_delito_jz IS NULL) THEN NEW.id_tipo_delito_jz = NEXT VALUE FOR seq_tipo_delito_jz; END^

CREATE TRIGGER trg_pena_bi FOR Pena ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_pena IS NULL) THEN NEW.id_pena = NEXT VALUE FOR seq_pena; END^

CREATE TRIGGER trg_sen_bi FOR Sentencia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_sentencia IS NULL) THEN NEW.id_sentencia = NEXT VALUE FOR seq_sentencia; END^

CREATE TRIGGER trg_ps_bi FOR Persona_Sentenciada ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_persona_sent IS NULL) THEN NEW.id_persona_sent = NEXT VALUE FOR seq_persona_sent; END^

CREATE TRIGGER trg_ds_bi FOR Detalle_Sentencia ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_detalle_sent IS NULL) THEN NEW.id_detalle_sent = NEXT VALUE FOR seq_detalle_sent; END^

CREATE TRIGGER trg_tvm_bi FOR Tipo_Violencia_Mujer ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_vm IS NULL) THEN NEW.id_tipo_vm = NEXT VALUE FOR seq_tipo_vm; END^

CREATE TRIGGER trg_ca_bi FOR Centro_Atencion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_centro_at IS NULL) THEN NEW.id_centro_at = NEXT VALUE FOR seq_centro_at; END^

CREATE TRIGGER trg_ta_bi FOR Tipo_Atencion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_at IS NULL) THEN NEW.id_tipo_at = NEXT VALUE FOR seq_tipo_at; END^

CREATE TRIGGER trg_hvm_bi FOR Hecho_Violencia_Mujer ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_vm IS NULL) THEN NEW.id_hecho_vm = NEXT VALUE FOR seq_hecho_vm; END^

CREATE TRIGGER trg_am_bi FOR Atencion_Mujer ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_atencion_m IS NULL) THEN NEW.id_atencion_m = NEXT VALUE FOR seq_atencion_m; END^

CREATE TRIGGER trg_tvif_bi FOR Tipo_VIF ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_vif IS NULL) THEN NEW.id_tipo_vif = NEXT VALUE FOR seq_tipo_vif; END^

CREATE TRIGGER trg_rav_bi FOR Relacion_Agresor_Victima ACTIVE BEFORE INSERT POSITION 0 AS

```
BEGIN IF (NEW.id_rel_agr_vict IS NULL) THEN NEW.id_rel_agr_vict = NEXT VALUE FOR seq_rel_agr_vict; END^


CREATE TRIGGER trg_hvif_bi FOR Hecho_VIF ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_vif IS NULL) THEN NEW.id_hecho_vif = NEXT VALUE FOR seq_hecho_vif; END^


CREATE TRIGGER trg_tvi_bi FOR Tipo_Violencia_Infantil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_vi IS NULL) THEN NEW.id_tipo_vi = NEXT VALUE FOR seq_tipo_vi; END^


CREATE TRIGGER trg_hvi_bi FOR Hecho_Violencia_Infantil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_vi IS NULL) THEN NEW.id_hecho_vi = NEXT VALUE FOR seq_hecho_vi; END^


CREATE TRIGGER trg_sti_bi FOR Sector_Trabajo_Infantil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_sector_ti IS NULL) THEN NEW.id_sector_ti = NEXT VALUE FOR seq_sector_ti; END^


CREATE TRIGGER trg_tti_bi FOR Tipo_Trabajo_Infantil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_ti IS NULL) THEN NEW.id_tipo_ti = NEXT VALUE FOR seq_tipo_ti; END^


CREATE TRIGGER trg_jti_bi FOR Jornada_Trabajo_Infantil ACTIVE BEFORE INSERT POSITION 0 AS
```

BEGIN IF (NEW.id_jornada_ti IS NULL) THEN NEW.id_jornada_ti = NEXT VALUE FOR seq_jornada_ti; END^

CREATE TRIGGER trg_ti_bi FOR Trabajo_Infantil ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_trabajo_inf IS NULL) THEN NEW.id_trabajo_inf = NEXT VALUE FOR seq_trabajo_inf; END^

CREATE TRIGGER trg_ea_bi FOR Embarazo_Adolescente ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_embarazo_ado IS NULL) THEN NEW.id_embarazo_ado = NEXT VALUE FOR seq_embarazo_ado; END^

CREATE TRIGGER trg_cp_bi FOR Control_Prenatal ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_control_pren IS NULL) THEN NEW.id_control_pren = NEXT VALUE FOR seq_control_pren; END^

CREATE TRIGGER trg_tdi_bi FOR Tipo_Discriminacion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_disc IS NULL) THEN NEW.id_tipo_disc = NEXT VALUE FOR seq_tipo_disc; END^

CREATE TRIGGER trg_gp_bi FOR Grupo_Poblacional ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_grupo_pob IS NULL) THEN NEW.id_grupo_pob = NEXT VALUE FOR seq_grupo_pob; END^

CREATE TRIGGER trg_hdi_bi FOR Hecho_Discriminacion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_hecho_disc IS NULL) THEN NEW.id_hecho_disc = NEXT VALUE FOR seq_hecho_disc; END^

CREATE TRIGGER trg_tf_bi FOR Tipo_Falta_Judicial ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_falta IS NULL) THEN NEW.id_tipo_falta = NEXT VALUE FOR seq_tipo_falta; END^

CREATE TRIGGER trg_san_bi FOR Sancion_Falta ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_sancion IS NULL) THEN NEW.id_sancion = NEXT VALUE FOR seq_sancion; END^

CREATE TRIGGER trg_fj_bi FOR Falta_Judicial ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_falta_jud IS NULL) THEN NEW.id_falta_jud = NEXT VALUE FOR seq_falta_jud; END^

CREATE TRIGGER trg_cap_bi FOR Capitulo_CIE10 ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_cap_cie10 IS NULL) THEN NEW.id_cap_cie10 = NEXT VALUE FOR seq_cap_cie10; END^

CREATE TRIGGER trg_cie_bi FOR CIE10 ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_cie10 IS NULL) THEN NEW.id_cie10 = NEXT VALUE FOR seq_cie10; END^

CREATE TRIGGER trg_ci_bi FOR Categoria_Indicador ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_cat_indic IS NULL) THEN NEW.id_cat_indic = NEXT VALUE FOR seq_cat_indic; END^

CREATE TRIGGER trg_ind_bi FOR Indicador_Salud ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_indicador IS NULL) THEN NEW.id_indicador = NEXT VALUE FOR seq_indicador; END^

CREATE TRIGGER trg_tcs_bi FOR Tipo_Centro_Salud ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_centro_s IS NULL) THEN NEW.id_tipo_centro_s = NEXT VALUE FOR seq_tipo_centro_s; END^

CREATE TRIGGER trg_cs_bi FOR Centro_Salud ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_centro_salud IS NULL) THEN NEW.id_centro_salud = NEXT VALUE FOR seq_centro_salud; END^

CREATE TRIGGER trg_rs_bi FOR Registro_Salud ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_reg_salud IS NULL) THEN NEW.id_reg_salud = NEXT VALUE FOR seq_reg_salud; END^

CREATE TRIGGER trg_rd_bi FOR Registro_Desnutricion ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_reg_desnut IS NULL) THEN NEW.id_reg_desnut = NEXT VALUE FOR seq_reg_desnut; END^

CREATE TRIGGER trg_rec_bi FOR Registro_Enfermedad_Cronica ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_reg_cronica IS NULL) THEN NEW.id_reg_cronica = NEXT VALUE FOR seq_reg_cronica; END^

CREATE TRIGGER trg_vec_bi FOR Vector_Enfermedad ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_vector IS NULL) THEN NEW.id_vector = NEXT VALUE FOR seq_vector; END^

CREATE TRIGGER trg_rv_bi FOR Registro_Enfermedad_Vector ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_reg_vector IS NULL) THEN NEW.id_reg_vector = NEXT VALUE FOR seq_reg_vector; END^

CREATE TRIGGER trg_ce_bi FOR Complicacion_Embarazo ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_complic_emb IS NULL) THEN NEW.id_complic_emb = NEXT VALUE FOR seq_complic_emb; END^

CREATE TRIGGER trg_rm_bi FOR Registro_Morbilidad_Materna ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_reg_morbi IS NULL) THEN NEW.id_reg_morbi = NEXT VALUE FOR seq_reg_morbi; END^

CREATE TRIGGER trg_tenc_bi FOR Tipo_Encuesta ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_tipo_enc IS NULL) THEN NEW.id_tipo_enc = NEXT VALUE FOR seq_tipo_enc; END^

CREATE TRIGGER trg_enc_bi FOR Encuesta ACTIVE BEFORE INSERT POSITION 0 AS

```
BEGIN IF (NEW.id_encuesta IS NULL) THEN NEW.id_encuesta = NEXT VALUE FOR seq_encuesta; END^


CREATE TRIGGER trg_re_bi FOR Respuesta_Encuesta ACTIVE BEFORE INSERT POSITION 0 AS

BEGIN IF (NEW.id_resp_encuesta IS NULL) THEN NEW.id_resp_encuesta = NEXT VALUE FOR seq_resp_encuesta; END^


SET TERM ; ^


COMMIT;


/*
==================================================================
===========

  17. ÍNDICES SECUNDARIOS (mejoran consultas frecuentes)

==================================================================
=========== */

CREATE INDEX idx_hecho_fecha      ON Hecho_Delictivo(fecha_hecho);

CREATE INDEX idx_hecho_muni       ON Hecho_Delictivo(id_municipio);

CREATE INDEX idx_hecho_tipo       ON Hecho_Delictivo(id_tipo_hecho);

CREATE INDEX idx_den_fecha        ON Denuncia(fecha_denuncia);

CREATE INDEX idx_den_muni         ON Denuncia(id_municipio);

CREATE INDEX idx_sen_fecha        ON Sentencia(fecha_sentencia);

CREATE INDEX idx_rs_anio          ON Registro_Salud(anio);

CREATE INDEX idx_rs_muni          ON Registro_Salud(id_municipio);

CREATE INDEX idx_rv_anio          ON Registro_Enfermedad_Vector(anio);
```

```
CREATE INDEX idx_rd_anio        ON Registro_Desnutricion(anio);

CREATE INDEX idx_ea_anio        ON Embarazo_Adolescente(anio);

CREATE INDEX idx_ti_anio        ON Trabajo_Infantil(anio);

CREATE INDEX idx_hvm_fecha      ON Hecho_Violencia_Mujer(fecha_hecho);

CREATE INDEX idx_hvif_fecha     ON Hecho_VIF(fecha_hecho);


COMMIT;
```

## Consultas SQL + Álgebra Relacional

### Símbolos de Referencia

- Selección: **σ**
- Proyección: **π**
- Join Natural: ⋈
- Left Join: ⟕
- Agregación/Grupo: **γ**
- Ordenamiento: **τ**
- Renombrado: **ρ**
- Unión: ∪

### 1. Selección (σ - Sigma)

Funciona como un filtro. Se encarga de extraer solo las filas que cumplen con una condición específica (por ejemplo, personas mayores de 18 años).

*Ejemplo: σ edad > 18 (Usuarios)*

### 2. Proyección (π - Pi)

Se encarga de elegir columnas específicas. Si solo necesitas los nombres y correos de una tabla gigante, usas proyección para descartar el resto de la información.

*Ejemplo: π nombre, email (Usuarios)*

### 3. Join Natural (⋈ - Corbatín)

Une dos tablas basándose en las columnas que tienen en común. Es la forma automática de combinar información relacionada (por ejemplo, unir "Ventas" con "Productos" usando el ID del producto).

### 4. Left Join (⟕ - Join Izquierdo)

Une dos tablas pero asegura que todas las filas de la tabla de la izquierda aparezcan, incluso si no encuentran una pareja en la tabla de la derecha (en ese caso, rellena con NULL).

### 5. Agregación / Agrupamiento (γ - Gamma)

Se usa para hacer cálculos estadísticos. Agrupa datos iguales y aplica funciones como sumar (SUM), contar (COUNT) o promediar (AVG). Es vital para reportes.

*Ejemplo: γ ciudad; COUNT(id) (Usuarios)*

**6. Ordenamiento (τ - Tau)**

Simplemente organiza las filas en un orden específico, ya sea de forma ascendente (↑) o descendente (↓) basándose en una o más columnas.

**7. Renombrado (ρ - Rho)**

Cambia el nombre de una tabla o de una columna temporalmente. Es muy útil cuando quieres unir una tabla consigo misma o cuando el nombre original es confuso.

**8. Unión (∪)**

Combina los resultados de dos consultas en una sola lista, siempre y cuando ambas tengan las mismas columnas. Une lo que salió de la "Bolsa A" con lo de la "Bolsa B".

**1. Cantidad de homicidios por año y departamento**

SQL

```
SELECT EXTRACT(YEAR FROM H.FECHA_HECHO) AS ANIO,
    D.NOMBRE_DEPARTAMENTO,
    COUNT(H.ID_HECHO) AS TOTAL_HOMICIDIOS
FROM HECHO_DELICTIVO H
JOIN TIPO_HECHO_DELICTIVO TH ON H.ID_TIPO_HECHO = TH.ID_TIPO_HECHO
JOIN MUNICIPIO          M  ON H.ID_MUNICIPIO = M.ID_MUNICIPIO
JOIN DEPARTAMENTO       D  ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO
WHERE UPPER(TH.NOMBRE_TIPO) LIKE '%HOMICIDIO%'
GROUP BY EXTRACT(YEAR FROM H.FECHA_HECHO), D.NOMBRE_DEPARTAMENTO
ORDER BY 1 DESC, 3 DESC;
```

**Álgebra Relacional** τ ANIO↓, TOTAL↓ ( γ ANIO, NOMBRE_DEPARTAMENTO; COUNT(ID_HECHO) → TOTAL ( σ NOMBRE_TIPO ~ 'Homicidio' ( Hecho_Delictivo ⋈ Tipo_Hecho_Delictivo ⋈ Municipio ⋈ Departamento ) ) )

sel HECHO_DELICTIVO

File Edit View History Statement Grid

```sql
SELECT EXTRACT(YEAR FROM H.FECHA_HECHO) AS ANIO,
       D.NOMBRE_DEPARTAMENTO,
       COUNT(H.ID_HECHO) AS TOTAL_HOMICIDIOS
FROM HECHO_DELICTIVO H
JOIN TIPO_HECHO_DELICTIVO TH ON H.ID_TIPO_HECHO = TH.ID_TIPO_HECHO
JOIN MUNICIPIO           M  ON H.ID_MUNICIPIO = M.ID_MUNICIPIO
JOIN DEPARTAMENTO        D  ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO
WHERE UPPER(TH.NOMBRE_TIPO) LIKE '%HOMICIDIO%'
GROUP BY EXTRACT(YEAR FROM H.FECHA_HECHO), D.NOMBRE_DEPARTAMENTO
ORDER BY 1 DESC, 3 DESC;
```

Statistics Data

| | ANIO | NOMBRE_DEPARTAMENTO | TOTAL_HOMICIDIOS |
|---|---|---|---|
| 117 | 2021 | Escuintla | 22 |
| 118 | 2021 | Guatemala | 21 |
| 119 | 2021 | Chimaltenango | 21 |
| 120 | 2021 | Quetzaltenango | 20 |
| 121 | 2021 | Jutiapa | 19 |
| 122 | 2021 | Alta Verapaz | 18 |
| 123 | 2021 | Zacapa | 16 |
| 124 | 2021 | Santa Rosa | 16 |
| 125 | 2021 | Petén | 14 |
| 126 | 2021 | Baja Verapaz | 12 |
| 127 | 2021 | Chiquimula | 11 |
| 128 | 2021 | Totonicapán | 11 |
| 129 | 2021 | Retalhuleu | 9 |
| 130 | 2021 | Jalapa | 7 |
| 131 | 2021 | El Progreso | 7 |
| 132 | 2021 | Izabal | 5 |

SYSDBA@localhost/3050:/var/lib/firebird/data/proyecto 2.... 132 row(s) f... 1 : 1 Transaction started

**2. Número de denuncias por violencia contra la mujer por municipio**

**SQL**

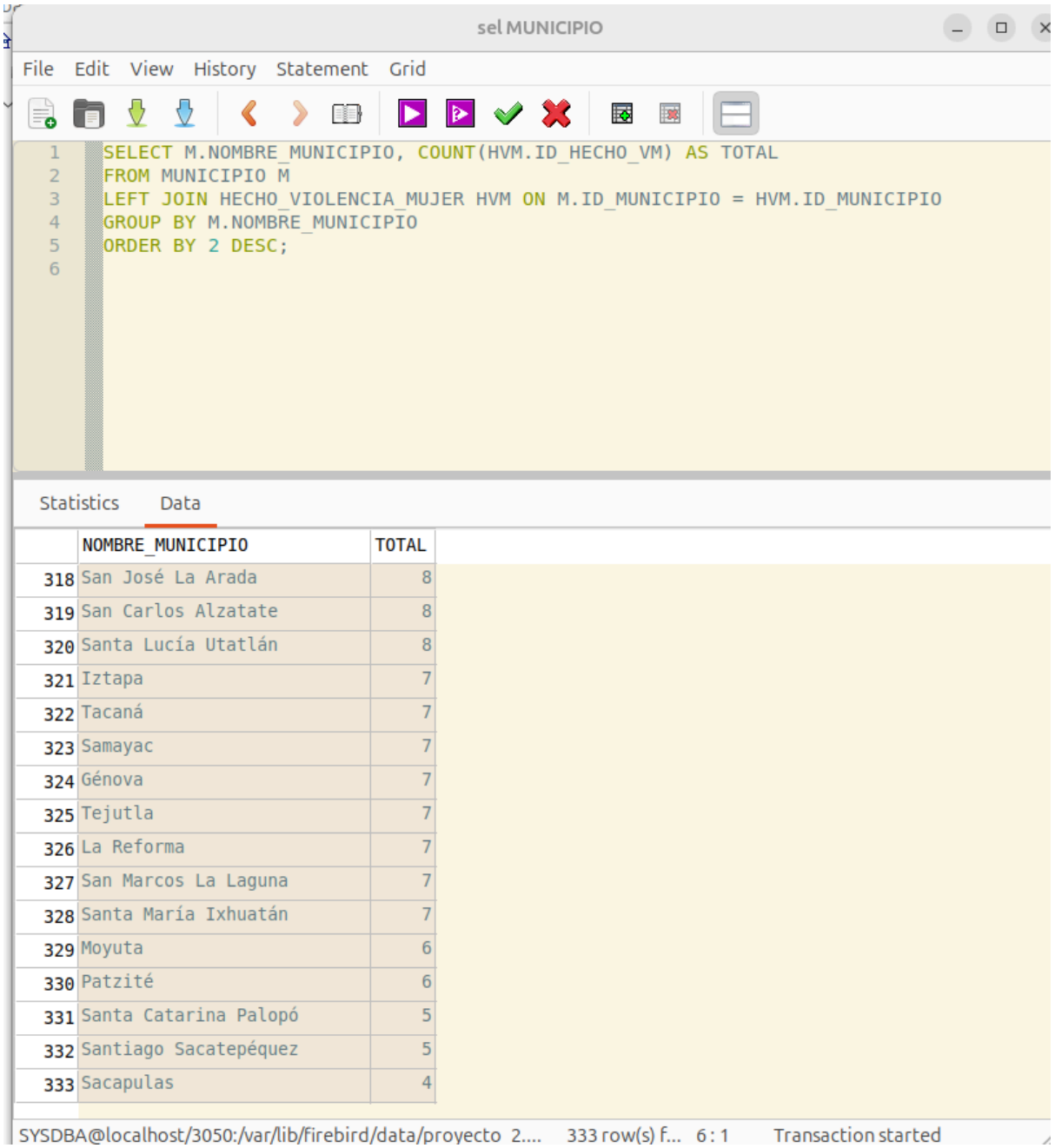
SELECT M.NOMBRE_MUNICIPIO, COUNT(HVM.ID_HECHO_VM) AS TOTAL

FROM MUNICIPIO M

LEFT JOIN HECHO_VIOLENCIA_MUJER HVM ON M.ID_MUNICIPIO = HVM.ID_MUNICIPIO

GROUP BY M.NOMBRE_MUNICIPIO

ORDER BY 2 DESC;




| | NOMBRE_MUNICIPIO | TOTAL |
|---|---|---|
| 318 | San José La Arada | 8 |
| 319 | San Carlos Alzatate | 8 |
| 320 | Santa Lucía Utatlán | 8 |
| 321 | Iztapa | 7 |
| 322 | Tacaná | 7 |
| 323 | Samayac | 7 |
| 324 | Génova | 7 |
| 325 | Tejutla | 7 |
| 326 | La Reforma | 7 |
| 327 | San Marcos La Laguna | 7 |
| 328 | Santa María Ixhuatán | 7 |
| 329 | Moyuta | 6 |
| 330 | Patzité | 6 |
| 331 | Santa Catarina Palopó | 5 |
| 332 | Santiago Sacatepéquez | 5 |
| 333 | Sacapulas | 4 |

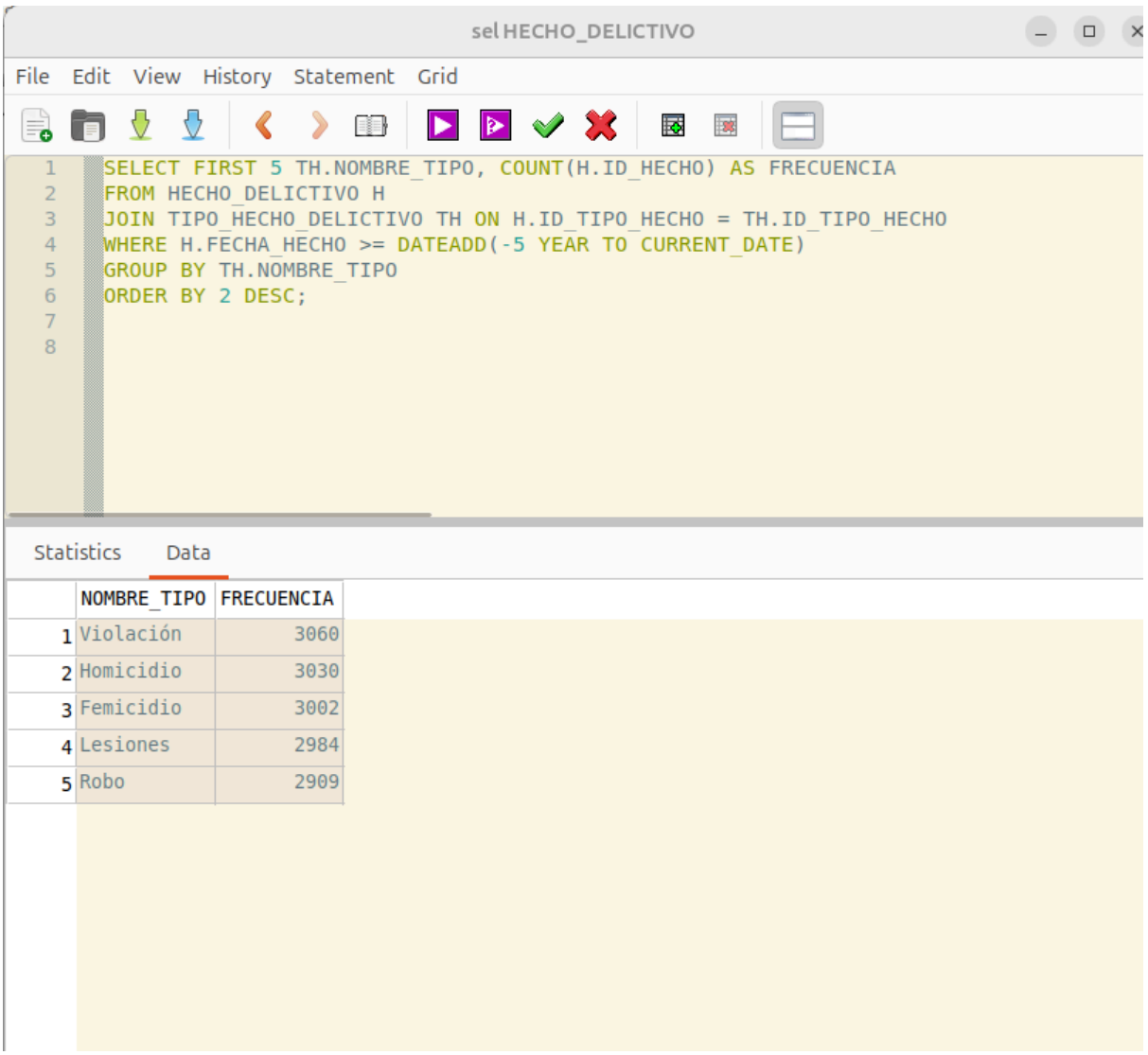
**Álgebra Relacional** $\tau$ TOTAL$\downarrow$ ( $\gamma$ NOMBRE_MUNICIPIO; COUNT(ID_HECHO_VM) $\rightarrow$ TOTAL ( Municipio $\bowtie$ Hecho_Violencia_Mujer ) )

**3. Top 5 tipos de hechos delictivos más frecuentes (últimos 5 años)**

**SQL**

```
SELECT FIRST 5 TH.NOMBRE_TIPO, COUNT(H.ID_HECHO) AS FRECUENCIA

FROM HECHO_DELICTIVO H

JOIN TIPO_HECHO_DELICTIVO TH ON H.ID_TIPO_HECHO = TH.ID_TIPO_HECHO

WHERE H.FECHA_HECHO >= DATEADD(-5 YEAR TO CURRENT_DATE)

GROUP BY TH.NOMBRE_TIPO

ORDER BY 2 DESC;
```

**Álgebra Relacional** TOP 5 ( $\tau$ FRECUENCIA$\downarrow$ ( $\gamma$ NOMBRE_TIPO; COUNT(ID_HECHO) $\rightarrow$ FRECUENCIA ( $\sigma$ FECHA_HECHO $\geq$ CURRENT_DATE - 5y ( Hecho_Delictivo $\bowtie$ Tipo_Hecho_Delictivo ) ) ) )




**4. Sentencias dictadas por tipo de delito y año**

**SQL**

SELECT EXTRACT(YEAR FROM S.FECHA_SENTENCIA) AS ANIO,

TDJ.NOMBRE_TIPO AS TIPO_DELITO,

COUNT(S.ID_SENTENCIA) AS TOTAL_SENTENCIAS

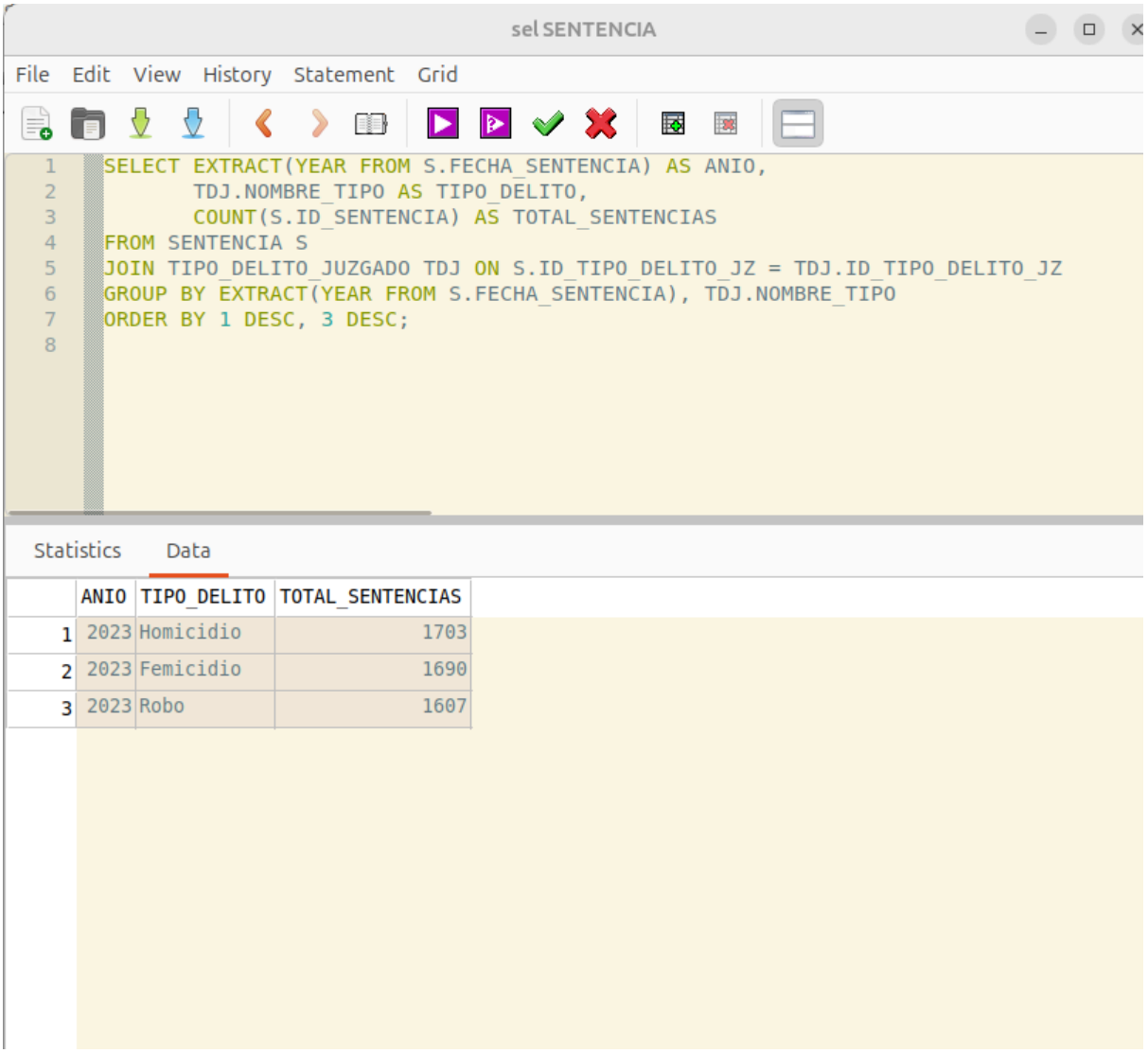
FROM SENTENCIA S

JOIN TIPO_DELITO_JUZGADO TDJ ON S.ID_TIPO_DELITO_JZ = TDJ.ID_TIPO_DELITO_JZ

GROUP BY EXTRACT(YEAR FROM S.FECHA_SENTENCIA), TDJ.NOMBRE_TIPO

ORDER BY 1 DESC, 3 DESC;

**Álgebra Relacional** γ ANIO, NOMBRE_TIPO; COUNT(ID_SENTENCIA) → TOTAL ( Sentencia ⋈ Tipo_Delito_Juzgado )




**5. Promedio de edad de las víctimas de violencia intrafamiliar**

**SQL**

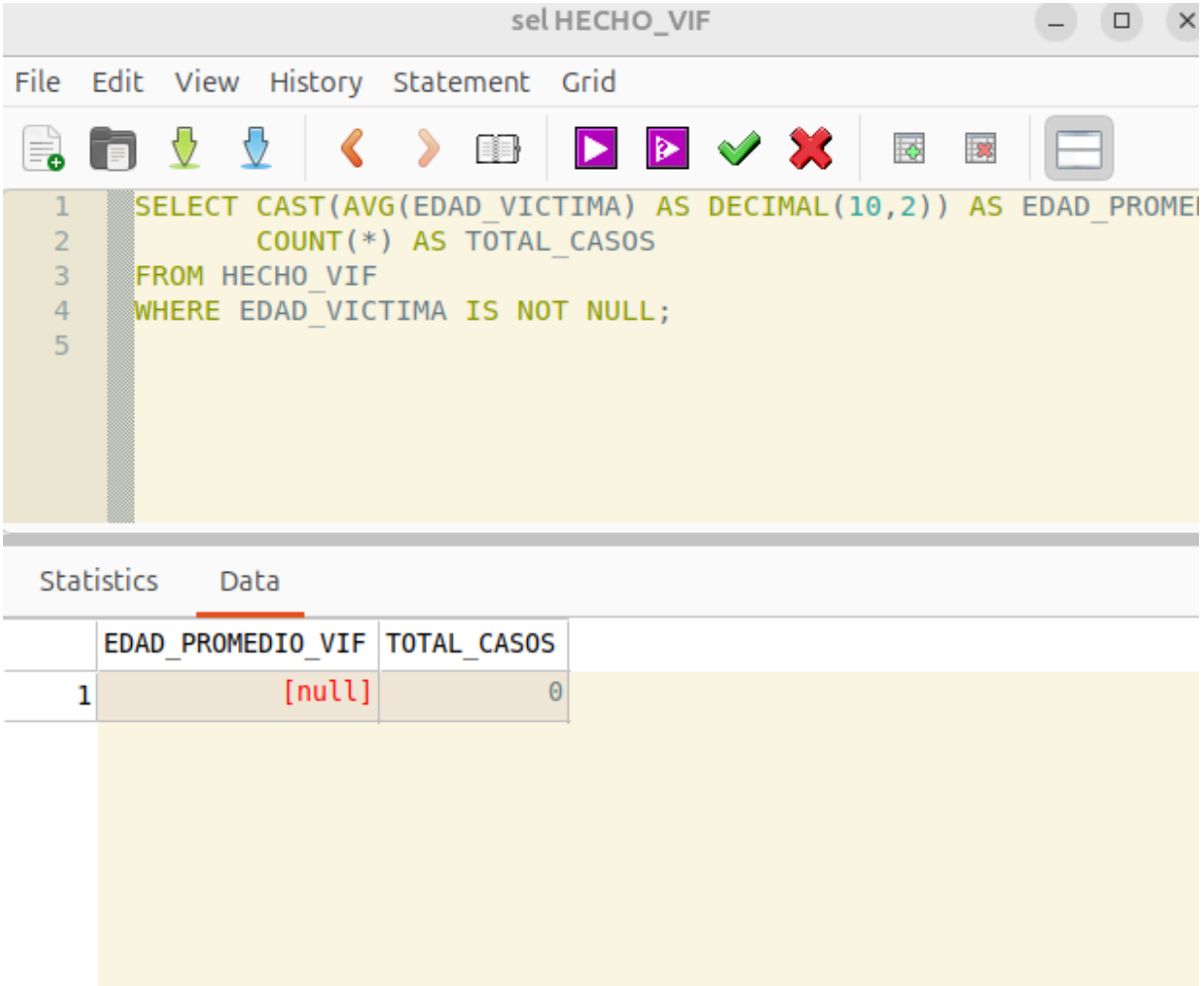
SELECT CAST(AVG(EDAD_VICTIMA) AS DECIMAL(10,2)) AS EDAD_PROMEDIO_VIF,

COUNT(*) AS TOTAL_CASOS

FROM HECHO_VIF

WHERE EDAD_VICTIMA IS NOT NULL;

**Álgebra Relacional** γ ; AVG(EDAD_VICTIMA) → PROMEDIO, COUNT(*) → TOTAL ( σ EDAD_VICTIMA ≠ NULL ( Hecho_VIF ) )




**6. Embarazos adolescentes (< 19 años) por región**

**SQL**

SELECT R.NOMBRE_REGION, SUM(EA.CANTIDAD_CASOS) AS TOTAL_EMBARAZOS

FROM EMBARAZO_ADOLESCENTE EA

JOIN GRUPO_ETARIO GE ON EA.ID_GRUPO_ETARIO = GE.ID_GRUPO_ETARIO

JOIN MUNICIPIO M ON EA.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

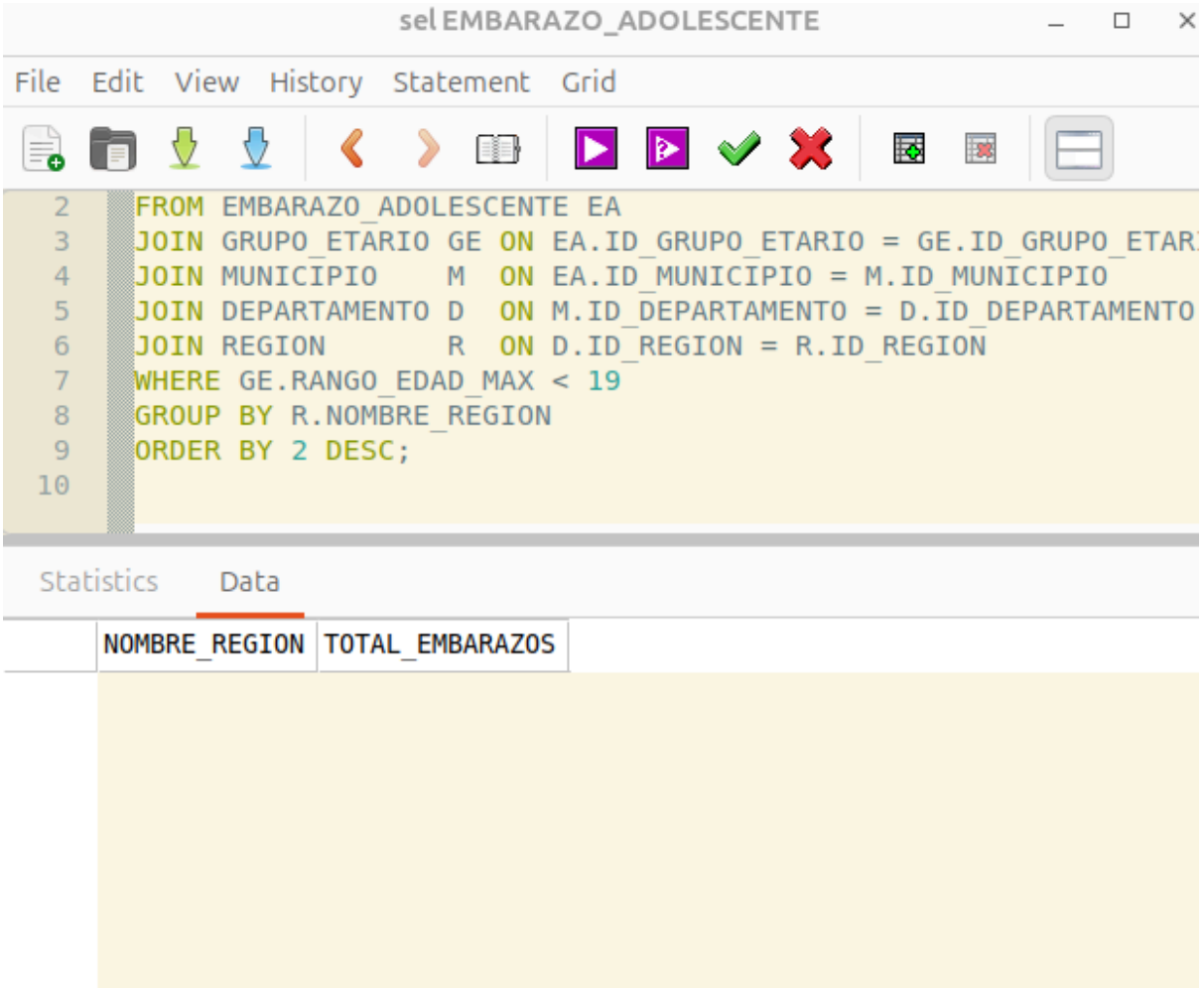
JOIN REGION R ON D.ID_REGION = R.ID_REGION

WHERE GE.RANGO_EDAD_MAX < 19

GROUP BY R.NOMBRE_REGION

ORDER BY 2 DESC;

**Álgebra Relacional** $\tau$ TOTAL$\downarrow$ ( $\gamma$ NOMBRE_REGION; SUM(CANTIDAD_CASOS) $\rightarrow$ TOTAL ( $\sigma$ RANGO_EDAD_MAX < 19 ( Embarazo_Adolescente $\bowtie$ Grupo_Etario $\bowtie$ Municipio $\bowtie$ Departamento $\bowtie$ Region ) ) )




**7. Violencia infantil y trabajo infantil**

**SQL**

SELECT COUNT(DISTINCT HVI.ID_HECHO_VI) AS TOTAL_CASOS_CRUZADOS

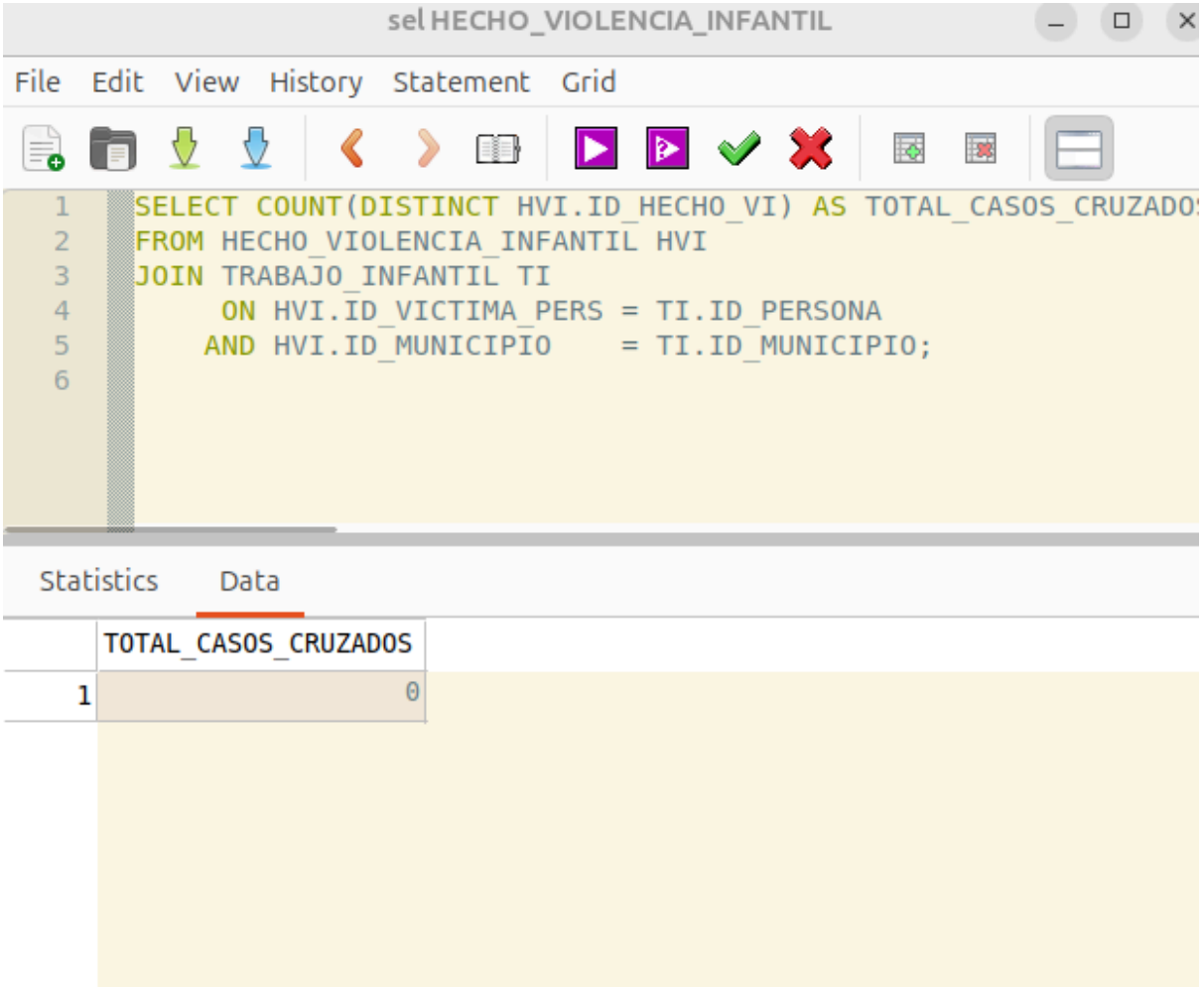
FROM HECHO_VIOLENCIA_INFANTIL HVI

JOIN TRABAJO_INFANTIL TI

ON HVI.ID_VICTIMA_PERS = TI.ID_PERSONA

AND HVI.ID_MUNICIPIO = TI.ID_MUNICIPIO;

**Álgebra Relacional** γ ; COUNT_DISTINCT(ID_HECHO_VI) → TOTAL ( Hecho_Violencia_Infantil ⋈(ID_VICTIMA_PERS = ID_PERSONA ∧ ID_MUNICIPIO = ID_MUNICIPIO) Trabajo_Infantil )




**8. Porcentaje de denuncias por discriminación según etnia**

**SQL**

SELECT E.NOMBRE_ETNIA,

COUNT(HD.ID_HECHO_DISC) AS TOTAL_HECHOS,

CAST(COUNT(HD.ID_HECHO_DISC) * 100.0 /

```
        (SELECT COUNT(*) FROM HECHO_DISCRIMINACION)

        AS DECIMAL(10,2)) AS PORCENTAJE

FROM HECHO_DISCRIMINACION HD

JOIN PERSONA P ON HD.ID_VICTIMA_PERS = P.ID_PERSONA

JOIN ETNIA   E ON P.ID_ETNIA = E.ID_ETNIA

GROUP BY E.NOMBRE_ETNIA

ORDER BY 2 DESC;
```

**Álgebra Relacional** Sea T = γ ; COUNT(*) → N (Hecho_Discriminacion) π NOMBRE_ETNIA, TOTAL, (TOTAL * 100 / T.N) → PORCENTAJE ( γ NOMBRE_ETNIA; COUNT(ID_HECHO_DISC) → TOTAL ( Hecho_Discriminacion ⋈ Persona ⋈ Etnia ) )

sel HECHO_DISCRIMINACION

File Edit View History Statement Grid

```
3        CAST(COUNT(HD.ID_HECHO_DISC) * 100.0 /
4             (SELECT COUNT(*) FROM HECHO_DISCRIMINACION)
5             AS DECIMAL(10,2)) AS PORCENTAJE
6  FROM HECHO_DISCRIMINACION HD
7  JOIN PERSONA P ON HD.ID_VICTIMA_PERS = P.ID_PERSONA
8  JOIN ETNIA   E ON P.ID_ETNIA = E.ID_ETNIA
9  GROUP BY E.NOMBRE_ETNIA
10 ORDER BY 2 DESC;
11
```

Statistics Data

| | NOMBRE_ETNIA | TOTAL_HECHOS | PORCENTAJE |
|---|---|---|---|
| 5 | Maya Q'eqchi' | 176 | 7.00 |
| 6 | Maya Achi' | 174 | 6.90 |
| 7 | Maya K'iche' | 169 | 6.70 |
| 8 | Maya Poqomchi' | 169 | 6.70 |
| 9 | Maya Ixil | 165 | 6.60 |
| 10 | Otro | 162 | 6.40 |
| 11 | Maya Tz'utujil | 162 | 6.40 |
| 12 | Maya Chuj | 159 | 6.30 |
| 13 | Maya Kaqchikel | 148 | 5.90 |
| 14 | Garífuna | 146 | 5.80 |
| 15 | Ladino | 145 | 5.80 |

**9. Escolaridad completada vs. falta judicial**

**SQL**

SELECT NE.NOMBRE_NIVEL AS ESCOLARIDAD, TFJ.NOMBRE_TIPO_FALTA, COUNT(FJ.ID_FALTA_JUD) AS TOTAL_FALTAS

FROM FALTA_JUDICIAL FJ

JOIN TIPO_FALTA_JUDICIAL TFJ ON FJ.ID_TIPO_FALTA = TFJ.ID_TIPO_FALTA

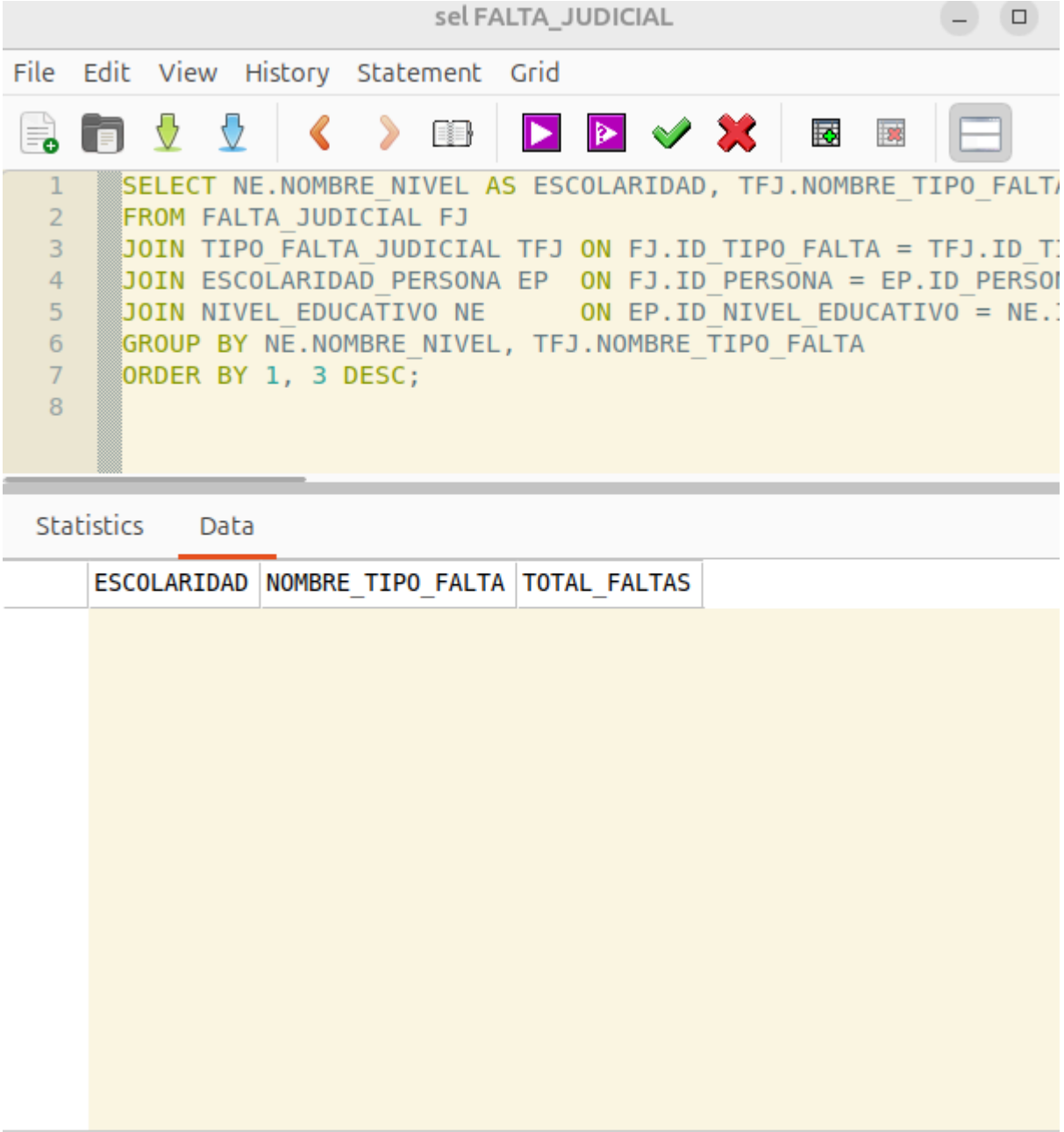
JOIN ESCOLARIDAD_PERSONA EP ON FJ.ID_PERSONA = EP.ID_PERSONA AND EP.COMPLETADO = 1

JOIN NIVEL_EDUCATIVO NE ON EP.ID_NIVEL_EDUCATIVO = NE.ID_NIVEL_EDUCATIVO

GROUP BY NE.NOMBRE_NIVEL, TFJ.NOMBRE_TIPO_FALTA

ORDER BY 1, 3 DESC;

**Álgebra Relacional** $\gamma$ NOMBRE_NIVEL, NOMBRE_TIPO_FALTA; COUNT(ID_FALTA_JUD) $\rightarrow$ TOTAL ( $\sigma$ COMPLETADO = 1 ( Falta_Judicial $\bowtie$ Tipo_Falta_Judicial $\bowtie$ Escolaridad_Persona $\bowtie$ Nivel_Educativo ) )




**10. Necropsias por año**

**SQL**

SELECT EXTRACT(YEAR FROM FECHA_NECROPSIA) AS ANIO, COUNT(ID_NECROPSIA) AS TOTAL_NECROPSIAS

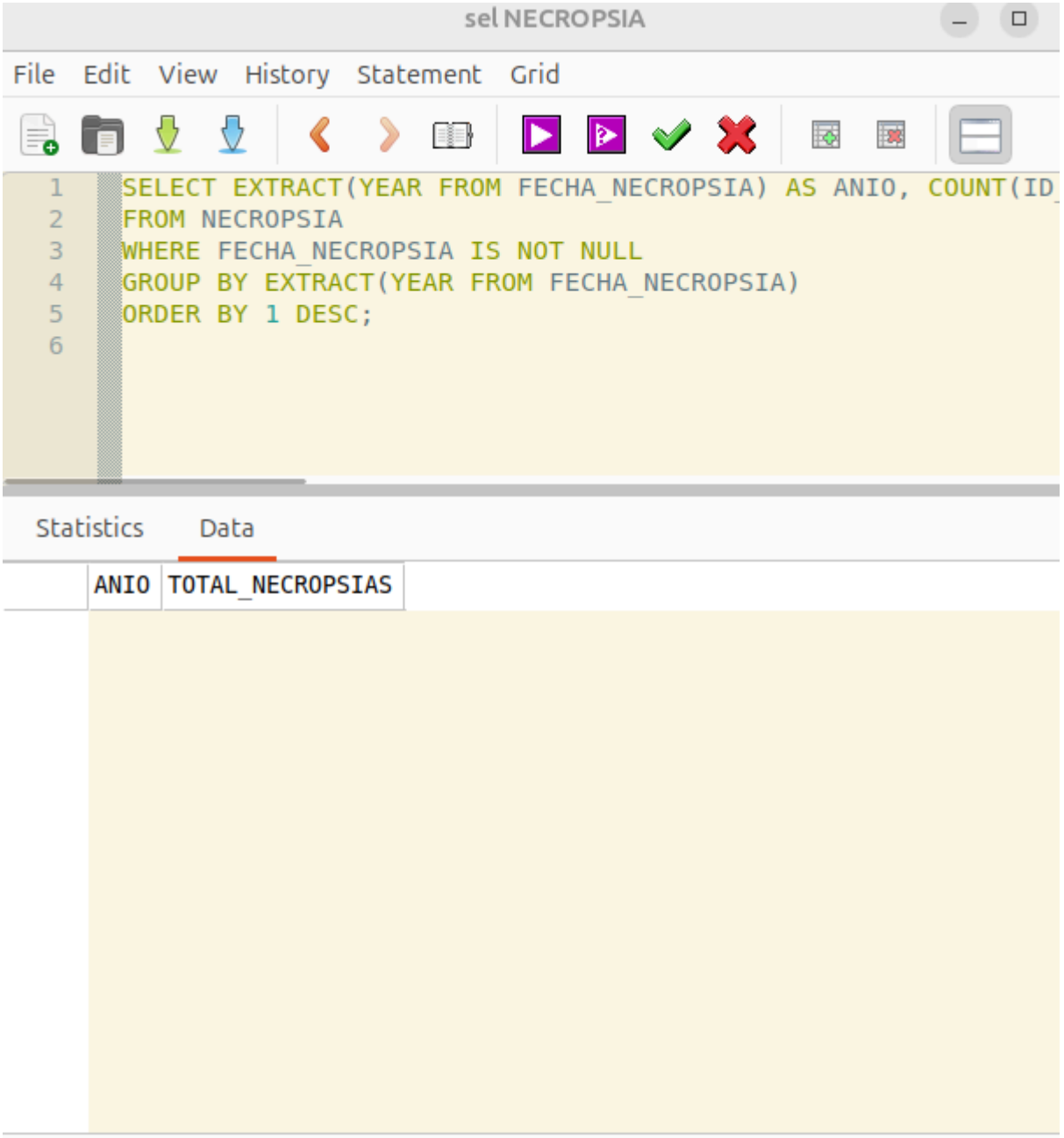
FROM NECROPSIA

WHERE FECHA_NECROPSIA IS NOT NULL

GROUP BY EXTRACT(YEAR FROM FECHA_NECROPSIA)

ORDER BY 1 DESC;

**Álgebra Relacional** τ ANIO↓ ( γ ANIO; COUNT(ID_NECROPSIA) → TOTAL ( ρ ANIO ← EXTRACT(YEAR, FECHA_NECROPSIA) ( σ FECHA_NECROPSIA ≠ NULL ( Necropsia ) ) ) )




**11. Tasa de violencia ESTRUCTURAL procesada**

**SQL**

SELECT D.FUE_PROCESADA, COUNT(*) AS TOTAL,

CAST(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM DENUNCIA D2 JOIN TIPO_DENUNCIA TD2 ON D2.ID_TIPO_DENUNCIA = TD2.ID_TIPO_DENUNCIA JOIN EJE_VIOLENCIA EV2 ON TD2.ID_EJE_VIOLENCIA = EV2.ID_EJE_VIOLENCIA WHERE UPPER(EV2.NOMBRE_EJE) LIKE '%ESTRUCTURAL%') AS DECIMAL(10,2)) AS PORCENTAJE

FROM DENUNCIA D

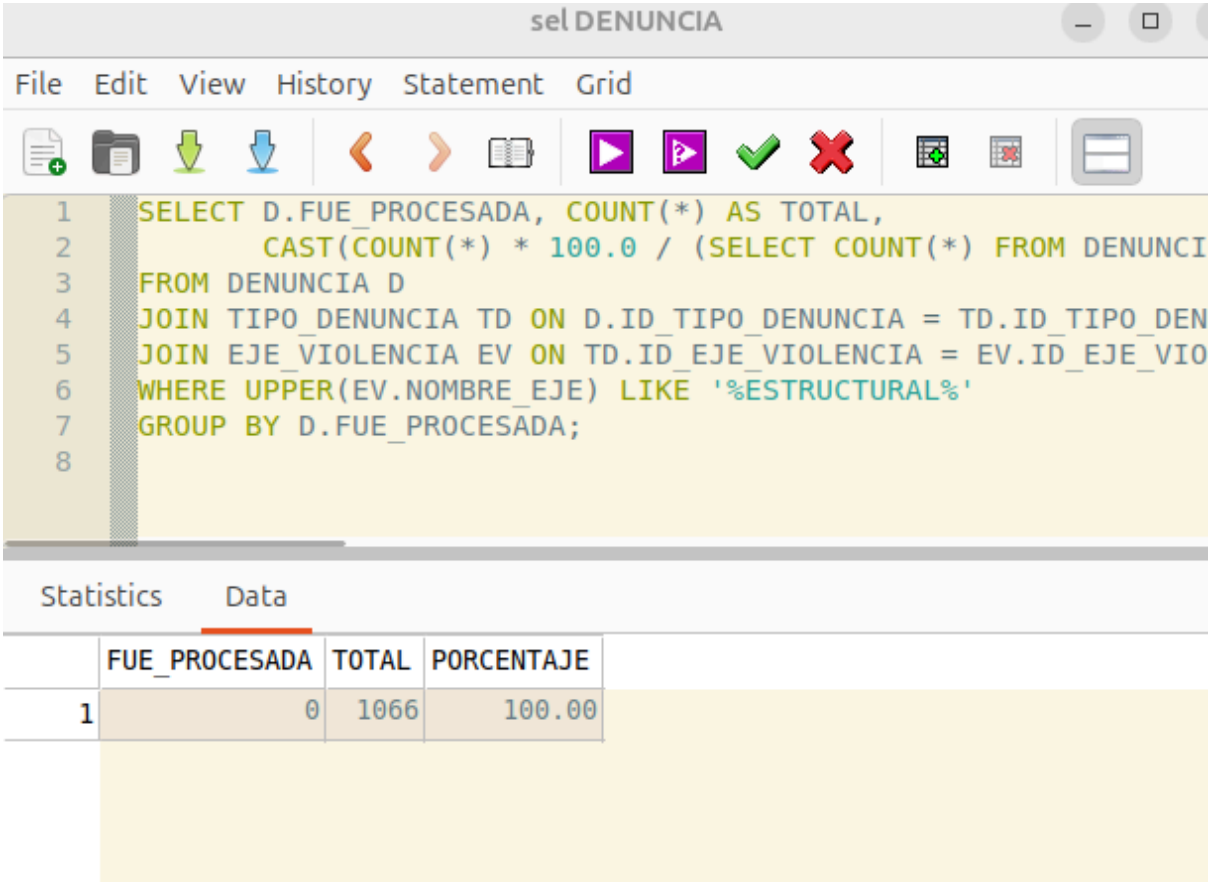
JOIN TIPO_DENUNCIA TD ON D.ID_TIPO_DENUNCIA = TD.ID_TIPO_DENUNCIA

JOIN EJE_VIOLENCIA EV ON TD.ID_EJE_VIOLENCIA = EV.ID_EJE_VIOLENCIA

WHERE UPPER(EV.NOMBRE_EJE) LIKE '%ESTRUCTURAL%'

GROUP BY D.FUE_PROCESADA;

**Álgebra Relacional** Sea Dest = σ NOMBRE_EJE ~ 'Estructural' ( Denuncia ⋈ Tipo_Denuncia ⋈ Eje_Violencia ) π FUE_PROCESADA, TOTAL, PORC ( γ FUE_PROCESADA; COUNT() → TOTAL, 100 * COUNT() / |Dest| → PORC ( Dest ) )



**12. Empleo actual del agresor vs. delitos**

**SQL**

SELECT TE.NOMBRE_TIPO AS TIPO_EMPLEO, COUNT(DISTINCT H.ID_HECHO) AS TOTAL_HECHOS

FROM HECHO_DELICTIVO H

JOIN AGRESOR A ON H.ID_HECHO = A.ID_HECHO

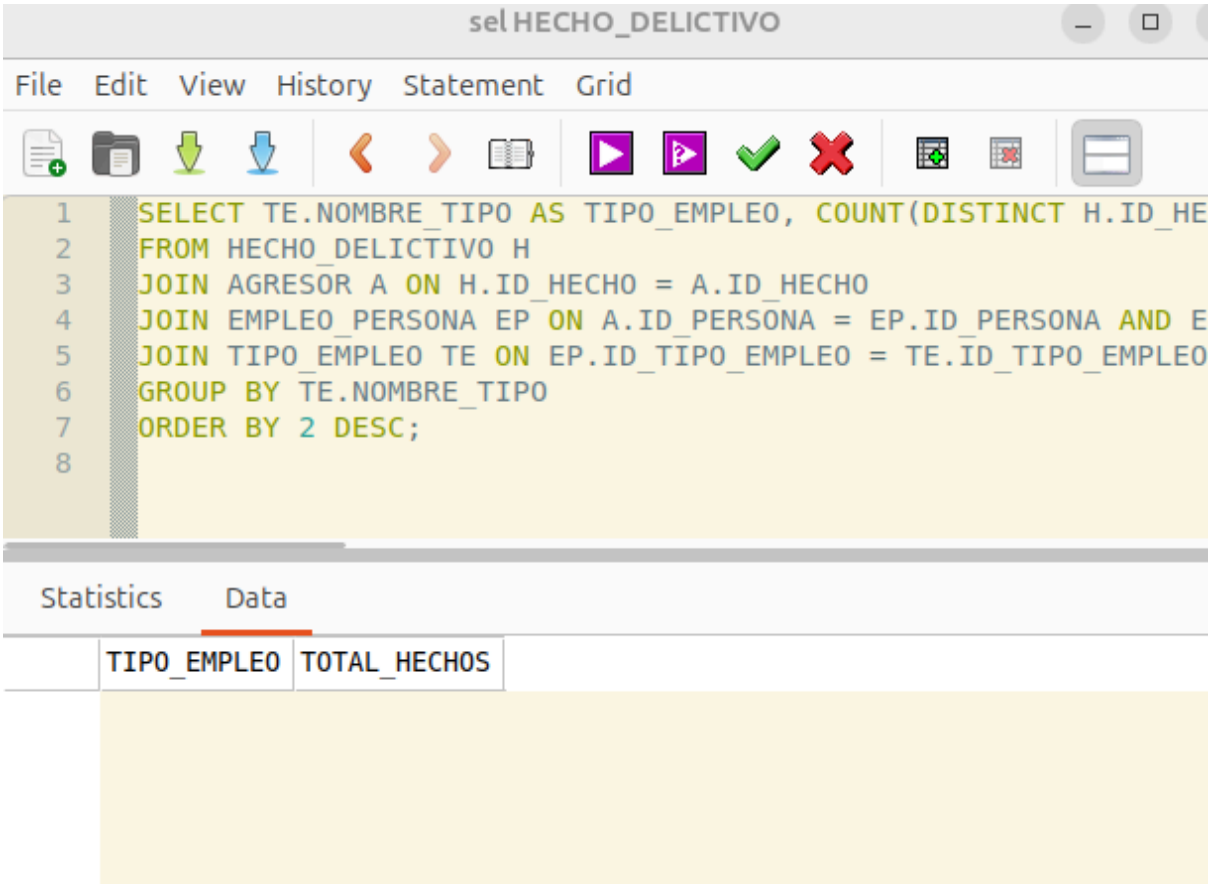
JOIN EMPLEO_PERSONA EP ON A.ID_PERSONA = EP.ID_PERSONA AND EP.ES_ACTUAL = 1

JOIN TIPO_EMPLEO TE ON EP.ID_TIPO_EMPLEO = TE.ID_TIPO_EMPLEO

GROUP BY TE.NOMBRE_TIPO

ORDER BY 2 DESC;

**Álgebra Relacional** $\gamma$ NOMBRE_TIPO; COUNT_DISTINCT(ID_HECHO) $\rightarrow$ TOTAL ( $\sigma$ ES_ACTUAL = 1 ( Hecho_Delictivo $\bowtie$ Agresor $\bowtie$ Empleo_Persona $\bowtie$ Tipo_Empleo ) )

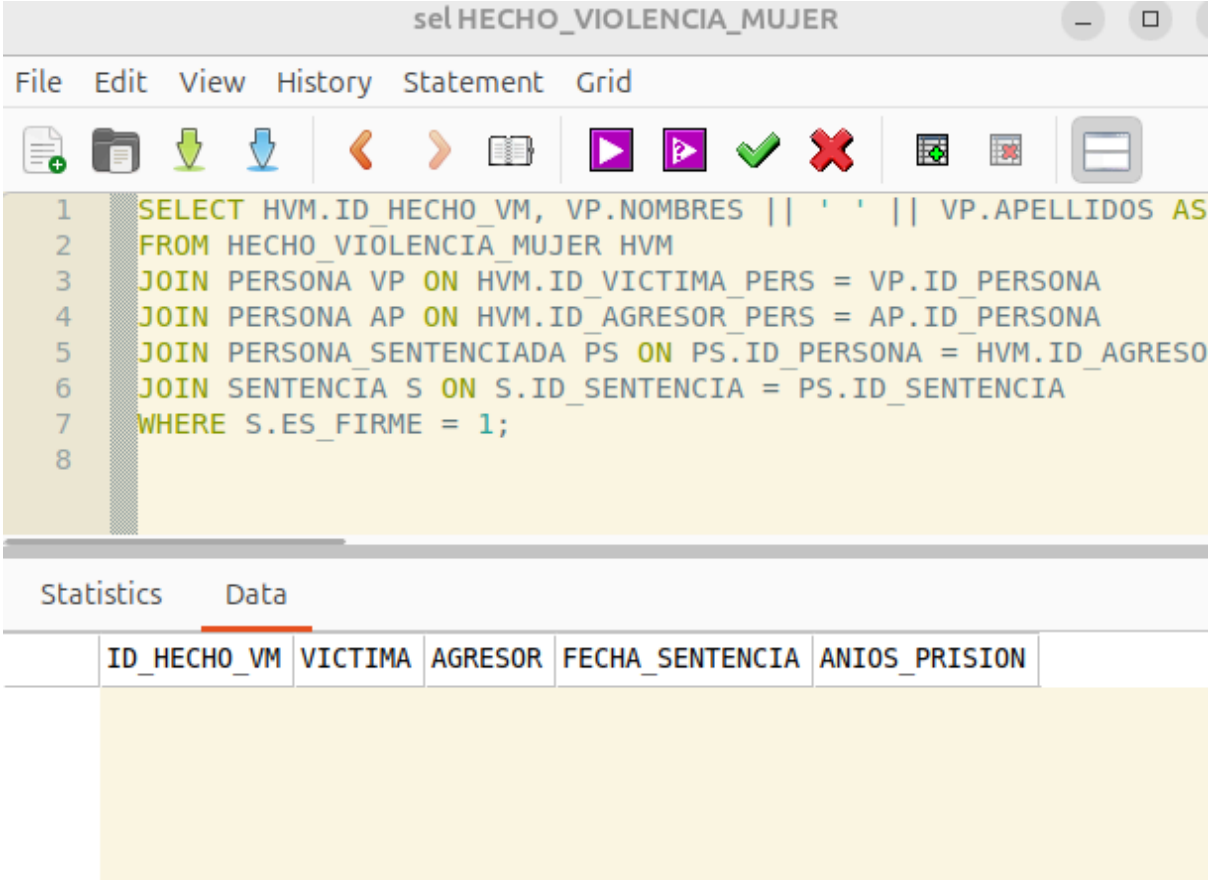


**13. Violencia mujer con sentencia FIRME**

**SQL**

```
SELECT HVM.ID_HECHO_VM, VP.NOMBRES || ' ' || VP.APELLIDOS AS VICTIMA, AP.NOMBRES || ' ' || AP.APELLIDOS AS AGRESOR, S.FECHA_SENTENCIA, S.ANIOS_PRISION

FROM HECHO_VIOLENCIA_MUJER HVM

JOIN PERSONA VP ON HVM.ID_VICTIMA_PERS = VP.ID_PERSONA

JOIN PERSONA AP ON HVM.ID_AGRESOR_PERS = AP.ID_PERSONA

JOIN PERSONA_SENTENCIADA PS ON PS.ID_PERSONA = HVM.ID_AGRESOR_PERS

JOIN SENTENCIA S ON S.ID_SENTENCIA = PS.ID_SENTENCIA

WHERE S.ES_FIRME = 1;
```

**Álgebra Relacional** $\pi$ ID_HECHO_VM, VICTIMA, AGRESOR, FECHA_SENTENCIA, ANIOS_PRISION ( $\sigma$ ES_FIRME = 1 ( HVM $\bowtie$ Persona_V $\bowtie$ Persona_A $\bowtie$ Persona_Sentenciada $\bowtie$ Sentencia ) )




**14. Idiomas de víctimas de discriminación**

**SQL**

SELECT I.NOMBRE_IDIOMA, COUNT(DISTINCT HD.ID_HECHO_DISC) AS TOTAL_VICTIMAS

FROM HECHO_DISCRIMINACION HD

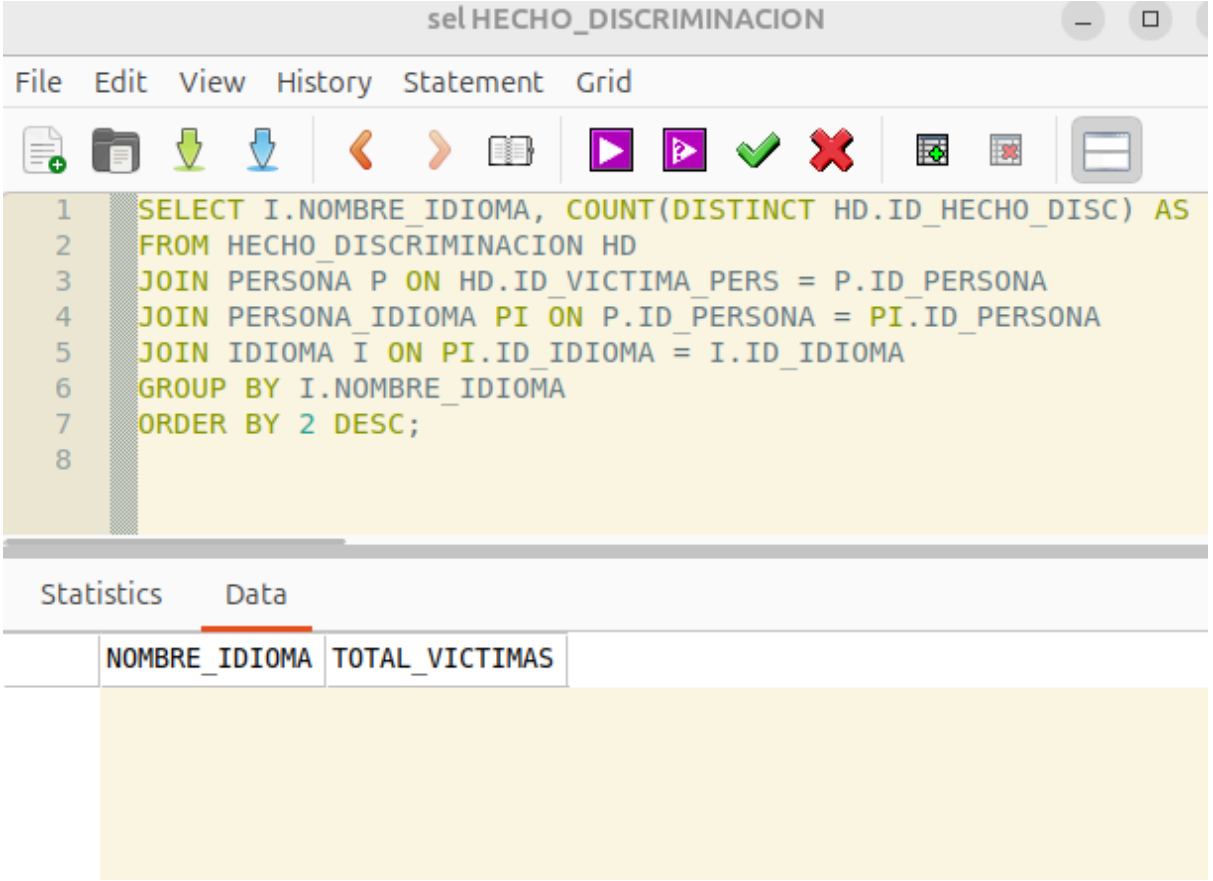
JOIN PERSONA P ON HD.ID_VICTIMA_PERS = P.ID_PERSONA

JOIN PERSONA_IDIOMA PI ON P.ID_PERSONA = PI.ID_PERSONA

JOIN IDIOMA I ON PI.ID_IDIOMA = I.ID_IDIOMA

GROUP BY I.NOMBRE_IDIOMA

ORDER BY 2 DESC;




**Álgebra Relacional** γ NOMBRE_IDIOMA; COUNT_DISTINCT(ID_HECHO_DISC) → TOTAL ( Hecho_Discriminacion ⋈ Persona ⋈ Persona_Idioma ⋈ Idioma )

**15. Personas por hogar con denuncias VIF**

**SQL**

SELECT H.ID_HOGAR, H.NUMERO_PERSONAS, COUNT(HVIF.ID_HECHO_VIF) AS TOTAL_DENUNCIAS

FROM HOGAR H

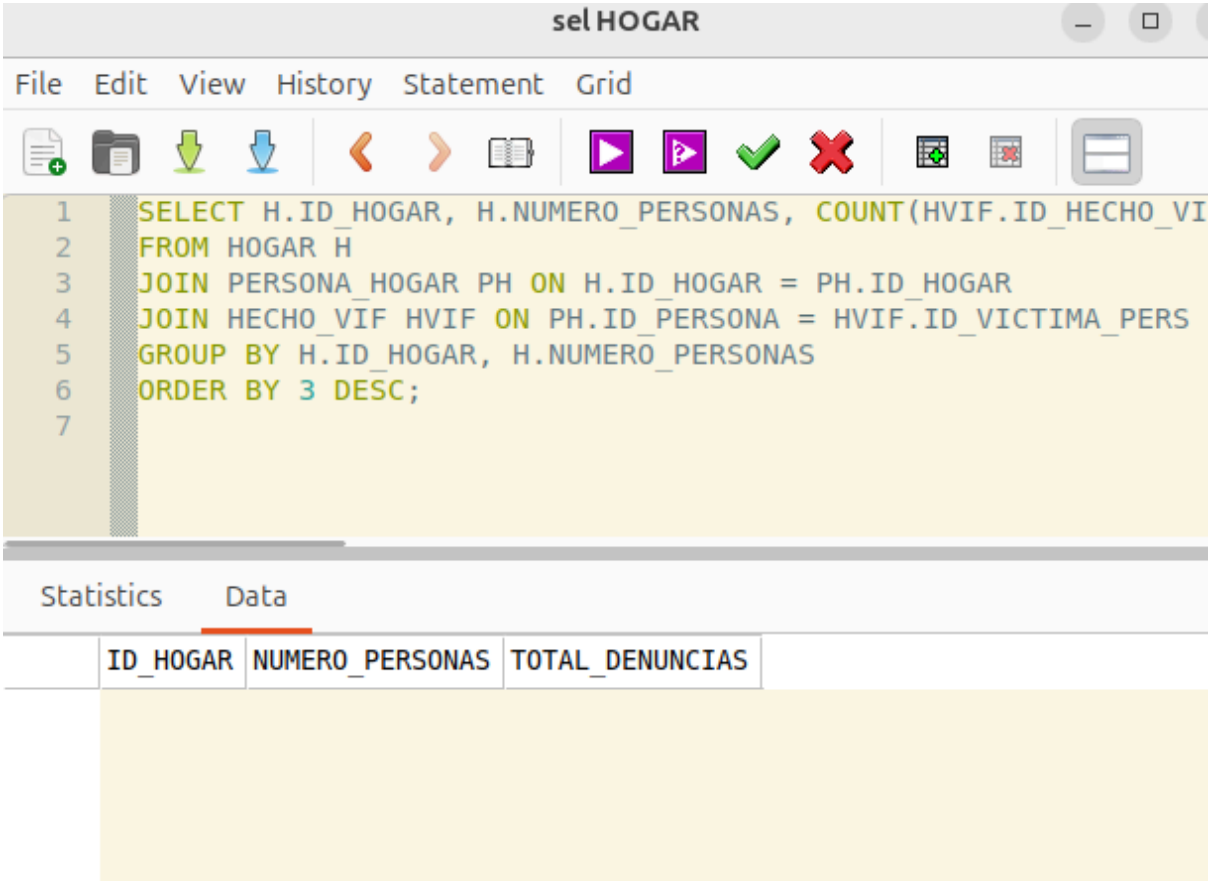
JOIN PERSONA_HOGAR PH ON H.ID_HOGAR = PH.ID_HOGAR

JOIN HECHO_VIF HVIF ON PH.ID_PERSONA = HVIF.ID_VICTIMA_PERS

GROUP BY H.ID_HOGAR, H.NUMERO_PERSONAS

ORDER BY 3 DESC;

**Álgebra Relacional** γ ID_HOGAR, NUMERO_PERSONAS; COUNT(ID_HECHO_VIF) → TOTAL ( Hogar ⋈ Persona_Hogar ⋈ Hecho_VIF )




**16. Violencia infantil: Escolarizados vs No escolarizados**

**SQL**

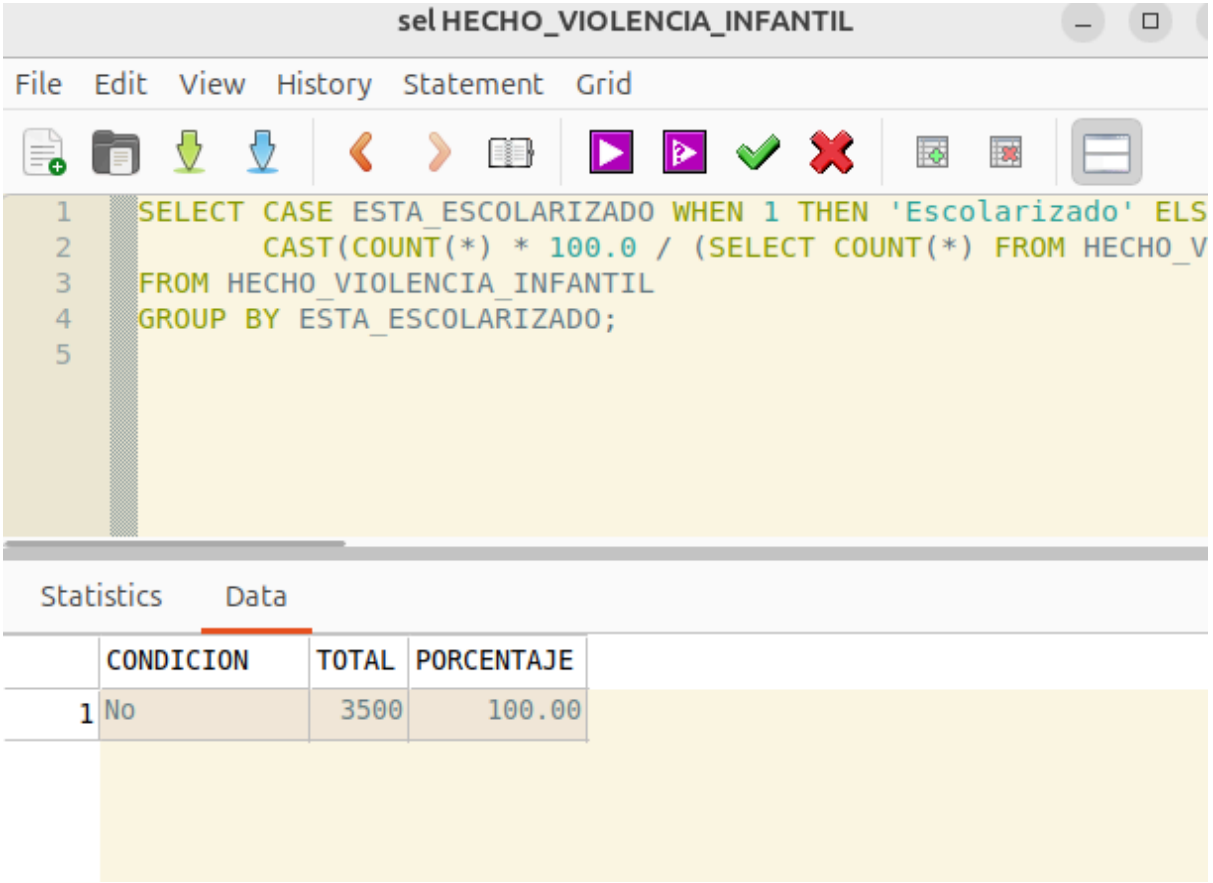
SELECT CASE ESTA_ESCOLARIZADO WHEN 1 THEN 'Escolarizado' ELSE 'No' END AS CONDICION, COUNT(*) AS TOTAL,

CAST(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM HECHO_VIOLENCIA_INFANTIL) AS DECIMAL(10,2)) AS PORCENTAJE

FROM HECHO_VIOLENCIA_INFANTIL

GROUP BY ESTA_ESCOLARIZADO;

**Álgebra Relacional** γ ESTA_ESCOLARIZADO; COUNT(*) → TOTAL ( Hecho_Violencia_Infantil )




**17. Trabajo infantil por sector**

**SQL**

SELECT S.NOMBRE_SECTOR_TI, COUNT(TI.ID_TRABAJO_INF) AS TOTAL_CASOS

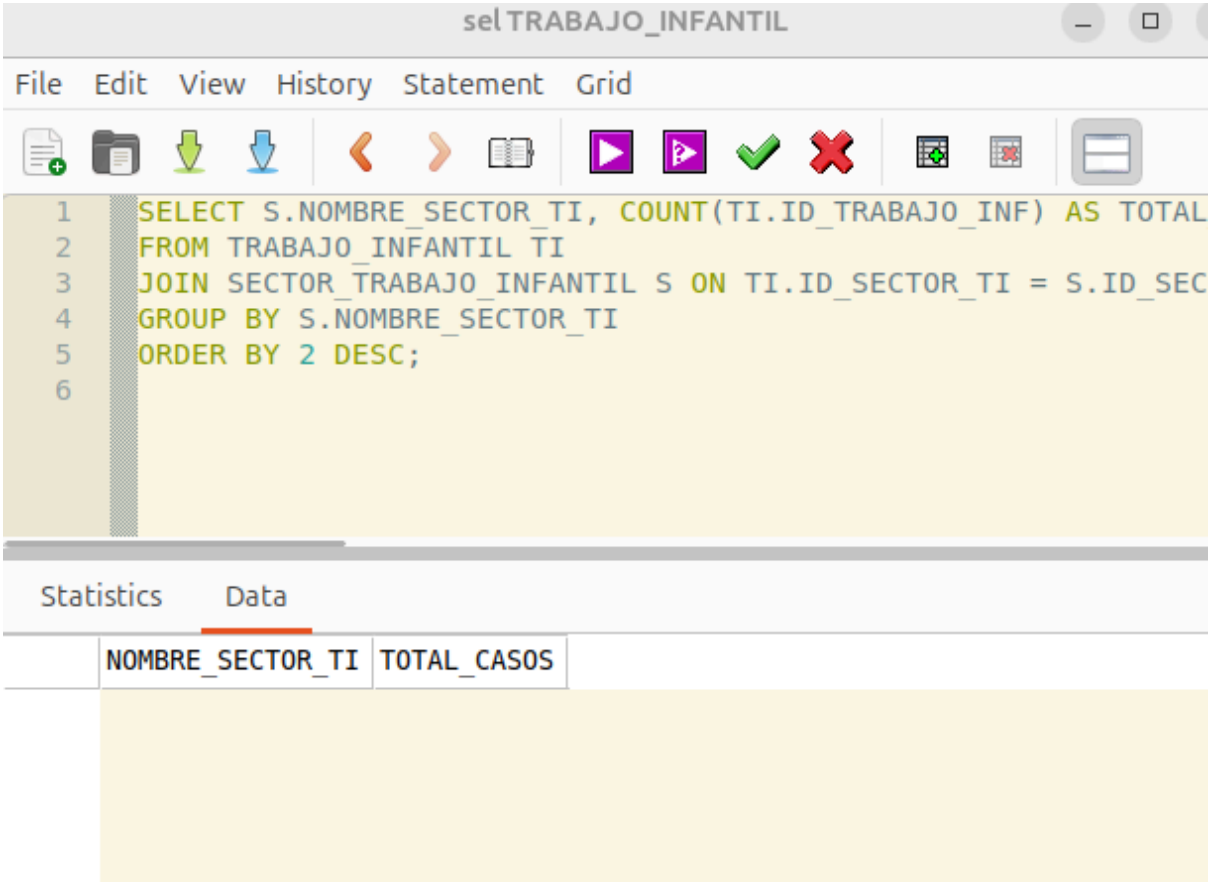
FROM TRABAJO_INFANTIL TI

JOIN SECTOR_TRABAJO_INFANTIL S ON TI.ID_SECTOR_TI = S.ID_SECTOR_TI

GROUP BY S.NOMBRE_SECTOR_TI

ORDER BY 2 DESC;

**Álgebra Relacional** $\tau$ TOTAL$\downarrow$ ( $\gamma$ NOMBRE_SECTOR_TI; COUNT(ID_TRABAJO_INF) $\rightarrow$ TOTAL ( Trabajo_Infantil $\bowtie$ Sector_Trabajo_Infantil ) )




**18. Edad vs Tipo de Violencia (Unión)**

**SQL**

SELECT 'Infantil' AS T, EDAD_VICTIMA, COUNT(*) FROM HECHO_VIOLENCIA_INFANTIL WHERE EDAD_VICTIMA IS NOT NULL GROUP BY EDAD_VICTIMA

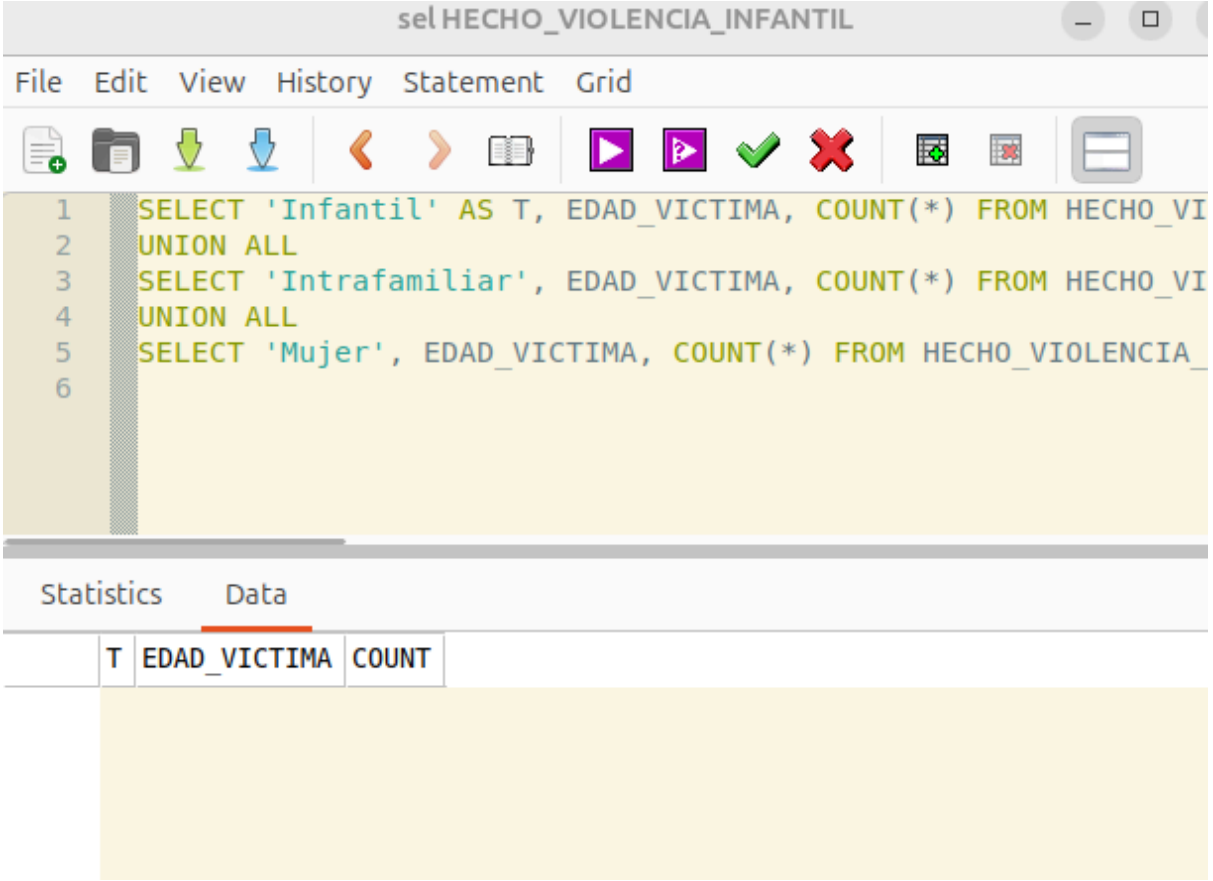
UNION ALL

SELECT 'Intrafamiliar', EDAD_VICTIMA, COUNT(*) FROM HECHO_VIF WHERE EDAD_VICTIMA IS NOT NULL GROUP BY EDAD_VICTIMA

UNION ALL

SELECT 'Mujer', EDAD_VICTIMA, COUNT(*) FROM HECHO_VIOLENCIA_MUJER WHERE EDAD_VICTIMA IS NOT NULL GROUP BY EDAD_VICTIMA;

**Álgebra Relacional** $\rho$ T ← 'Infantil' ( $\gamma$ EDAD; COUNT() (HVI) ) ∪ $\rho$ T ← 'Intrafamiliar' ( $\gamma$ EDAD; COUNT() (HVIF) ) ∪ $\rho$ T ← 'Mujer' ( $\gamma$ EDAD; COUNT() (HVM) )




**19. Desnutrición AGUDA por municipio**

**SQL**

SELECT D.NOMBRE_DEPARTAMENTO, M.NOMBRE_MUNICIPIO, RD.ANIO, SUM(RD.CANTIDAD_CASOS) AS TOTAL_AGUDA

FROM REGISTRO_DESNUTRICION RD

JOIN MUNICIPIO M ON RD.ID_MUNICIPIO = M.ID_MUNICIPIO

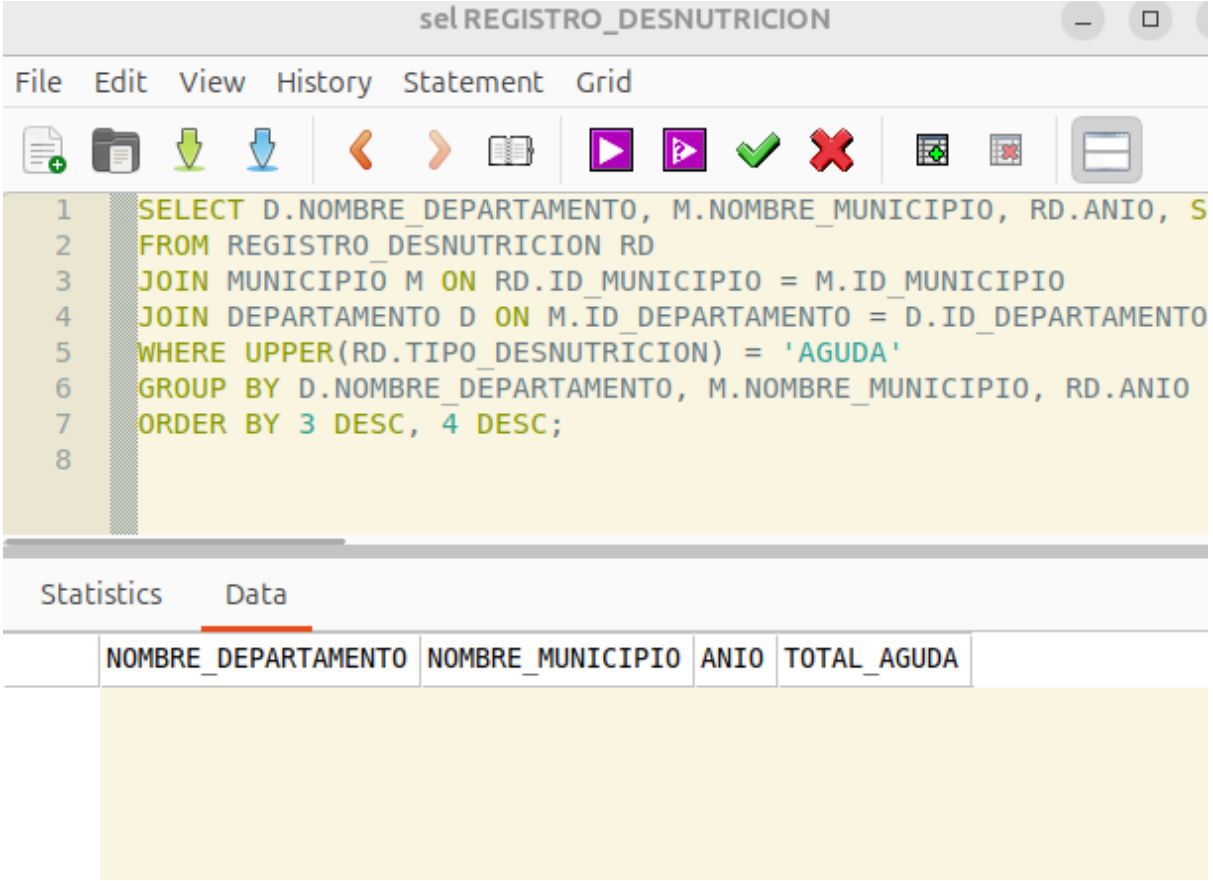
JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

WHERE UPPER(RD.TIPO_DESNUTRICION) = 'AGUDA'

GROUP BY D.NOMBRE_DEPARTAMENTO, M.NOMBRE_MUNICIPIO, RD.ANIO

ORDER BY 3 DESC, 4 DESC;

**Álgebra Relacional** $\gamma$ DEPTO, MUNI, ANIO; SUM(CANTIDAD_CASOS) $\rightarrow$ TOTAL ( $\sigma$ TIPO = 'AGUDA' ( Registro_Desnutricion $\bowtie$ Municipio $\bowtie$ Departamento ) )




**20. Retardo desarrollo por región y sexo**

**SQL**

SELECT R.NOMBRE_REGION, GE.NOMBRE_GRUPO_ETARIO, S.NOMBRE_SEXO, SUM(RS.CANTIDAD_CASOS) AS TOTAL_RETARDO

FROM REGISTRO_SALUD RS

JOIN MUNICIPIO M ON RS.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

JOIN REGION R ON D.ID_REGION = R.ID_REGION

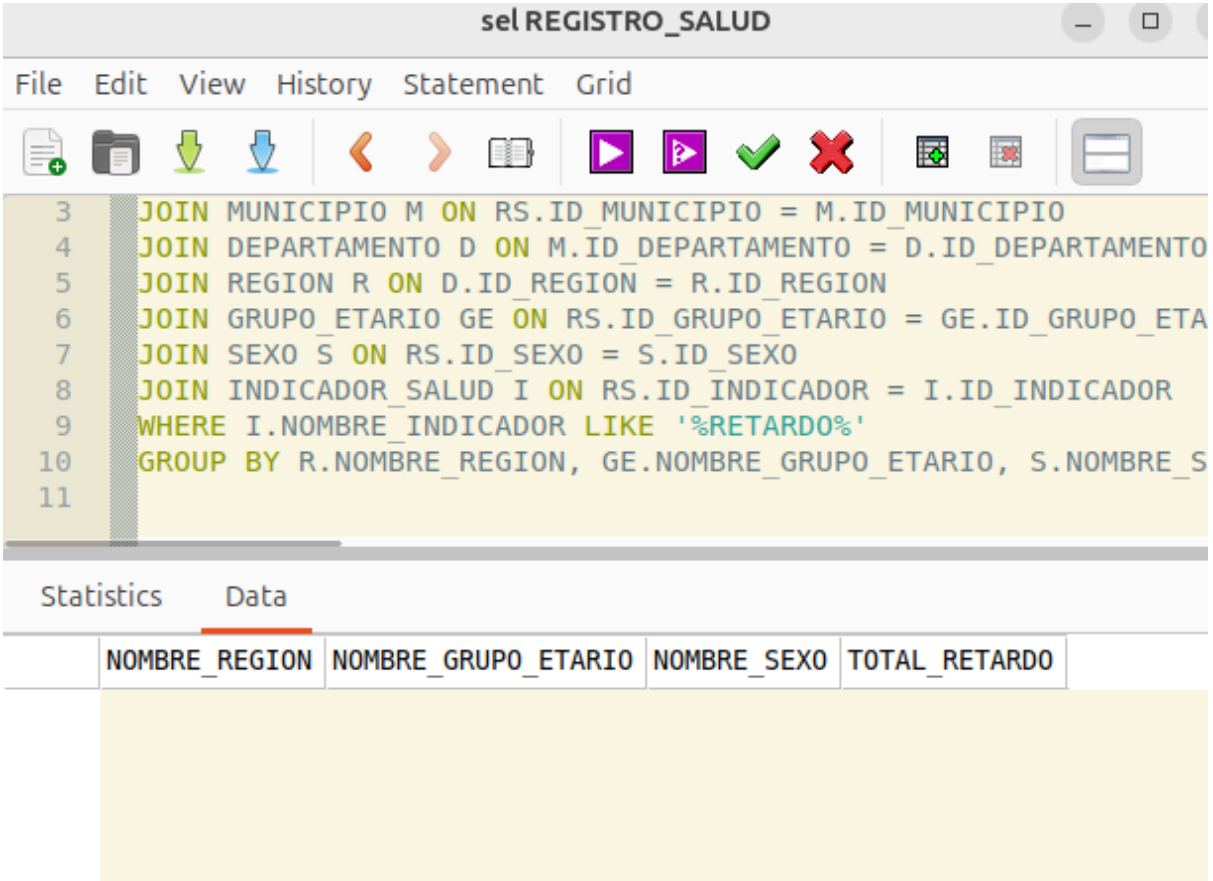
JOIN GRUPO_ETARIO GE ON RS.ID_GRUPO_ETARIO = GE.ID_GRUPO_ETARIO

JOIN SEXO S ON RS.ID_SEXO = S.ID_SEXO

JOIN INDICADOR_SALUD I ON RS.ID_INDICADOR = I.ID_INDICADOR

WHERE I.NOMBRE_INDICADOR LIKE '%RETARDO%'

GROUP BY R.NOMBRE_REGION, GE.NOMBRE_GRUPO_ETARIO, S.NOMBRE_SEXO;




**Álgebra Relacional** γ REGION, GRUPO, SEXO; SUM(CASOS) → T ( σ INDICADOR ~ 'Retardo' ( RS ⋈ Muni ⋈ Depto ⋈ Region ⋈ GE ⋈ Sexo ⋈ Indicador ) )

**21. Enfermedades crónicas por depto**

**SQL**

SELECT D.NOMBRE_DEPARTAMENTO, C.NOMBRE_DIAG, SUM(RC.CANTIDAD_CASOS) AS TOTAL_CASOS

FROM REGISTRO_ENFERMEDAD_CRONICA RC

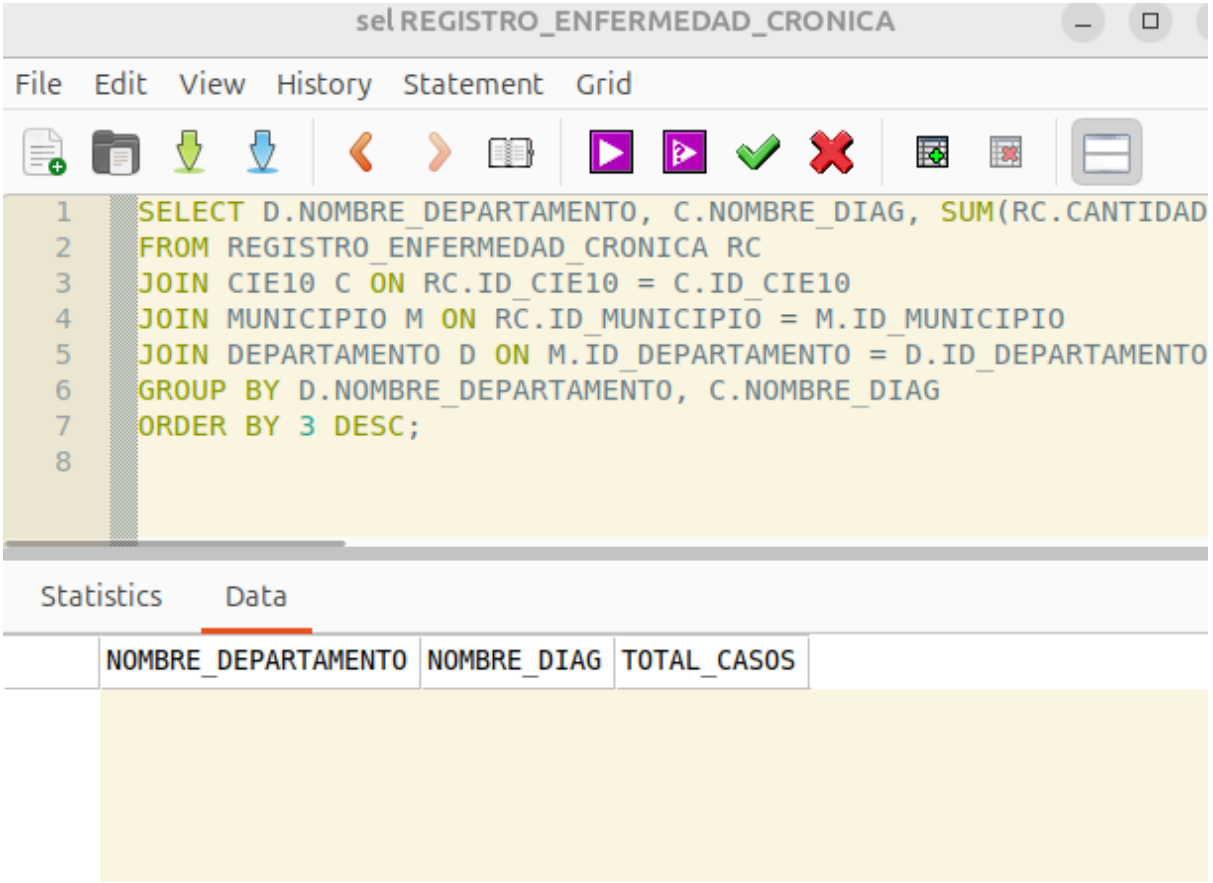
JOIN CIE10 C ON RC.ID_CIE10 = C.ID_CIE10

JOIN MUNICIPIO M ON RC.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

GROUP BY D.NOMBRE_DEPARTAMENTO, C.NOMBRE_DIAG

ORDER BY 3 DESC;




**Álgebra Relacional** $\tau$ TOTAL$\downarrow$ ( $\gamma$ DEPTO, DIAG; SUM(CASOS) $\rightarrow$ T ( REC $\bowtie$ CIE10 $\bowtie$ Municipio $\bowtie$ Departamento ) )

**22. Dengue evolución (2012-2024)**

**SQL**

SELECT REV.ANIO, D.NOMBRE_DEPARTAMENTO, REV.NOMBRE_ENFERMEDAD, SUM(REV.CANTIDAD_CASOS) AS TOTAL_CASOS

FROM REGISTRO_ENFERMEDAD_VECTOR REV

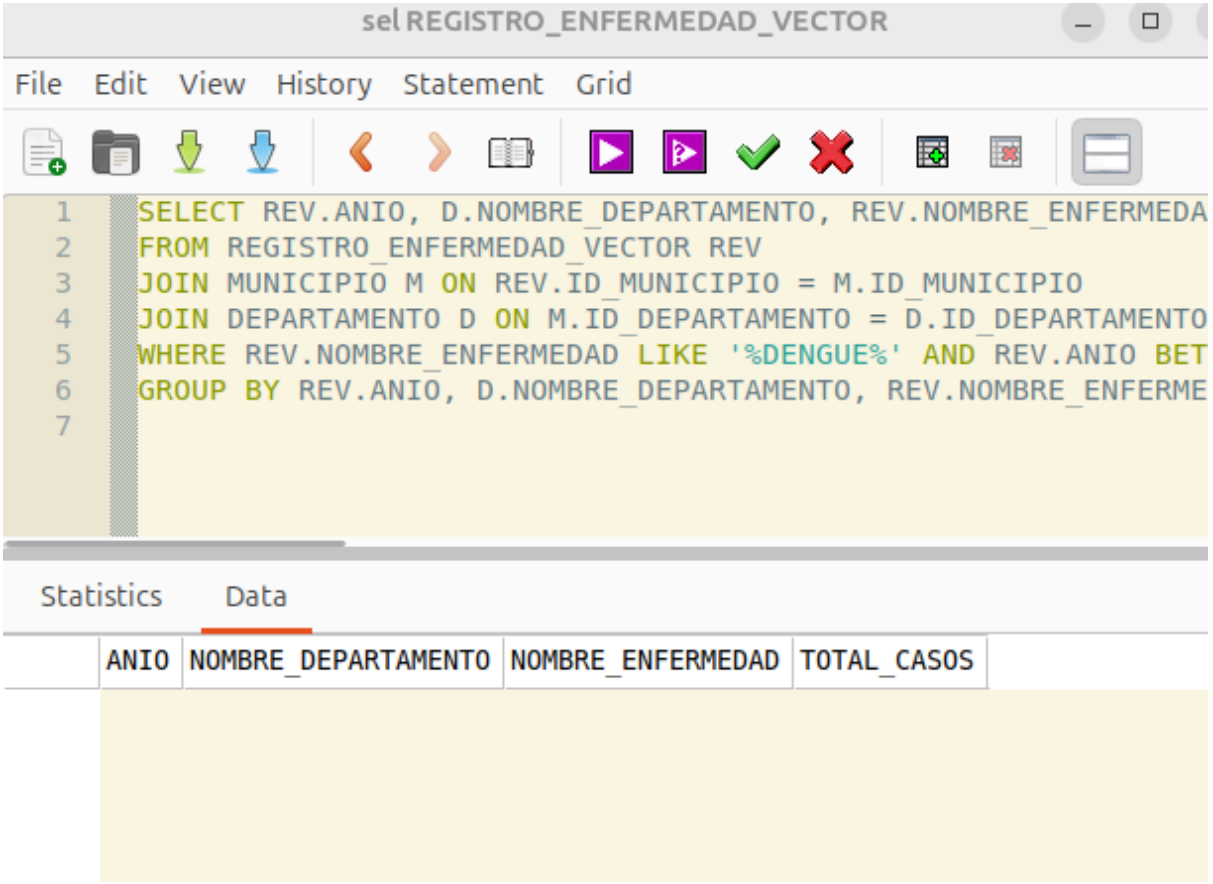
JOIN MUNICIPIO M ON REV.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

WHERE REV.NOMBRE_ENFERMEDAD LIKE '%DENGUE%' AND REV.ANIO BETWEEN 2012 AND 2024

GROUP BY REV.ANIO, D.NOMBRE_DEPARTAMENTO, REV.NOMBRE_ENFERMEDAD;

**Álgebra Relacional** γ ANIO, DEPTO, ENF; SUM(CASOS) → T ( σ ENF ~ 'Dengue' ∧ 2012 ≤ ANIO ≤ 2024 ( REV ⋈ Municipio ⋈ Departamento ) )




**23. Malaria por grupo etario y sexo**

**SQL**

SELECT M.NOMBRE_MUNICIPIO, GE.NOMBRE_GRUPO_ETARIO, S.NOMBRE_SEXO, SUM(REV.CANTIDAD_CASOS) AS TOTAL_MALARIA

FROM REGISTRO_ENFERMEDAD_VECTOR REV

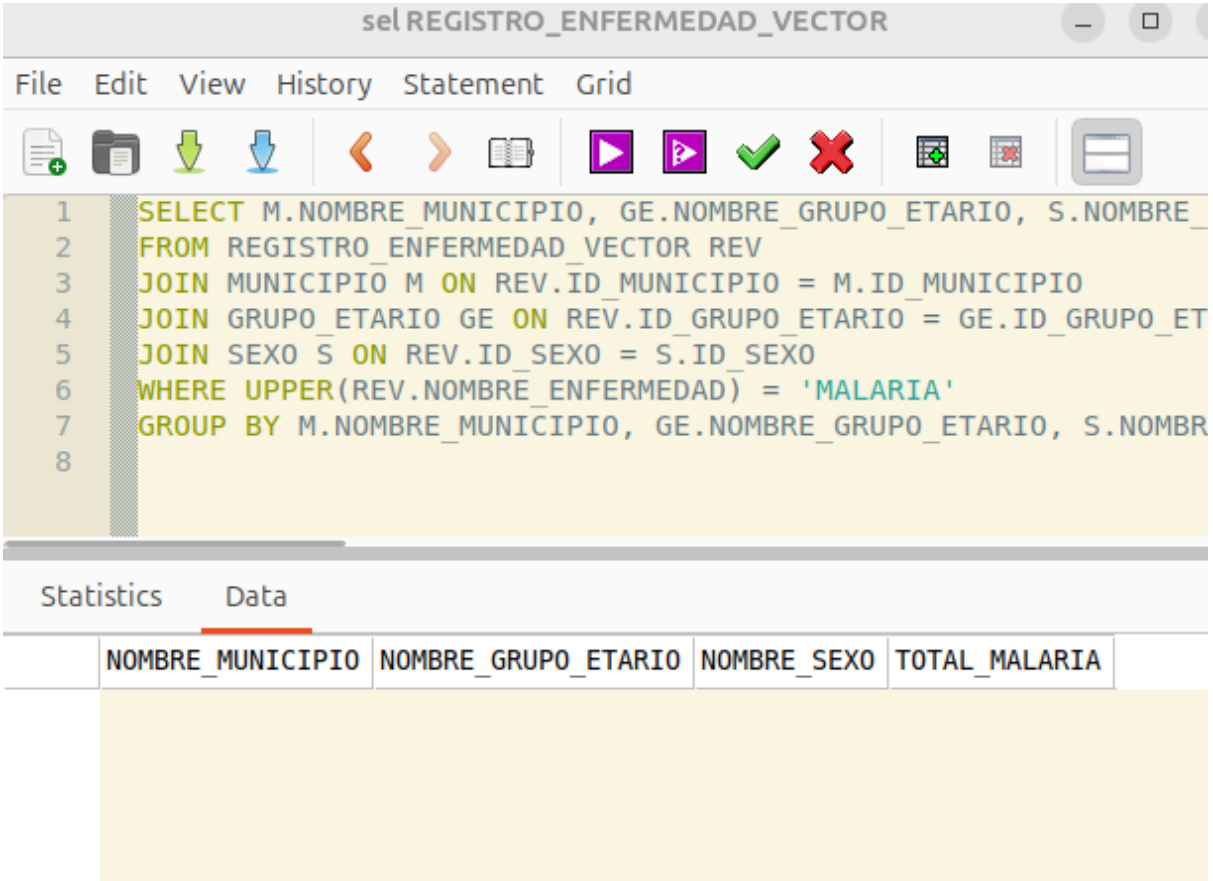
JOIN MUNICIPIO M ON REV.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN GRUPO_ETARIO GE ON REV.ID_GRUPO_ETARIO = GE.ID_GRUPO_ETARIO

JOIN SEXO S ON REV.ID_SEXO = S.ID_SEXO

WHERE UPPER(REV.NOMBRE_ENFERMEDAD) = 'MALARIA'

GROUP BY M.NOMBRE_MUNICIPIO, GE.NOMBRE_GRUPO_ETARIO, S.NOMBRE_SEXO;




**Álgebra Relacional** γ MUNI, GRUPO, SEXO; SUM(CASOS) → T ( σ ENF = 'Malaria' ( REV ⋈ Municipio ⋈ GE ⋈ Sexo ) )

**24. Desnutrición aguda vs VIF (CTE)**

**SQL**

WITH DESNUT AS (

SELECT M.ID_DEPARTAMENTO, SUM(RD.CANTIDAD_CASOS) AS CASOS FROM REGISTRO_DESNUTRICION RD JOIN MUNICIPIO M ON RD.ID_MUNICIPIO = M.ID_MUNICIPIO WHERE RD.TIPO_DESNUTRICION = 'AGUDA' GROUP BY M.ID_DEPARTAMENTO

),

VIF AS (

SELECT M.ID_DEPARTAMENTO, COUNT(HV.ID_HECHO_VIF) AS CASOS FROM HECHO_VIF HV JOIN MUNICIPIO M ON HV.ID_MUNICIPIO = M.ID_MUNICIPIO GROUP BY M.ID_DEPARTAMENTO

)

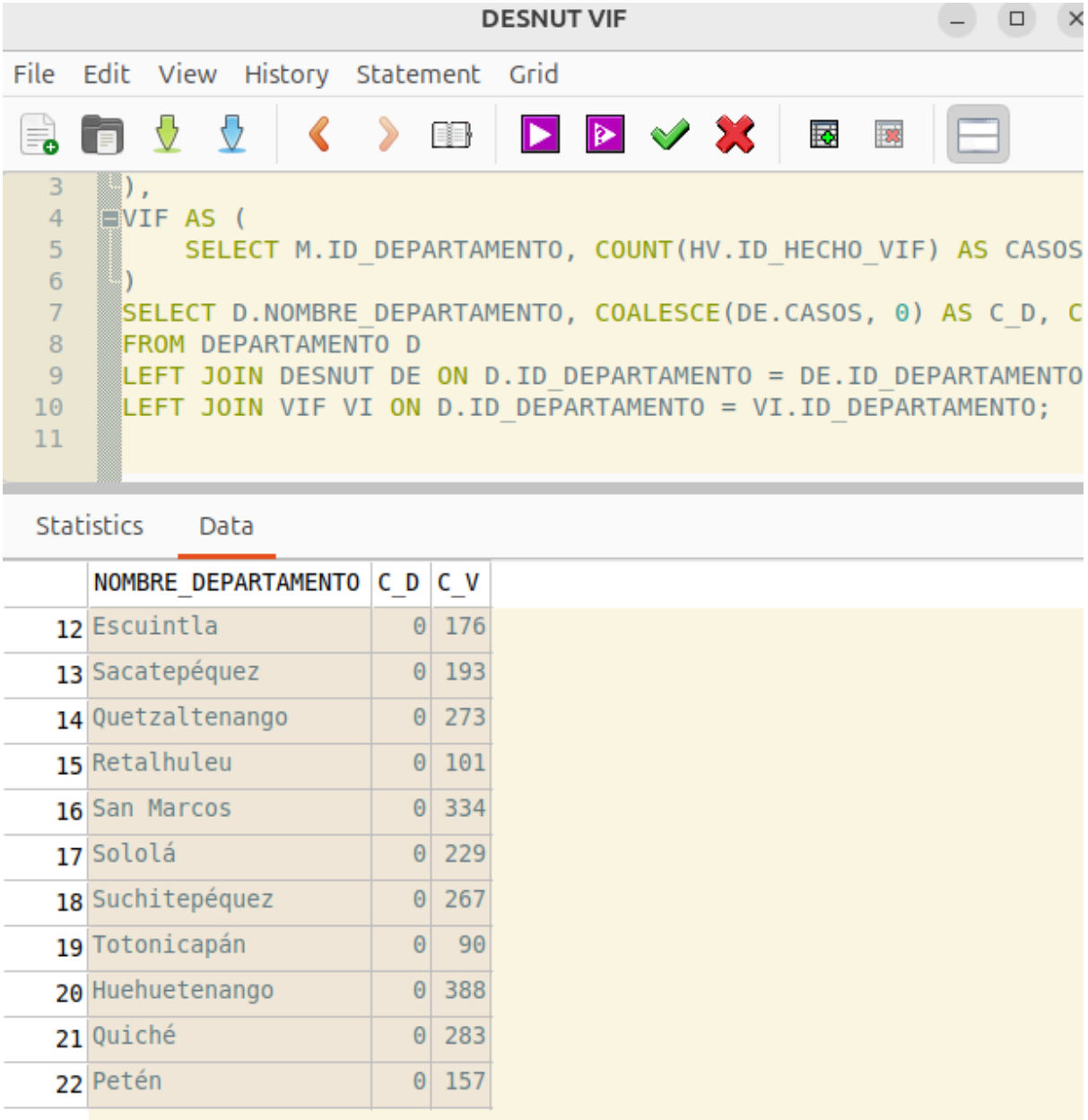
SELECT D.NOMBRE_DEPARTAMENTO, COALESCE(DE.CASOS, 0) AS C_D, COALESCE(VI.CASOS, 0) AS C_V

FROM DEPARTAMENTO D

LEFT JOIN DESNUT DE ON D.ID_DEPARTAMENTO = DE.ID_DEPARTAMENTO

LEFT JOIN VIF VI ON D.ID_DEPARTAMENTO = VI.ID_DEPARTAMENTO;

**Álgebra Relacional** $\pi$ DEPTO, C_d, C_v ( Departamento $\bowtie$ ( $\gamma$ ID_D; SUM(CASOS) $\rightarrow$ C_d ( $\sigma$ TIPO='AGUDA' (Desnutrición) ) ) $\bowtie$ ( $\gamma$ ID_D; COUNT(*) $\rightarrow$ C_v (VIF) ) )




| | NOMBRE_DEPARTAMENTO | C_D | C_V |
|---|---|---|---|
| 12 | Escuintla | 0 | 176 |
| 13 | Sacatepéquez | 0 | 193 |
| 14 | Quetzaltenango | 0 | 273 |
| 15 | Retalhuleu | 0 | 101 |
| 16 | San Marcos | 0 | 334 |
| 17 | Sololá | 0 | 229 |
| 18 | Suchitepéquez | 0 | 267 |
| 19 | Totonicapán | 0 | 90 |
| 20 | Huehuetenango | 0 | 388 |
| 21 | Quiché | 0 | 283 |
| 22 | Petén | 0 | 157 |

**25. Top 5 Municipios Chagas + Zika + Chikungunya**

**SQL**

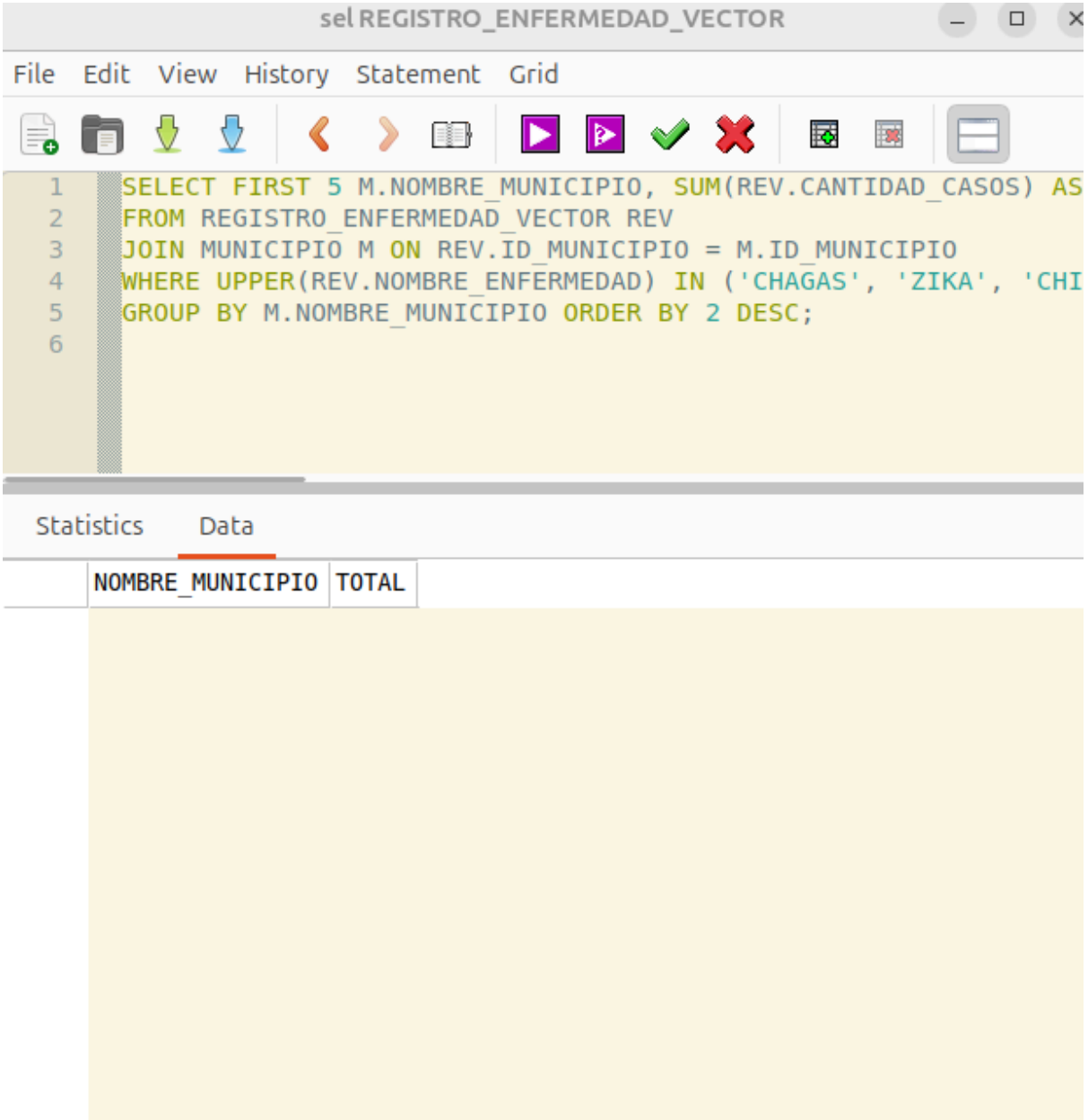
SELECT FIRST 5 M.NOMBRE_MUNICIPIO, SUM(REV.CANTIDAD_CASOS) AS TOTAL

FROM REGISTRO_ENFERMEDAD_VECTOR REV

JOIN MUNICIPIO M ON REV.ID_MUNICIPIO = M.ID_MUNICIPIO

WHERE UPPER(REV.NOMBRE_ENFERMEDAD) IN ('CHAGAS', 'ZIKA', 'CHIKUNGUNYA')

GROUP BY M.NOMBRE_MUNICIPIO ORDER BY 2 DESC;




**Álgebra Relacional** TOP 5 ( τ TOTAL↓ ( γ MUNI; SUM(CASOS) → TOTAL ( σ ENF ∈ {'Chagas','Zika','Chik'} ( REV ⋈ Municipio ) ) ) )

**26. Enfermedades vectores vs Area (Urbana/Rural)**

**SQL**

SELECT D.NOMBRE_DEPARTAMENTO, AG.NOMBRE_AREA, SUM(REV.CANTIDAD_CASOS) AS TOTAL_CASOS

FROM REGISTRO_ENFERMEDAD_VECTOR REV

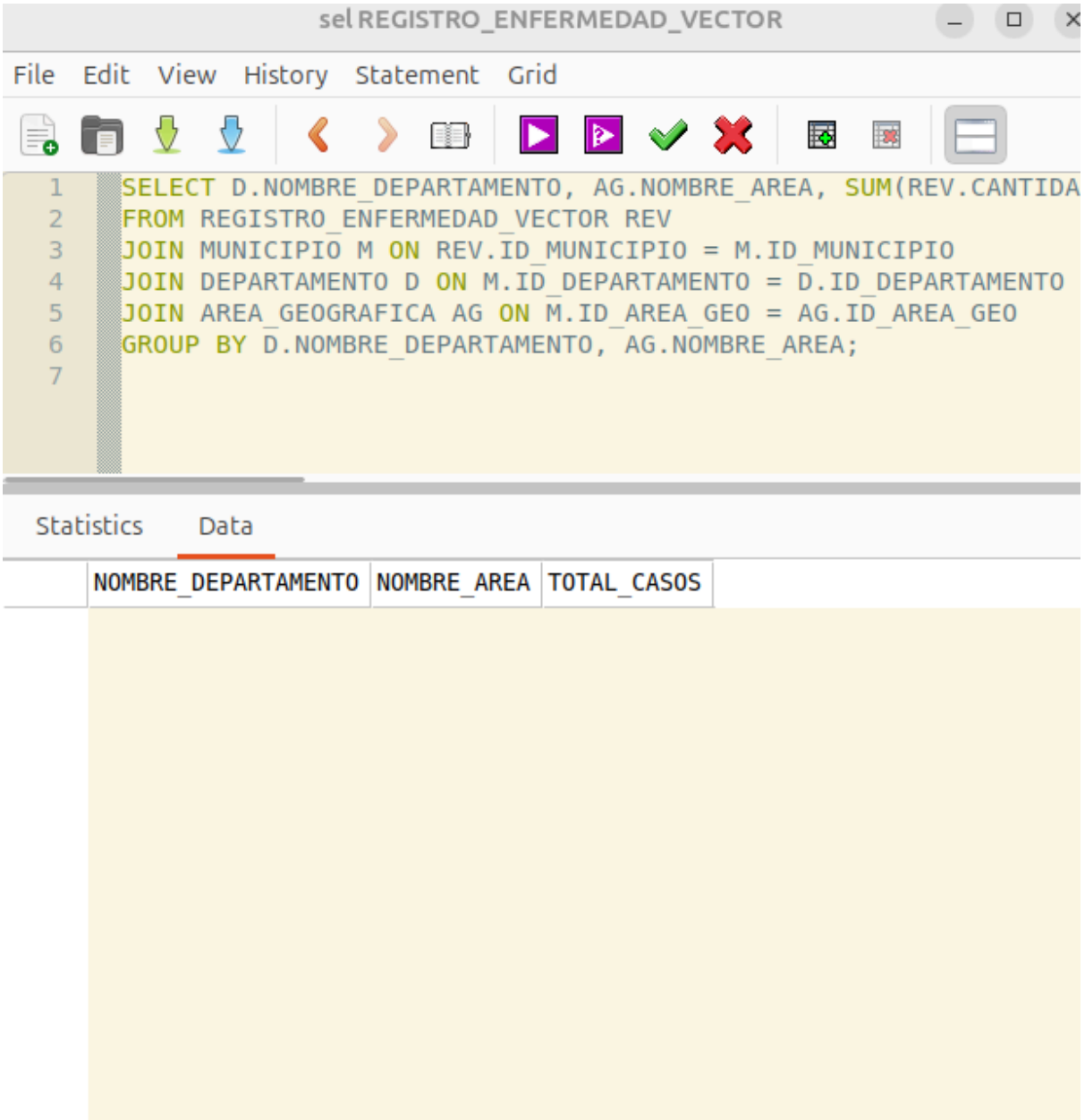
JOIN MUNICIPIO M ON REV.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

JOIN AREA_GEOGRAFICA AG ON M.ID_AREA_GEO = AG.ID_AREA_GEO

GROUP BY D.NOMBRE_DEPARTAMENTO, AG.NOMBRE_AREA;

**Álgebra Relacional** $\gamma$ DEPTO, AREA; SUM(CASOS) $\rightarrow$ T ( REV $\bowtie$ Municipio $\bowtie$ Departamento $\bowtie$ Area_Geo )



**27. Tasa Morbilidad Materna por año**

**SQL**

SELECT RMM.ANIO, D.NOMBRE_DEPARTAMENTO, SUM(RMM.CANTIDAD_CASOS) AS T_MORB, SUM(RMM.MUERTES_MATERNAS) AS T_MUER

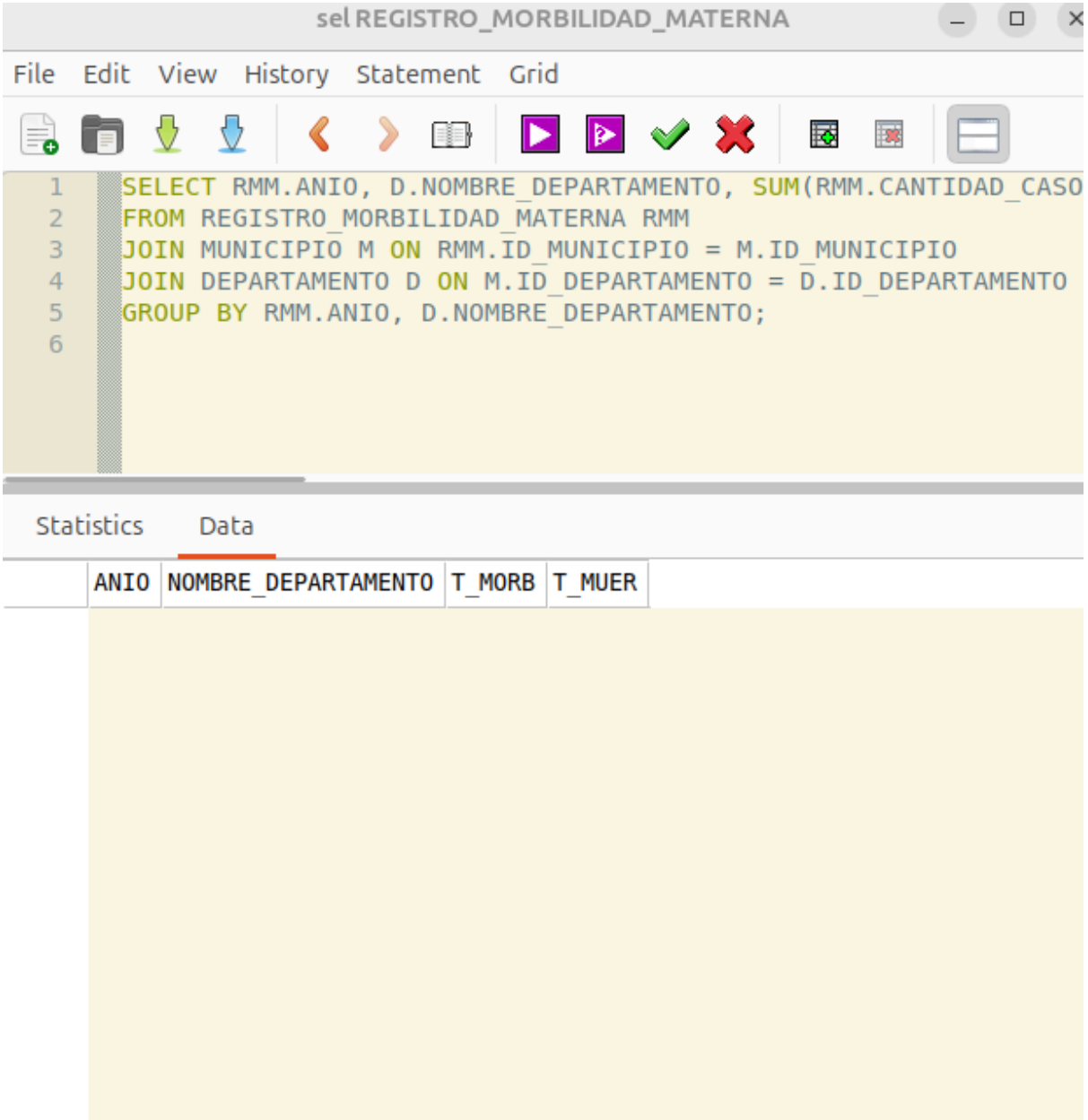
FROM REGISTRO_MORBILIDAD_MATERNA RMM

JOIN MUNICIPIO M ON RMM.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN DEPARTAMENTO D ON M.ID_DEPARTAMENTO = D.ID_DEPARTAMENTO

GROUP BY RMM.ANIO, D.NOMBRE_DEPARTAMENTO;

**Álgebra Relacional** $\gamma$ ANIO, DEPTO; SUM(CASOS) $\rightarrow$ T, SUM(MUERTES) $\rightarrow$ M ( Reg_Morbilidad $\bowtie$ Municipio $\bowtie$ Departamento )




**28. Correlación Desnutrición vs Delitos**

**SQL**

WITH DES AS ( SELECT ID_MUNICIPIO, SUM(CANTIDAD_CASOS) AS CASOS FROM REGISTRO_DESNUTRICION GROUP BY ID_MUNICIPIO ),

HEC AS ( SELECT ID_MUNICIPIO, COUNT(*) AS DELITOS FROM HECHO_DELICTIVO GROUP BY ID_MUNICIPIO )

SELECT M.NOMBRE_MUNICIPIO, COALESCE(DES.CASOS, 0) AS C_DES, COALESCE(HEC.DELITOS, 0) AS C_HEC

FROM MUNICIPIO M

LEFT JOIN DES ON M.ID_MUNICIPIO = DES.ID_MUNICIPIO

LEFT JOIN HEC ON M.ID_MUNICIPIO = HEC.ID_MUNICIPIO;

DES HEC

File Edit View History Statement Grid

```
WITH DES AS ( SELECT ID_MUNICIPIO, SUM(CANTIDAD_CASOS) AS CAS
HEC AS ( SELECT ID_MUNICIPIO, COUNT(*) AS DELITOS FROM HECHO_
SELECT M.NOMBRE_MUNICIPIO, COALESCE(DES.CASOS, 0) AS C_DES, C
FROM MUNICIPIO M
LEFT JOIN DES ON M.ID_MUNICIPIO = DES.ID_MUNICIPIO
LEFT JOIN HEC ON M.ID_MUNICIPIO = HEC.ID_MUNICIPIO;
```

Statistics Data

| | NOMBRE_MUNICIPIO | C_DES | C_HEC |
|---|---|---|---|
| 329 | San Andrés | 0 | 47 |
| 330 | La Libertad | 0 | 49 |
| 331 | San Francisco | 0 | 51 |
| 332 | Santa Ana | 0 | 57 |
| 333 | Dolores | 0 | 45 |
| 334 | San Luis | 0 | 56 |
| 335 | Sayaxché | 0 | 42 |
| 336 | Melchor de Mencos | 0 | 33 |
| 337 | Poptún | 0 | 40 |
| 338 | Las Cruces | 0 | 58 |
| 339 | El Chal | 0 | 39 |

**Álgebra Relacional** $\pi$ MUNI, C_des, C_hec ( Municipio $\bowtie$ $\gamma$ MUNI; SUM(CASOS) (Reg_Desnut) $\bowtie$ $\gamma$ MUNI; COUNT(*) (Hecho_Delictivo) )

**29. Violencia estructural Urbana vs Rural**

**SQL**

SELECT AG.NOMBRE_AREA, COUNT(HD.ID_HECHO_DISC) AS TOTAL_DISC

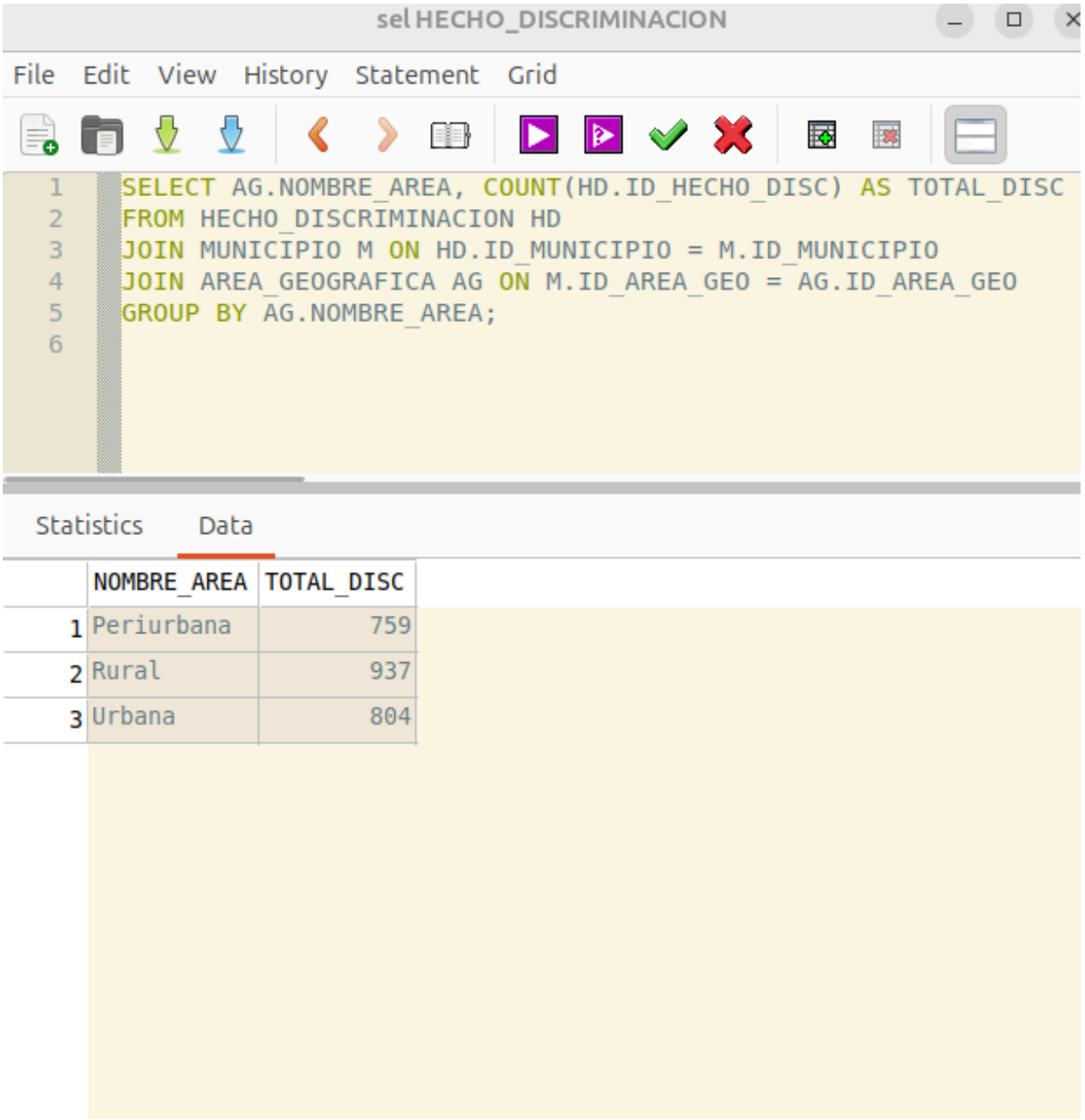
FROM HECHO_DISCRIMINACION HD

JOIN MUNICIPIO M ON HD.ID_MUNICIPIO = M.ID_MUNICIPIO

JOIN AREA_GEOGRAFICA AG ON M.ID_AREA_GEO = AG.ID_AREA_GEO

GROUP BY AG.NOMBRE_AREA;

**Álgebra Relacional** $\gamma$ AREA; COUNT(ID_HECHO_DISC) $\rightarrow$ T ( Hecho_Disc $\bowtie$ Municipio $\bowtie$ Area_Geo )




**30. Exhumaciones por tipo de delito**

**SQL**

```
SELECT TH.NOMBRE_TIPO AS DELITO, COUNT(E.ID_EXHUMACION) AS TOTAL

FROM EXHUMACION E

JOIN NECROPSIA N ON E.ID_NECROPSIA = N.ID_NECROPSIA

JOIN HECHO_DELICTIVO H ON N.ID_HECHO = H.ID_HECHO

JOIN TIPO_HECHO_DELICTIVO TH ON H.ID_TIPO_HECHO = TH.ID_TIPO_HECHO

GROUP BY TH.NOMBRE_TIPO;
```

**Álgebra Relacional** $\gamma$ NOMBRE_TIPO; COUNT(ID_EXHUMACION) $\rightarrow$ T ( Exhumacion $\bowtie$ Necropsia $\bowtie$ Hecho_Delictivo $\bowtie$ Tipo_Hecho )

sel EXHUMACION

File Edit View History Statement Grid

```
SELECT TH.NOMBRE_TIPO AS DELITO, COUNT(E.ID_EXHUMACION) AS TO
FROM EXHUMACION E
JOIN NECROPSIA N ON E.ID_NECROPSIA = N.ID_NECROPSIA
JOIN HECHO_DELICTIVO H ON N.ID_HECHO = H.ID_HECHO
JOIN TIPO_HECHO_DELICTIVO TH ON H.ID_TIPO_HECHO = TH.ID_TIPO_
GROUP BY TH.NOMBRE_TIPO;
```

Statistics Data

| DELITO | TOTAL |
|---|---|

## Conclusión:

El desarrollo del modelo de base de datos permitió estructurar de manera organizada la información relacionada con personas, denuncian, hechos delictivos, contexto geográfico y variables de salud. A través del uso del modelo entidad–relación de Peter Chen, diagrama de tablas, se logró identificar las entidades principales, sus atributos y las relaciones existentes entre ellas, garantizando una representación clara y coherente del sistema.

La implementación del esquema relacional en SQL, junto con la normalización de los datos, contribuye a evitar redundancias, mejorar la integridad de la información y facilitar futuras consultas y análisis. Asimismo, la inclusión de diferentes áreas como justicia, violencia y salud proporciona una visión integral que puede ser utilizada para la toma de decisiones y el análisis estadístico.

En conclusión, el sistema diseñado no solo cumple con los requerimientos técnicos de almacenamiento de datos, sino que también constituye una herramienta sólida para el análisis de información social y delictiva, permitiendo identificar patrones, tendencias y posibles áreas de intervención.

**Bibliografía**


- https://www.ine.gob.gt/sistema/uploads/2014/01/13/M2mwsNGpouJ8sjK4nuDqiu8qMFLkgBXX.pdf
- https://www.inegi.org.mx/rnm/index.php/catalog/919/study-description
- https://osarguatemala.org/embarazo/datos/


**GitHub:**

https://github.com/MelanniTzul/Proyecto2-Base-de-datos-1.git